

本节内容

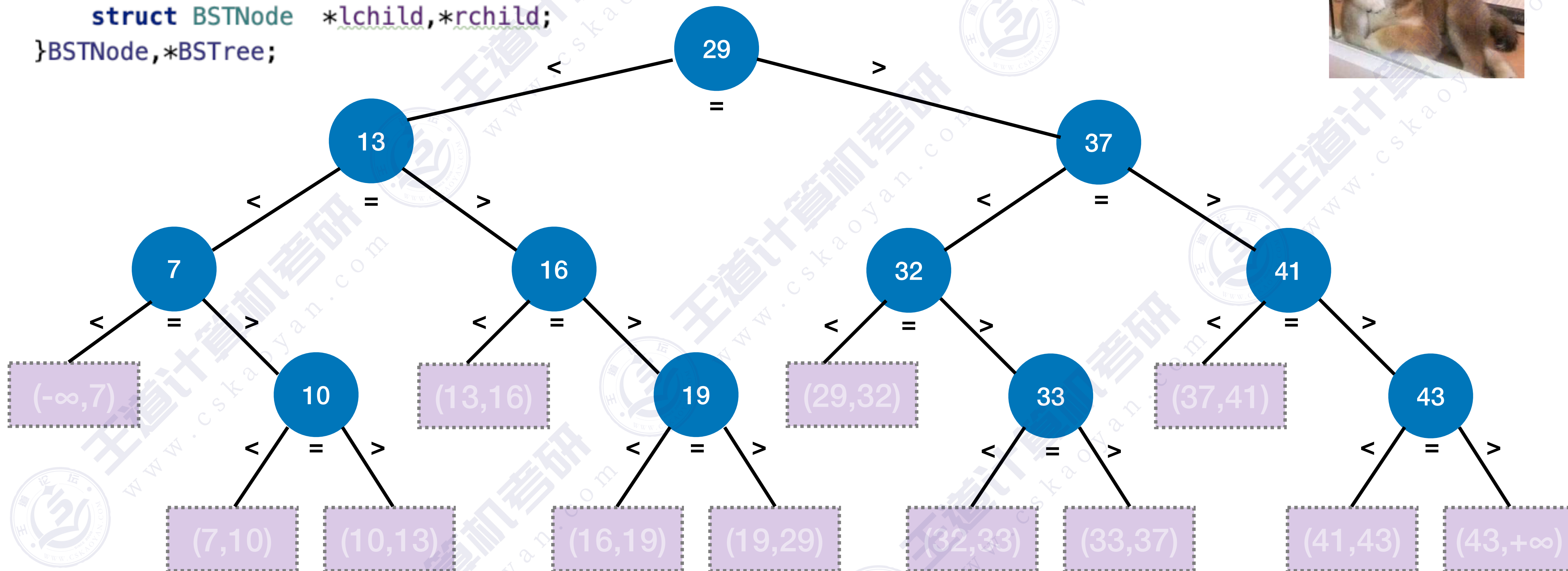
B树

回顾：二叉查找树 (BST)

// 二叉排序树结点

```
typedef struct BSTNode{  
    int key;  
    struct BSTNode *lchild,*rchild;  
}BSTNode,*BSTree;
```

能不能变成m
叉查找树?



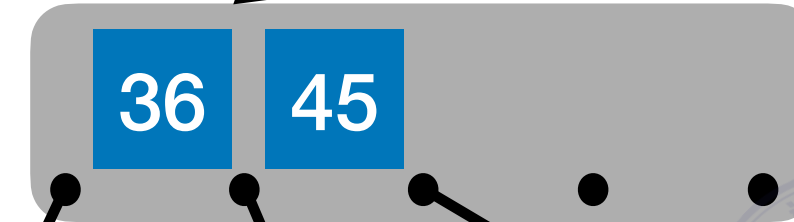
5叉查找树

//5叉排序树的结点定义

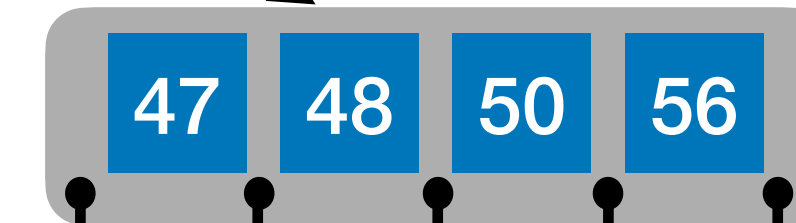
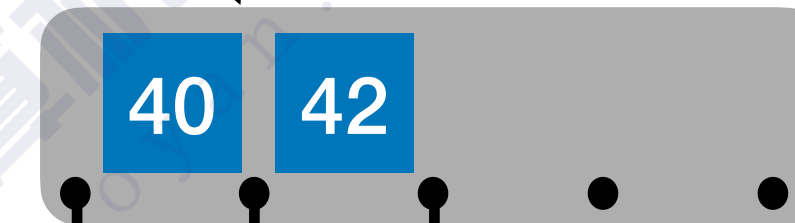
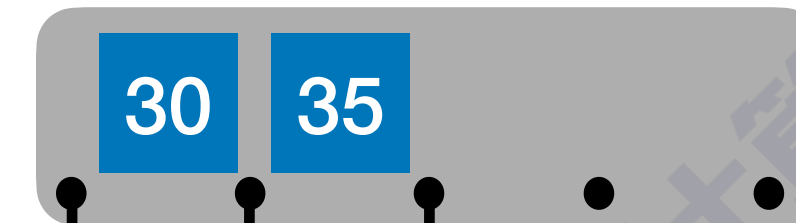
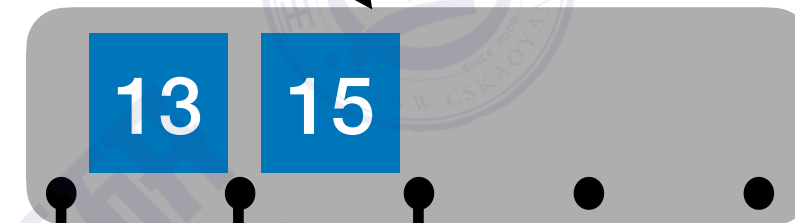
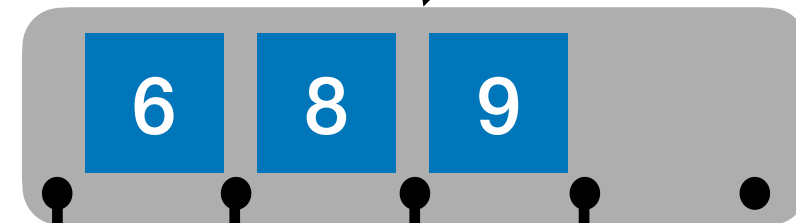
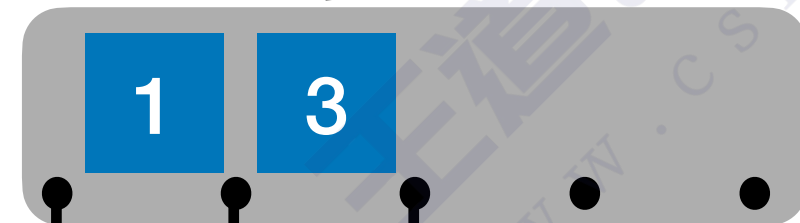
```
struct Node {  
    ElemType keys[4];           //最多4个关键字  
    struct Node * child[5];    //最多5个孩子  
    int num;                   //结点中有几个关键字  
};
```



最少1个关键字，2个分叉
最多4个关键字，5个分叉



结点内关键字有序

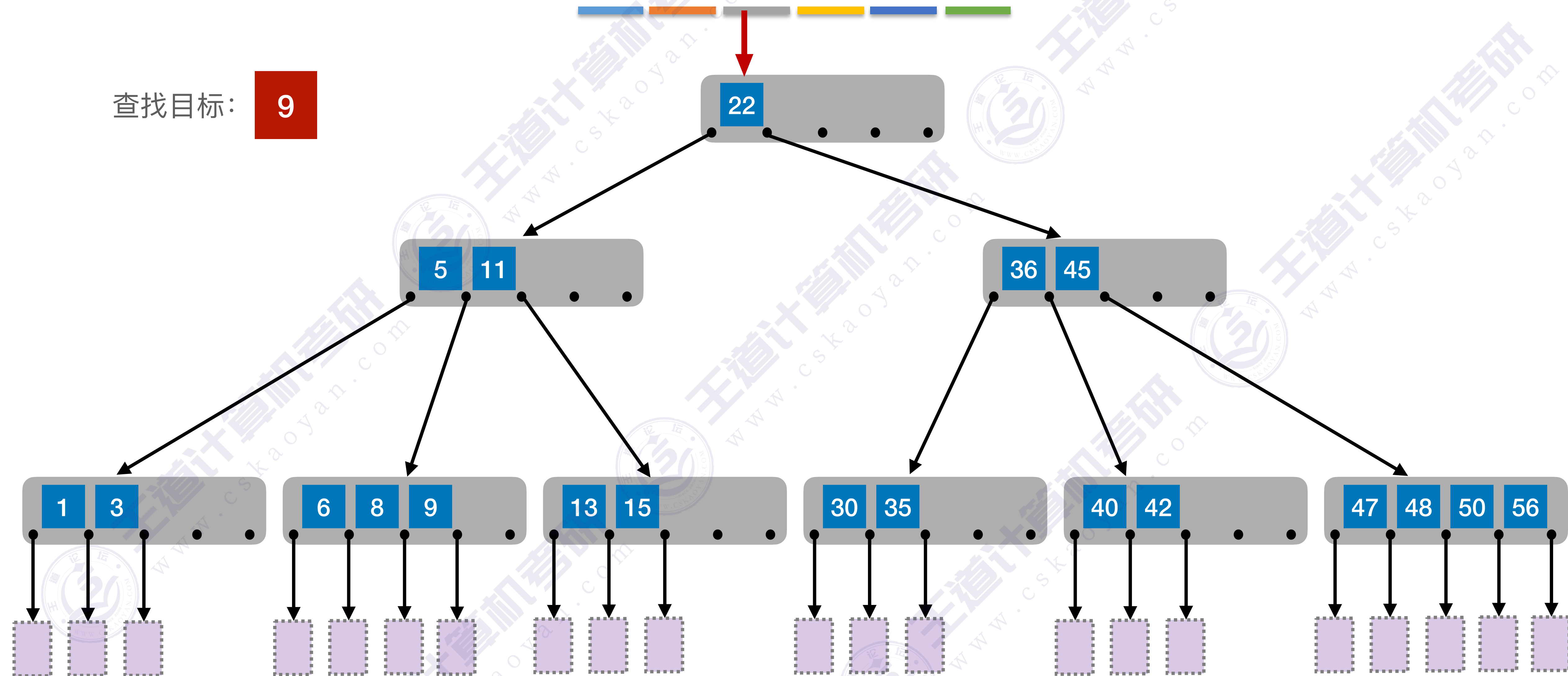


失败结点

如何查找

查找目标：

9

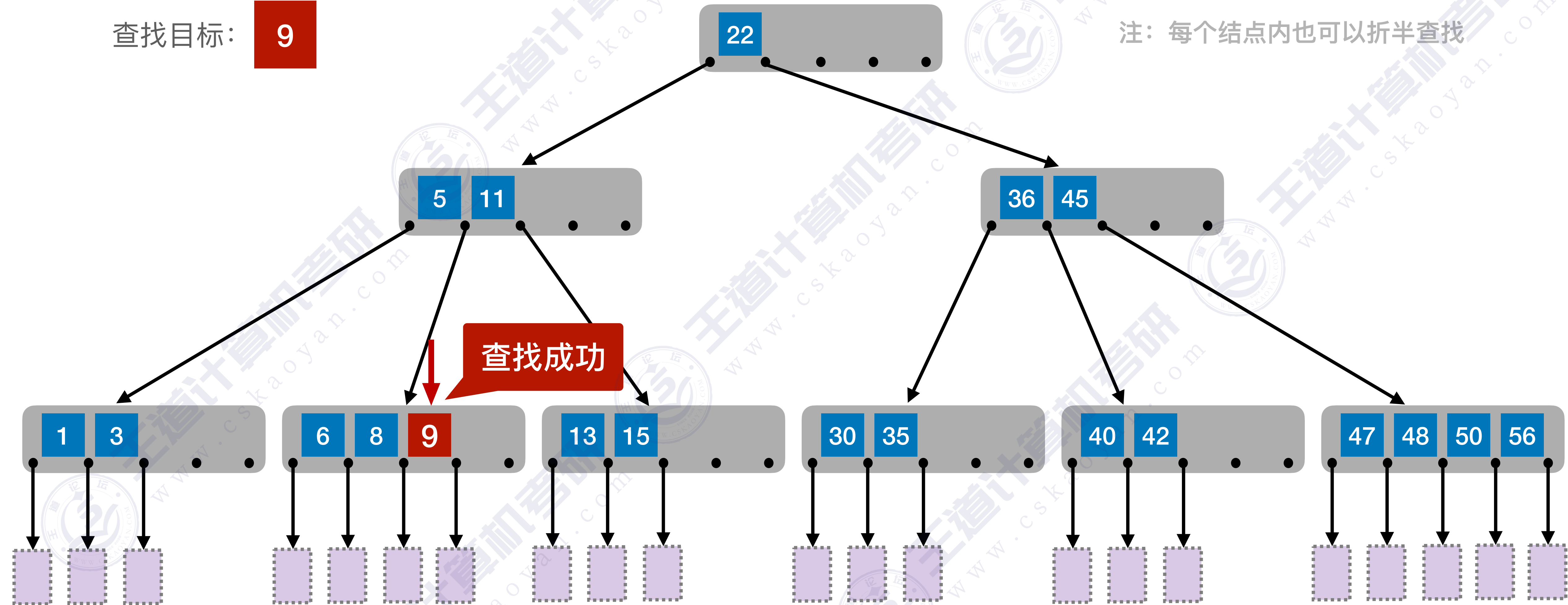


如何查找

查找目标：

9

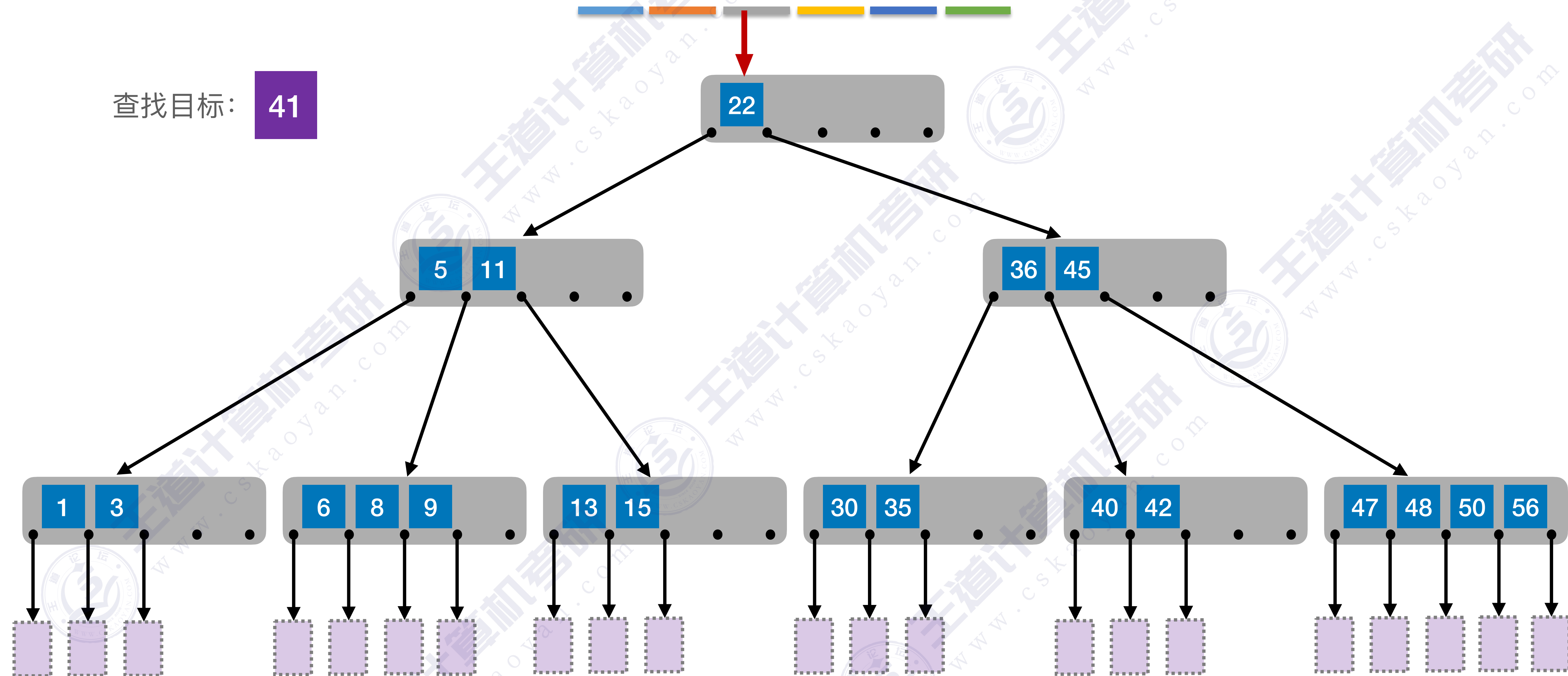
注：每个结点内也可以折半查找



如何查找

查找目标：

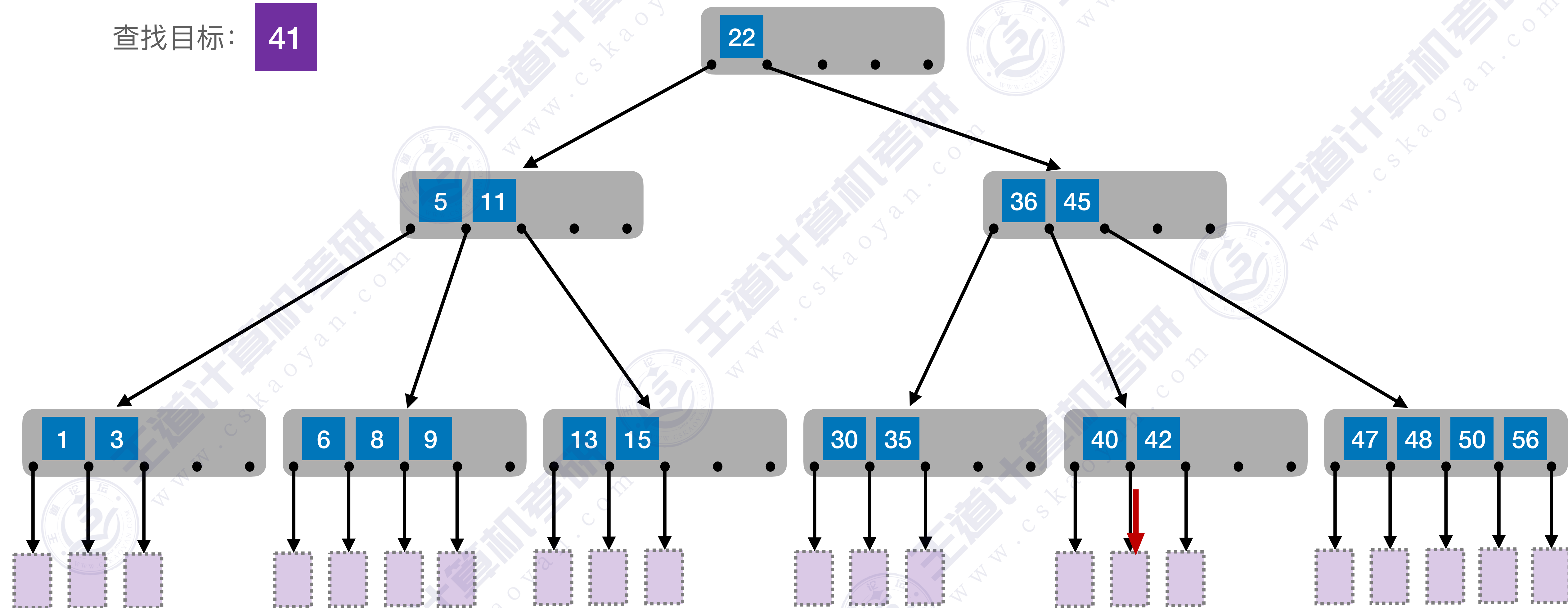
41



如何查找

查找目标：

41

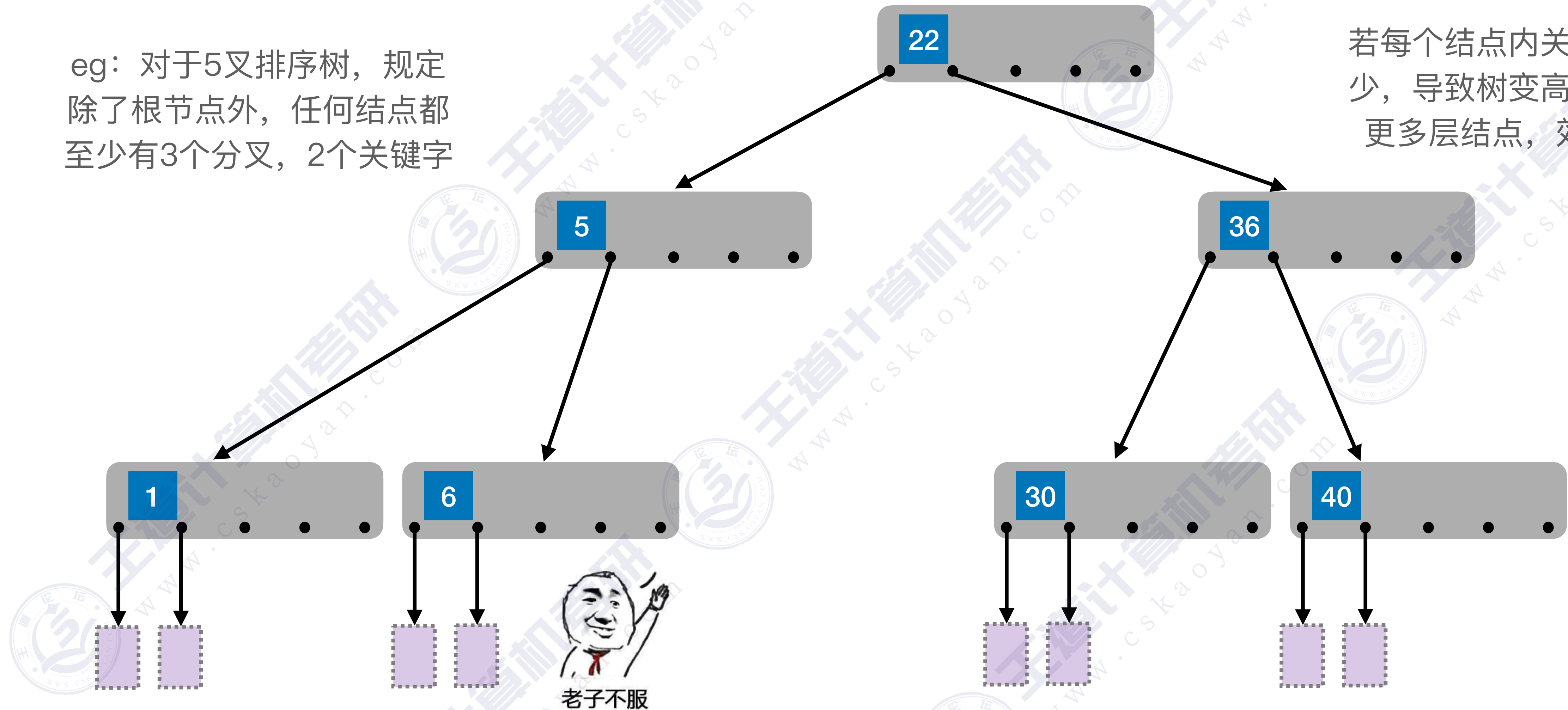


查找失败

如何保证查找效率

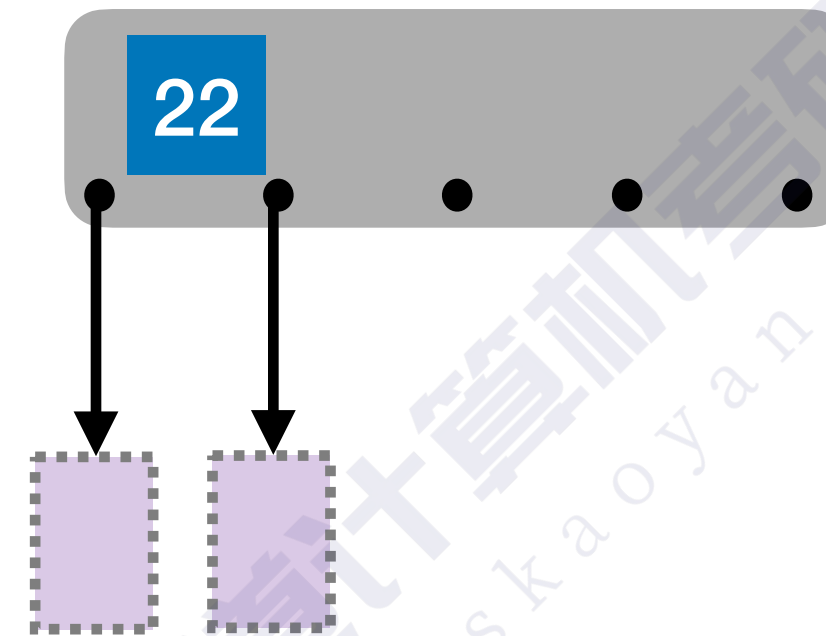
eg: 对于5叉排序树, 规定除了根节点外, 任何结点都至少有3个分叉, 2个关键字

若每个结点内关键字太少, 导致树变高, 要查更多层结点, 效率低



策略: m 叉查找树中, 规定除了根节点外, 任何结点至少有 $\lceil m/2 \rceil$ 个分叉, 即至少含有 $\lceil m/2 \rceil - 1$ 个关键字

如何保证查找效率



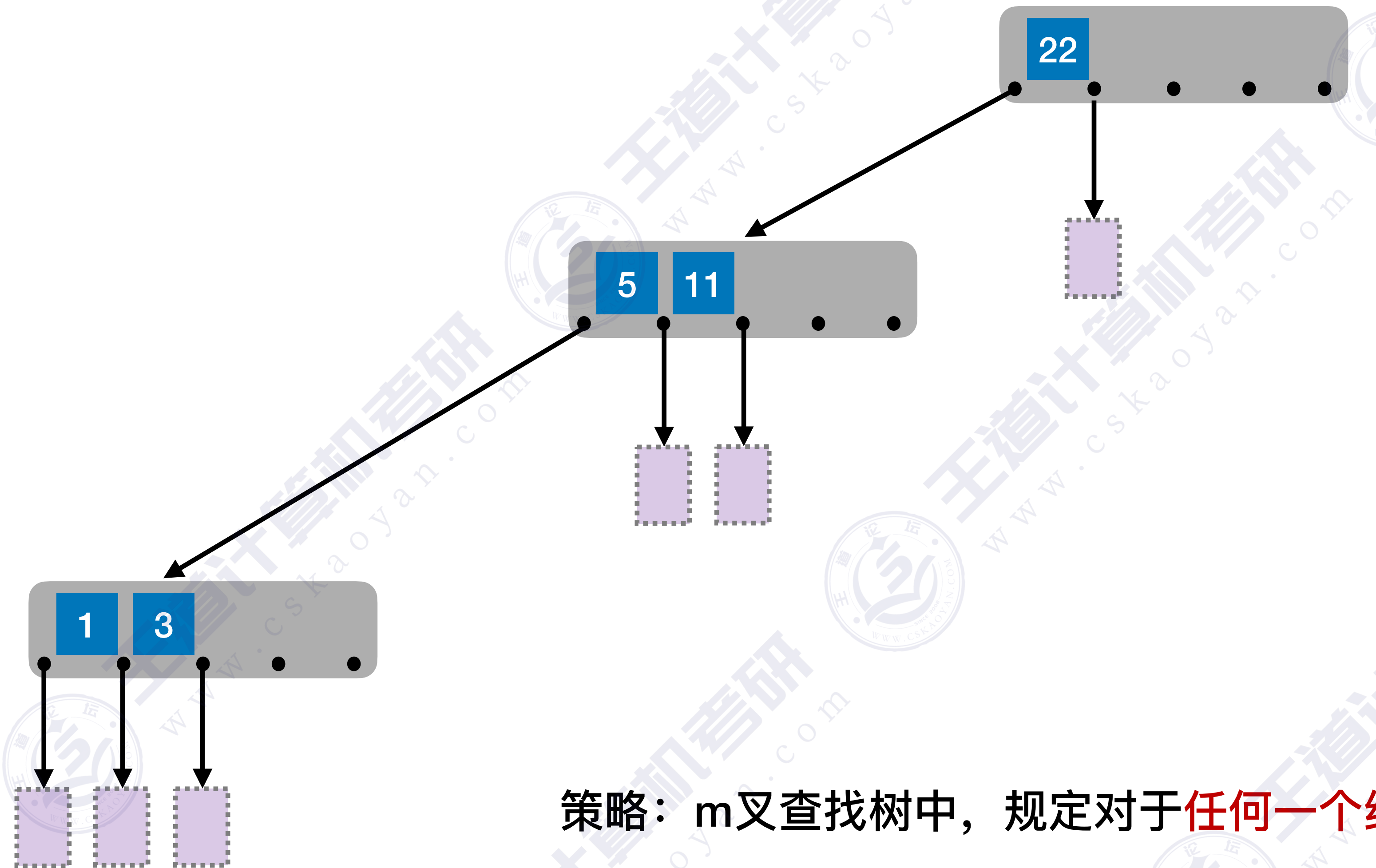
这个我也没有 没办法



如果整个树只有1个元素，根节点只有两个分叉

策略：m叉查找树中，规定除了根节点外，任何结点至少有 $\lceil m/2 \rceil$ 个分叉，即至少含有 $\lceil m/2 \rceil - 1$ 个关键字

如何保证查找效率

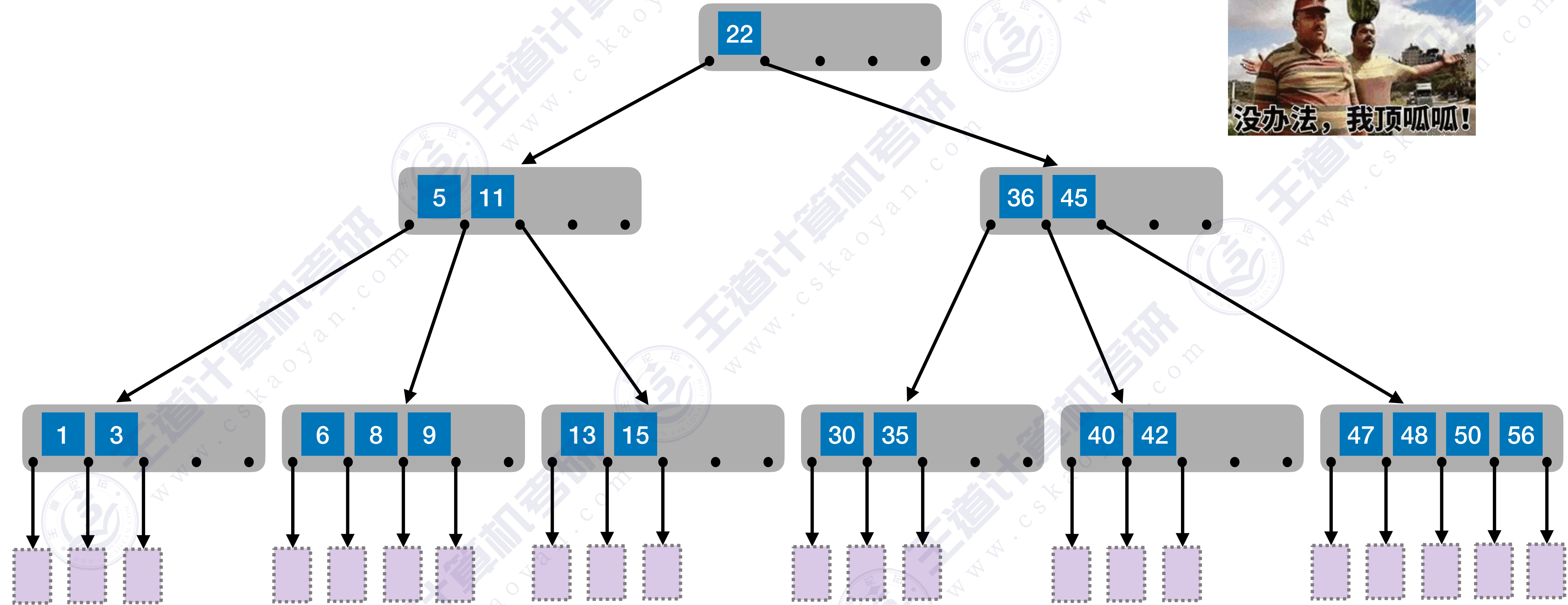


还不够

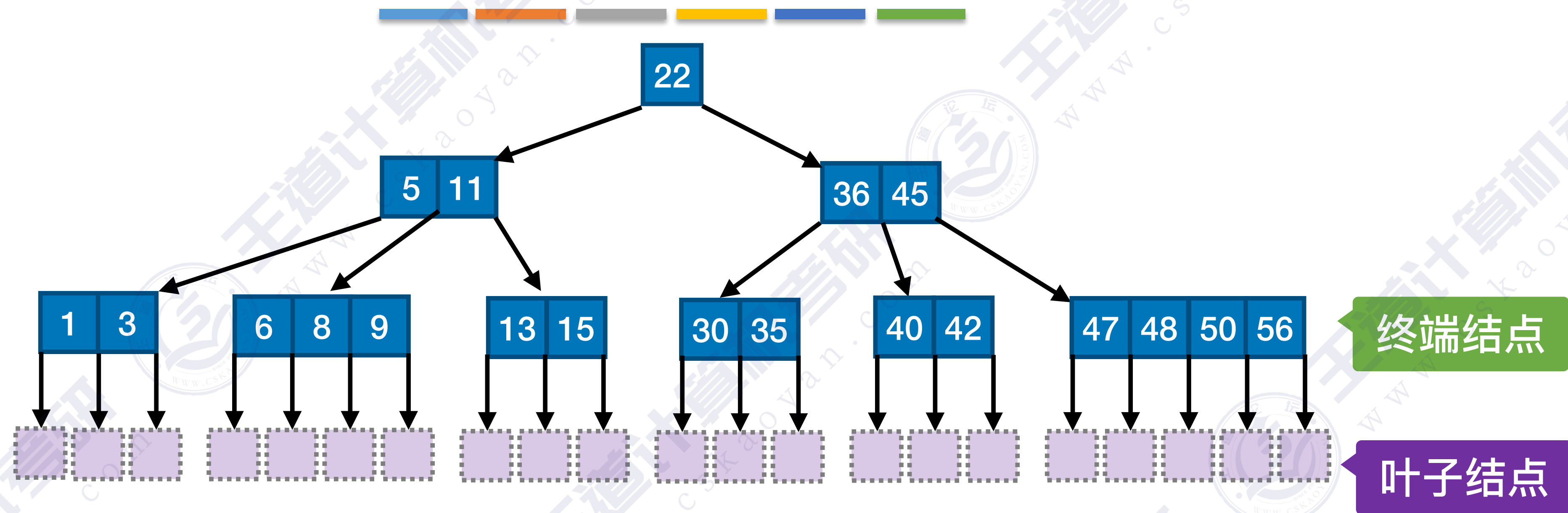
不够“平衡”，树会很高，要查很多层结点

策略：m叉查找树中，规定对于**任何一个结点**，其所有子树的高度都要相同。

B树



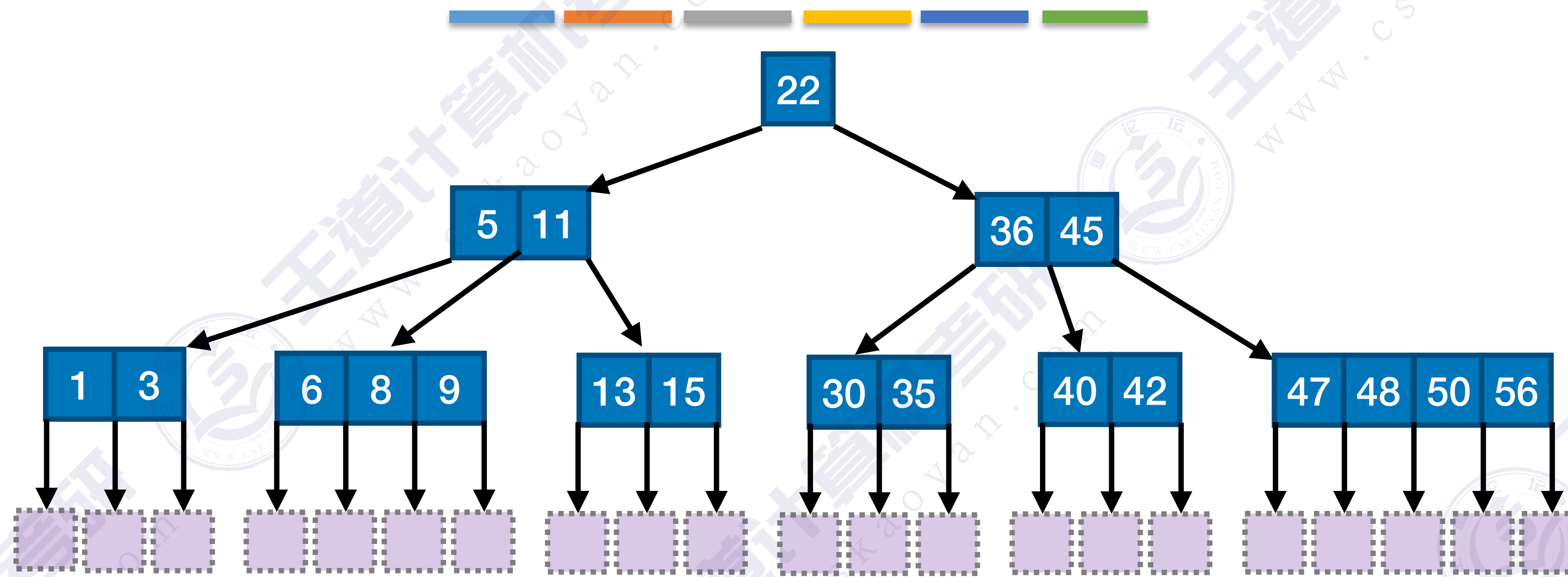
B树



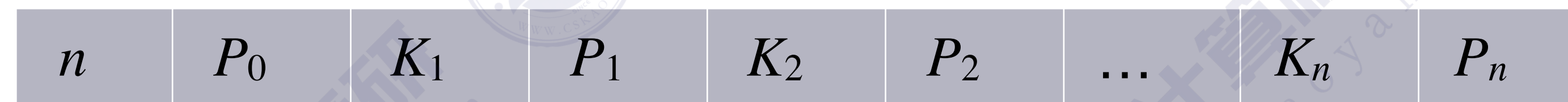
B树，又称**多路平衡查找树**，B树中所被允许的孩子个数的最大值称为B树的阶，通常用 m 表示。一棵 **m 阶B树**或为空树，或为满足如下特性的 m 叉树：

- 1) 树中每个结点至多有 m 棵子树，即至多含有 $m-1$ 个关键字。
- 2) 若根结点不是终端结点，则至少有两棵子树。
- 3) **除根结点外的所有非叶结点至少有 $\lceil m/2 \rceil$ 棵子树**，即至少含有 $\lceil m/2 \rceil - 1$ 个关键字。
- 5) 所有的**叶结点**都出现在同一层次上，并且不带信息（可以视为外部结点或类似于折半查找判定树的查找失败结点，实际上这些结点不存在，指向这些结点的指针为空）。

B树



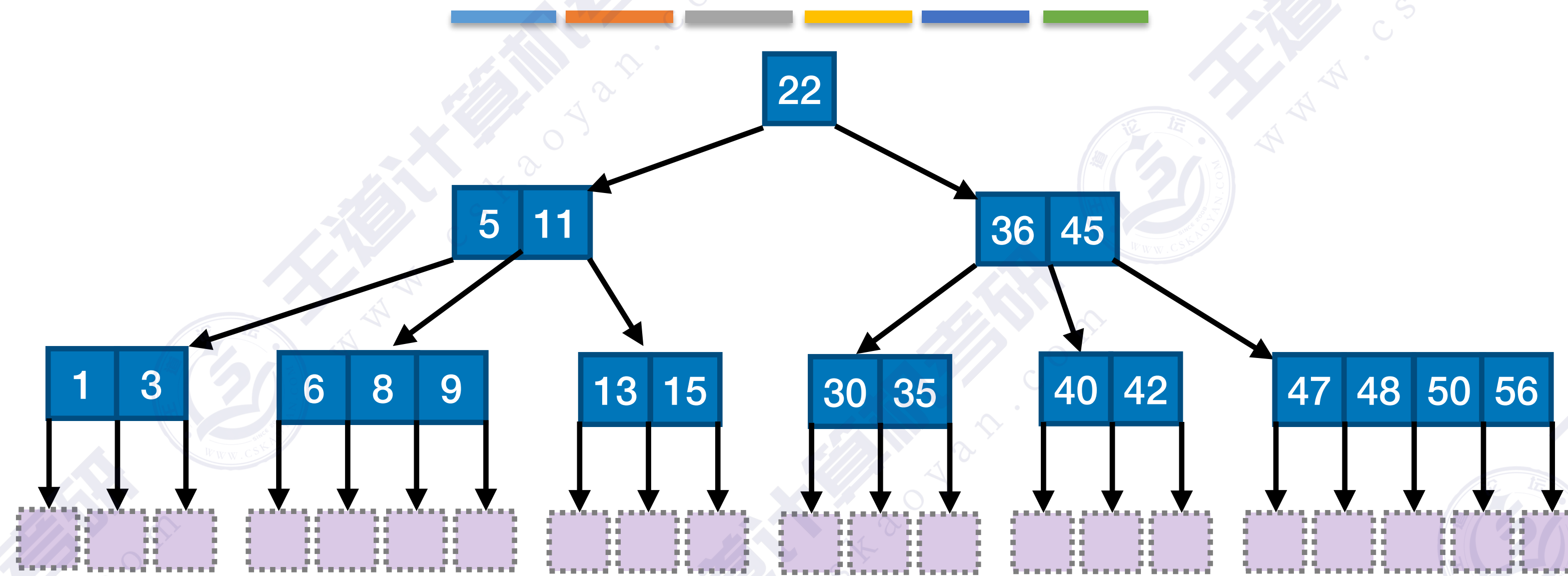
4) 所有非叶结点的结构如下:



其中, K_i ($i = 1, 2, \dots, n$) 为结点的关键字, 且满足 $K_1 < K_2 < \dots < K_n$; P_i ($i = 0, 1, \dots, n$) 为指向子树根结点的指针, 且指针 P_{i-1} 所指子树中所有结点的关键字均小于 K_i , P_i 所指子树中所有结点的关键字均大于 K_i , n

($\lceil m/2 \rceil - 1 \leq n \leq m - 1$) 为结点中关键字的个数。

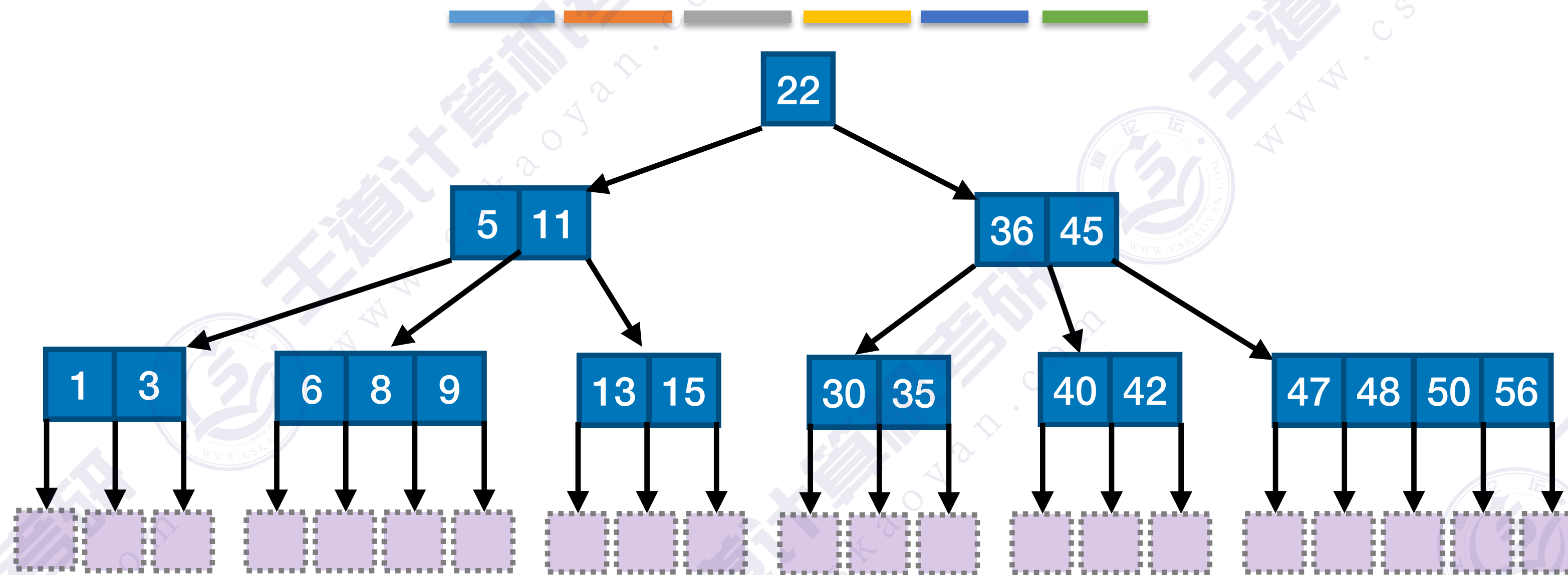
B树



m阶B树的核心特性:

- 1) 根节点的子树数 $\in [2, m]$, 关键字数 $\in [1, m-1]$ 。
其他结点的子树数 $\in [\lceil m/2 \rceil, m]$; 关键字数 $\in [\lceil m/2 \rceil - 1, m-1]$
- 2) 对任一结点, 其所有子树高度都相同
- 3) 关键字的值: 子树0 < 关键字1 < 子树1 < 关键字2 < 子树2 < (类比二叉查找树 左 < 中 < 右)

B树的高度

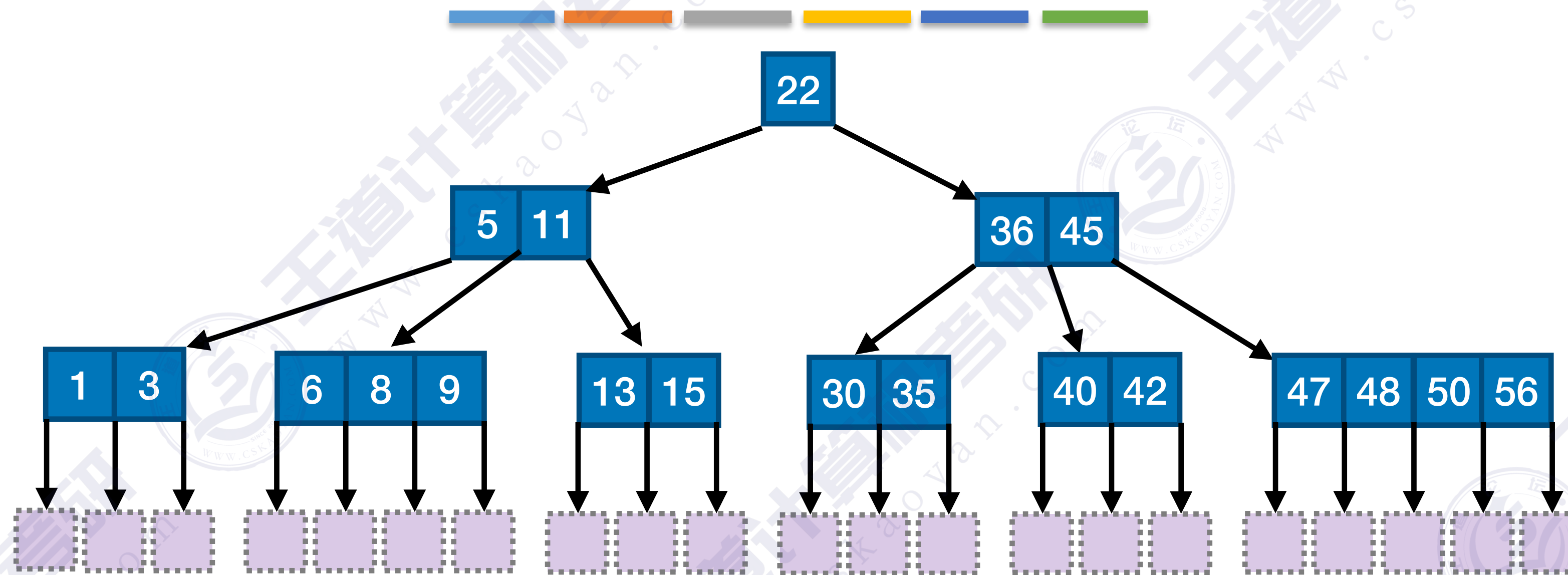


注：大部分学校算B树的**高度不包括叶子结点**（失败结点）

问题：含 n 个关键字的 m 阶B树，最小高度、最大高度是多少？

最小高度——让每个结点尽可能的满，有 $m-1$ 个关键字， m 个分叉，则有
$$n \leq (m-1)(1 + m + m^2 + m^3 + \dots + m^{h-1}) = m^h - 1,$$
 因此 $h \geq \log_m(n+1)$

B树的高度



注：大部分学校算B树的**高度不包括叶子结点（失败结点）**

最大高度——让各层的分叉尽可能的少，即根节点只有2个分叉，其他结点只有 $\lceil m/2 \rceil$ 个分叉
各层结点至少有：第一层 1、第二层 2、第三层 $2\lceil m/2 \rceil$... 第h层 $2(\lceil m/2 \rceil)^{h-2}$
第h+1层共有**叶子结点（失败结点）** $2(\lceil m/2 \rceil)^{h-1}$ 个

n个关键字的B树必有n+1个叶子结点，则 $n + 1 \geq 2(\lceil m/2 \rceil)^{h-1}$ ，即 $h \leq \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$

n个关键字将数域切分为n+1个区间

B树的高度

问题：含n个关键字的m叉B树，最小高度、最大高度是多少？

最大高度——让每个结点包含的关键字、分叉尽可能的少。记 $k=\lceil m/2 \rceil$

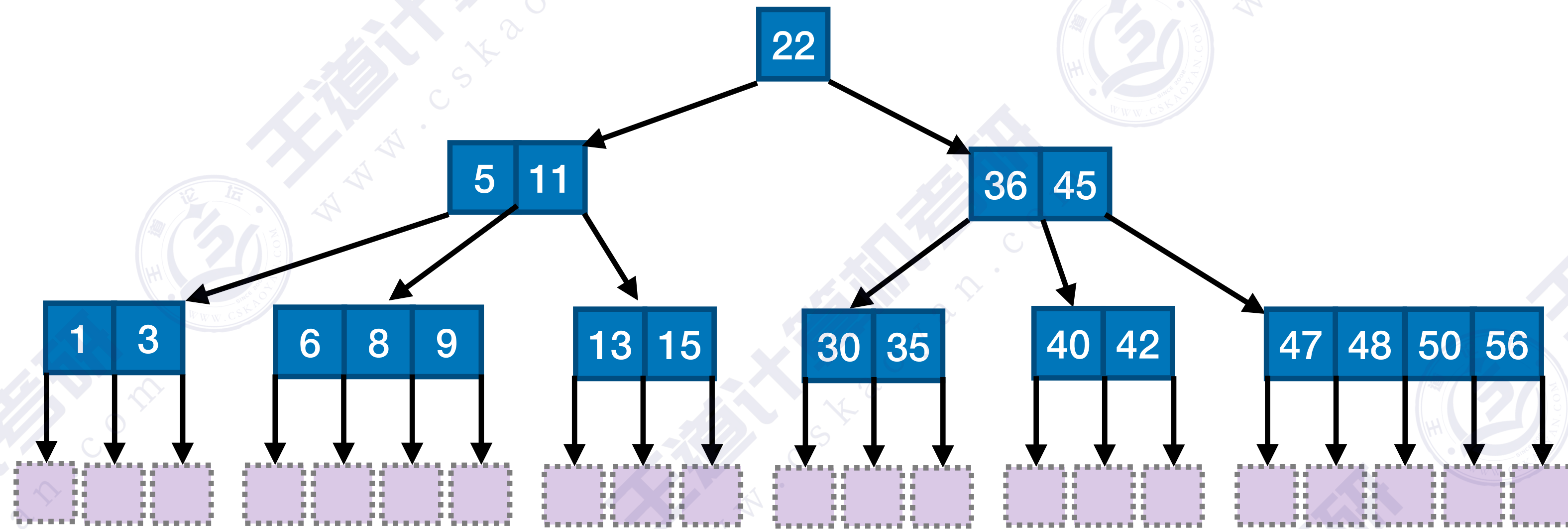
	最少结点数	最少关键字数
第一层	1	1
第二层	2	$2(k-1)$
第三层	$2k$	$2k(k-1)$
第四层	$2k^2$	$2k^2(k-1)$
...	...	
第h层	$2k^{h-2}$	$2k^{h-2}(k-1)$

h层的m阶B树至少包含关键字总数 $1+2(k-1)(k^0+k^1+k^2+\dots+k^{h-2}) = 1+2(k^{h-1}-1)$

若关键字总数少于这个值，则高度一定小于h，因此 $n \geq 1+2(k^{h-1}-1)$

得，
$$h \leq \log_k \frac{n+1}{2} + 1 = \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$$

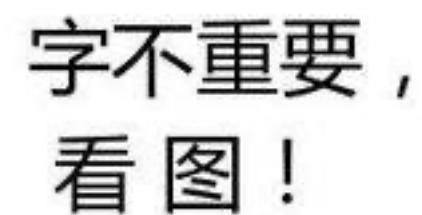
B树的高度



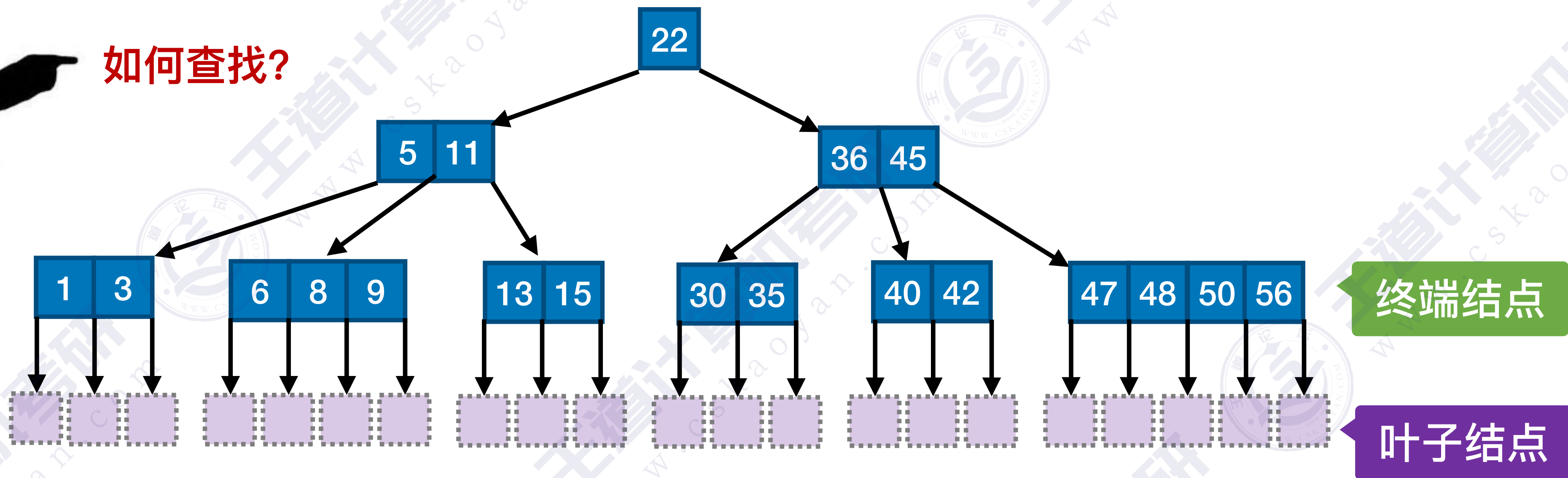
注：大部分学校算B树的**高度不包括叶子结点（失败结点）**

问题：含n个关键字的m阶B树，最小高度、最大高度是多少？

$$\log_m(n+1) \leq h \leq \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$$



如何查找？



m阶B树的核心特性:

尽可能“满”

1) 根节点的子树数 $\in[2, m]$, 关键字数 $\in[1, m-1]$ 。

其他结点的子树数 $\in [\lceil m/2 \rceil, m]$; 关键字数 $\in [\lceil m/2 \rceil - 1, m - 1]$

尽可能“平衡”

2) 对任一结点, 其所有子树高度都相同

3) 关键字的值: 子树0<关键字1<子树1<关键字2<子树2<.... (类比二叉查找树 左<中<右)

含n个关键字的m叉B树, $\log_m(n+1) \leq h \leq \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$

本节内容

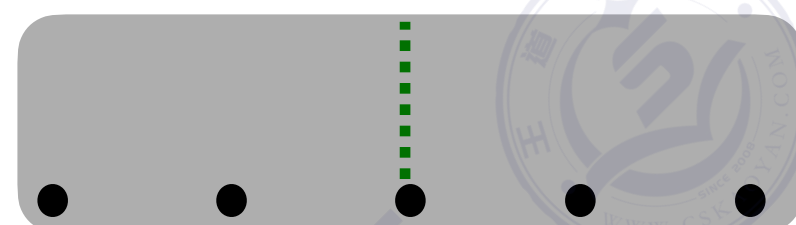
B树

插入和删除

B树的插入



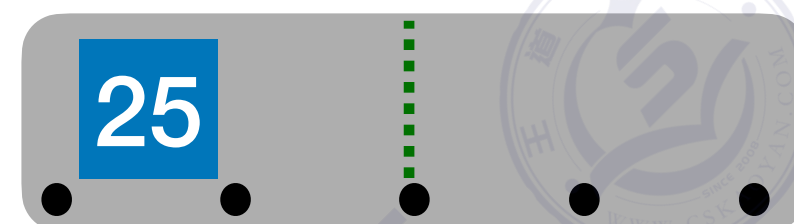
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



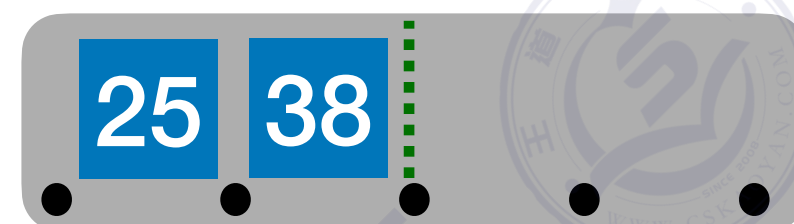
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



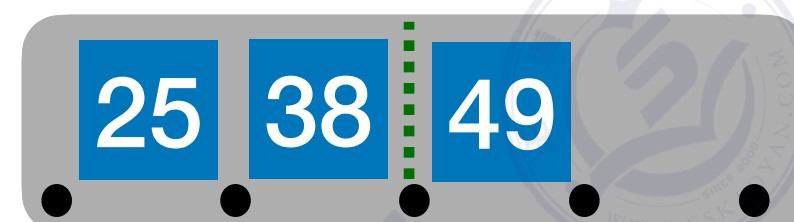
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



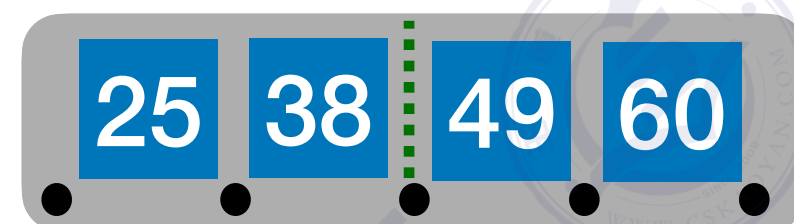
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



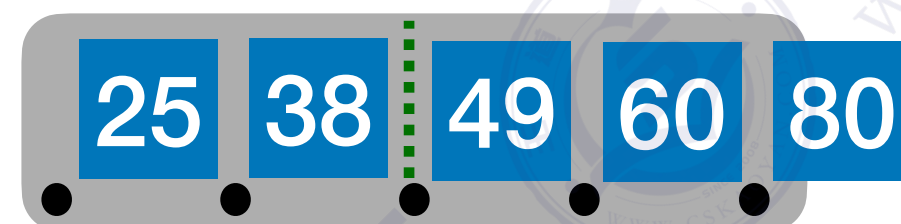
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



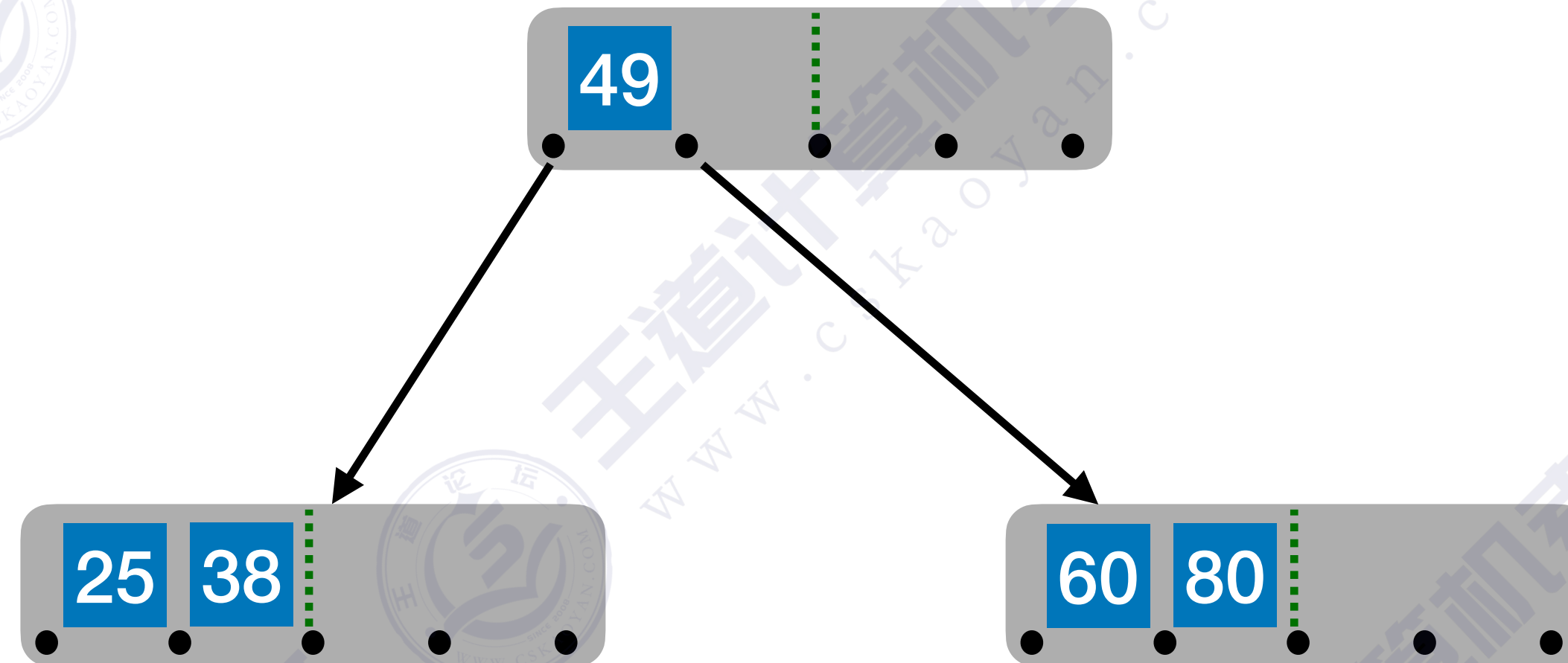
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

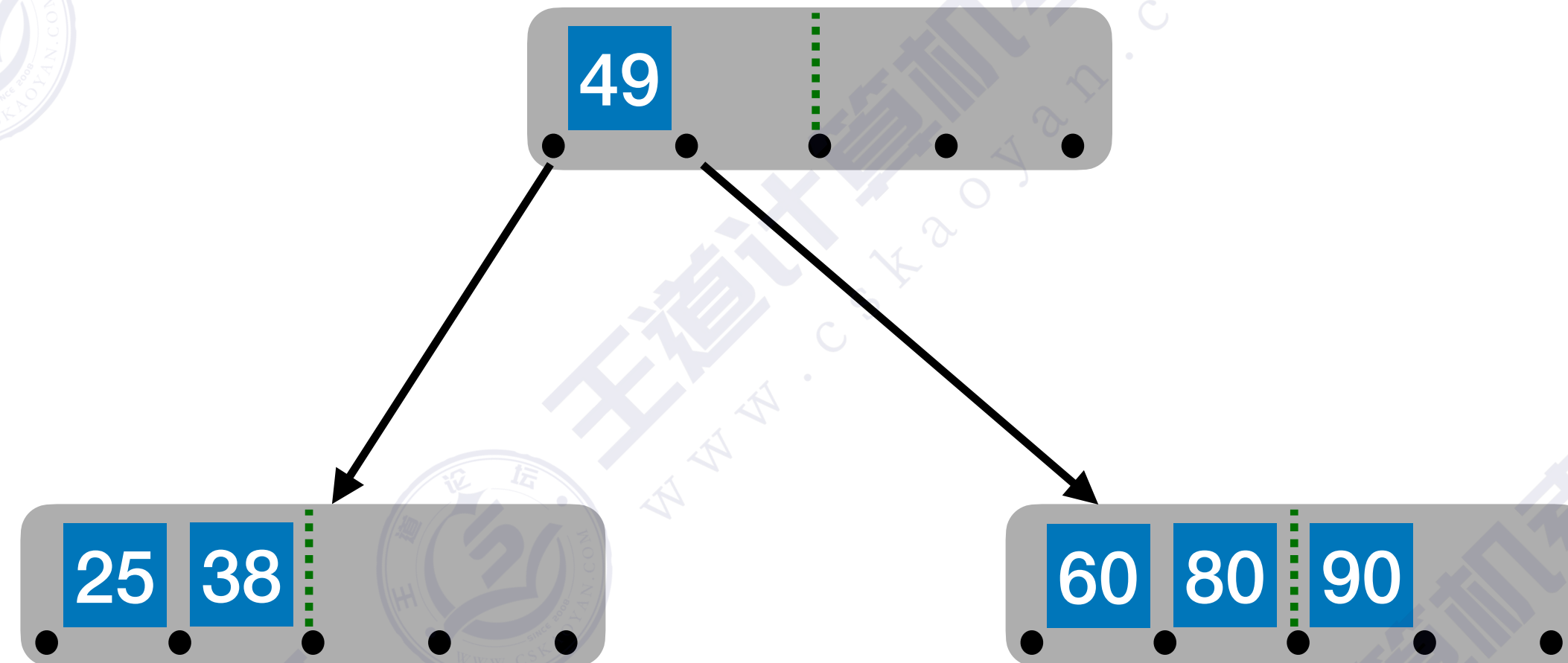


在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

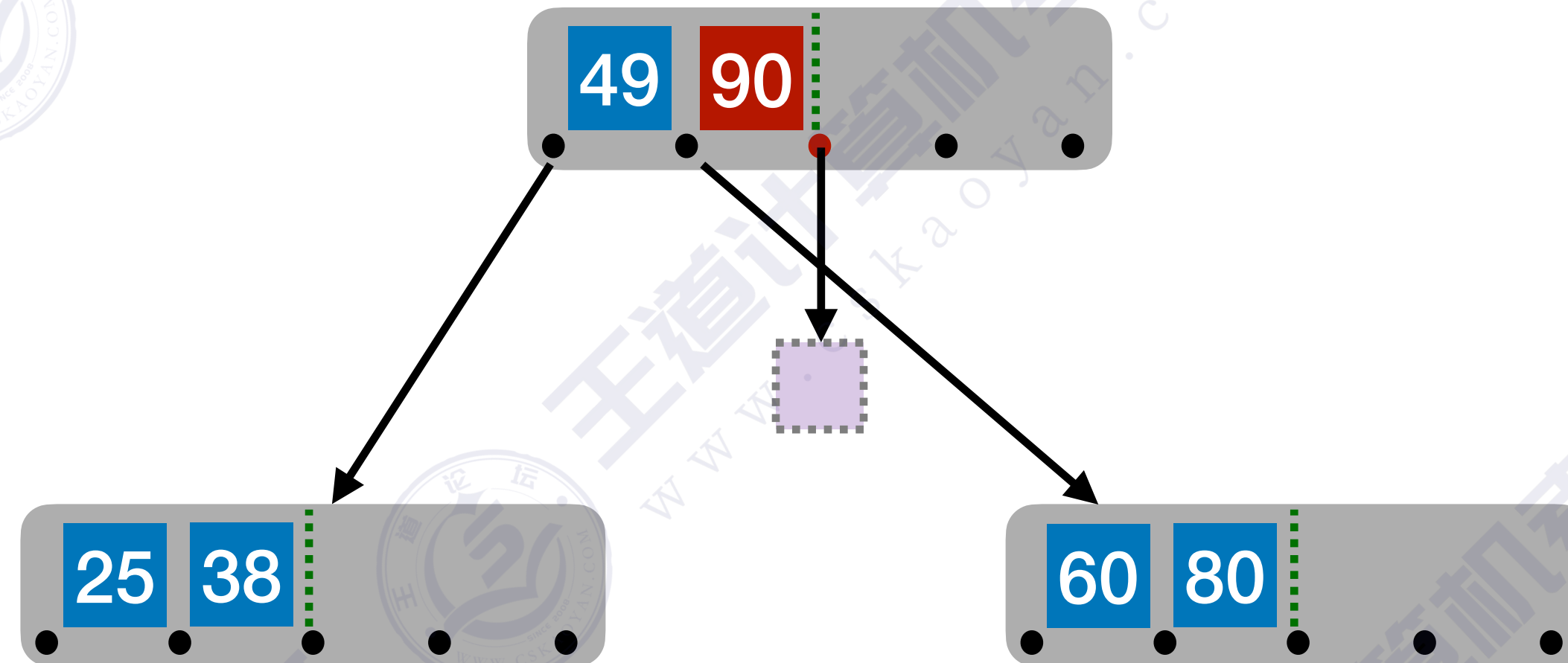


新元素一定是插入到最底层“终端节点”，用“查找”来确定插入位置

B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



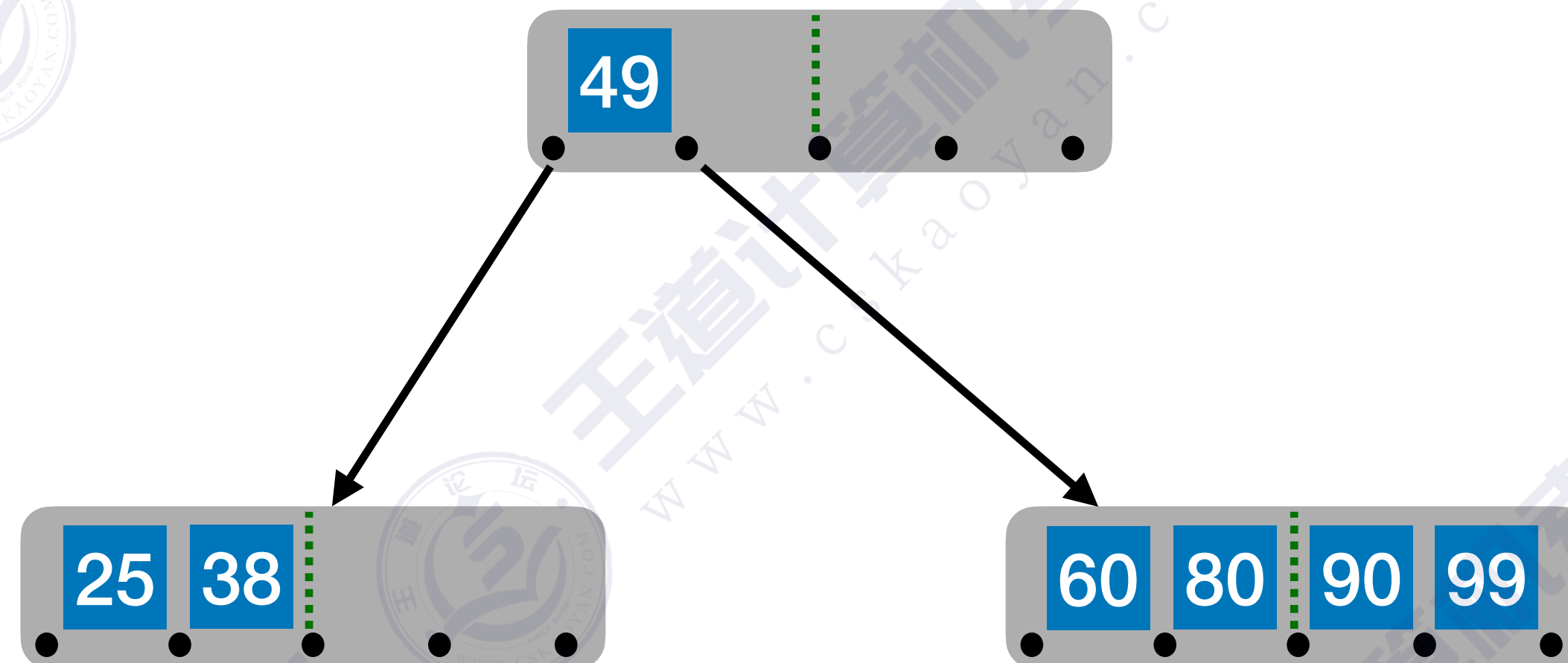
错误示范

注意：B树的失败结点只能出现在最下面一层

B树的插入



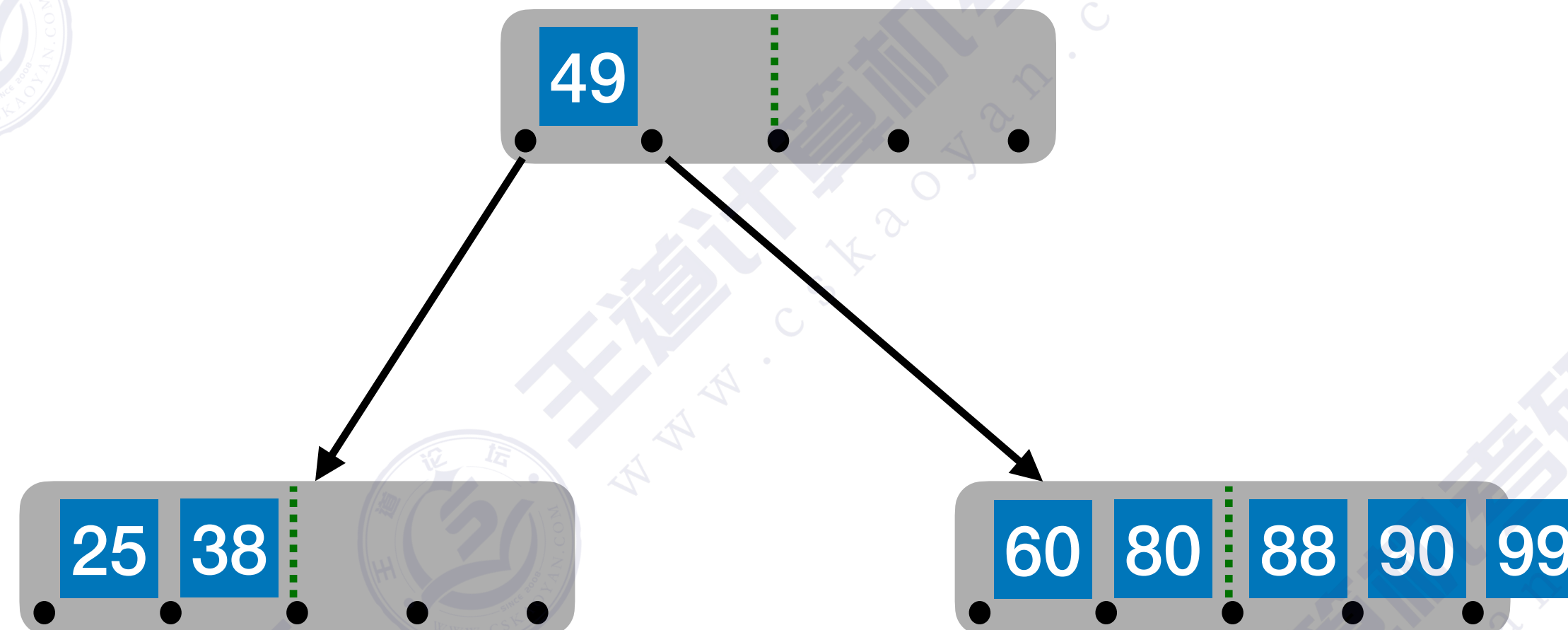
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

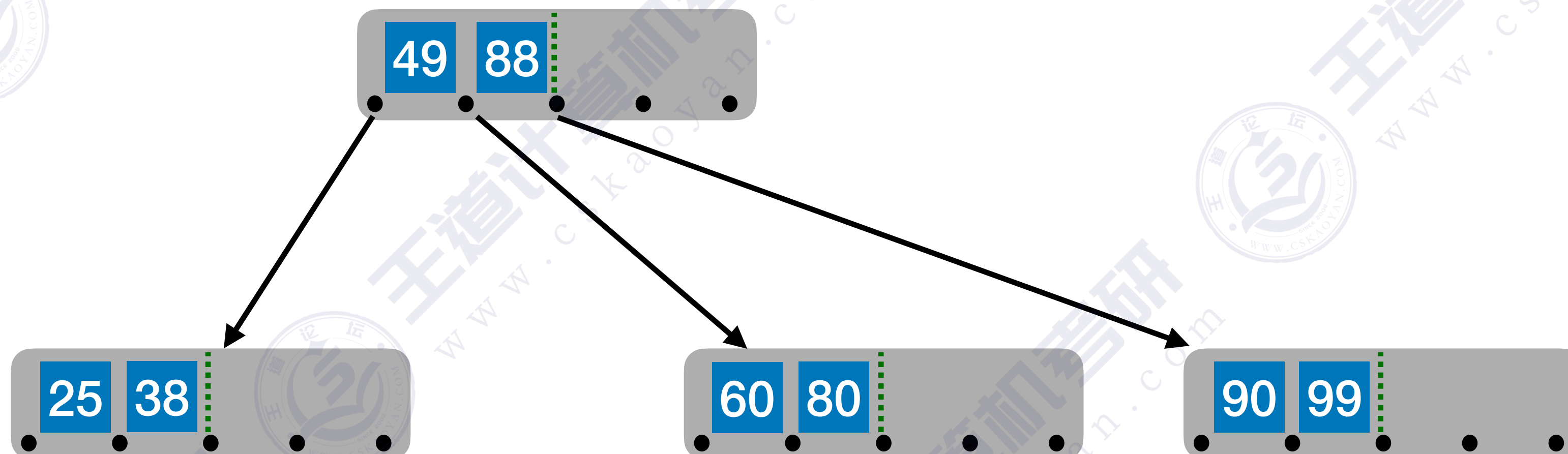


在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入



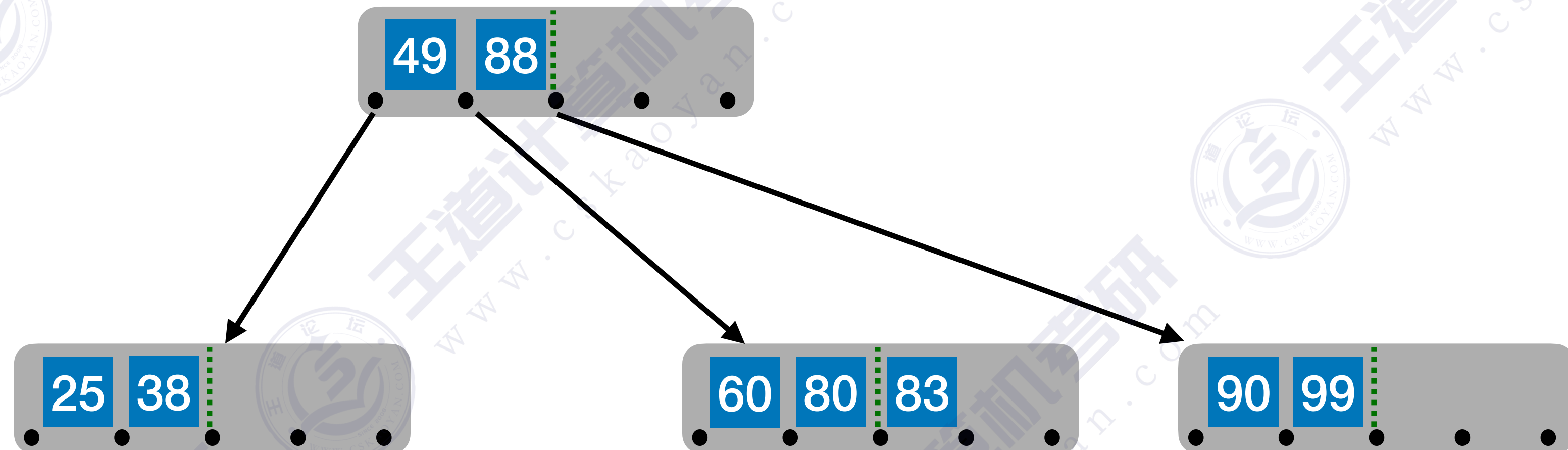
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



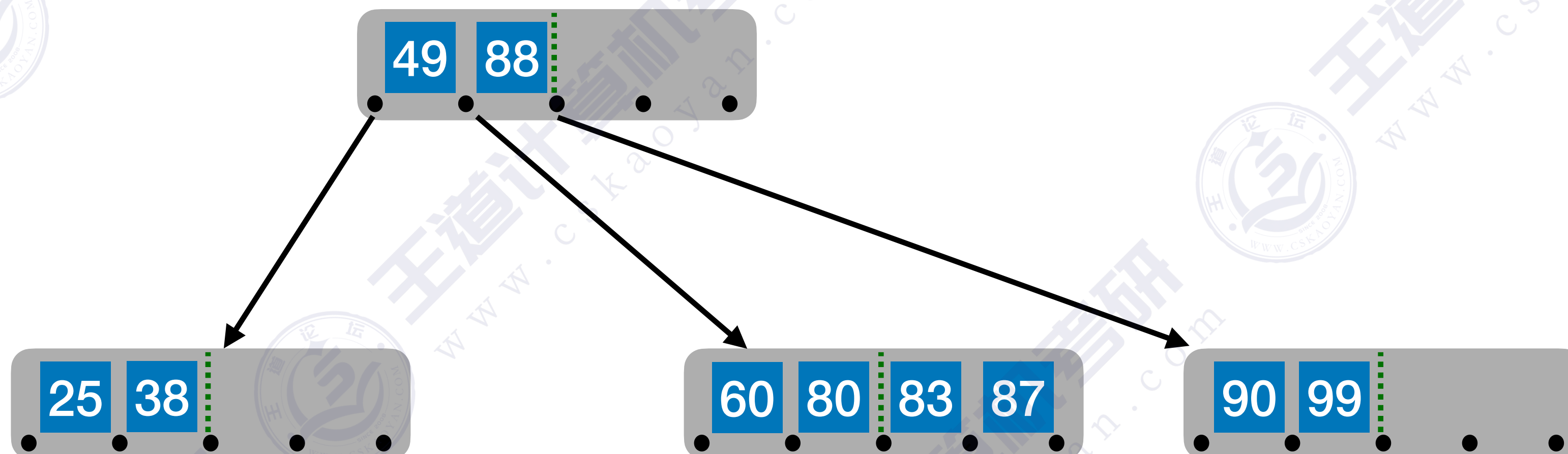
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

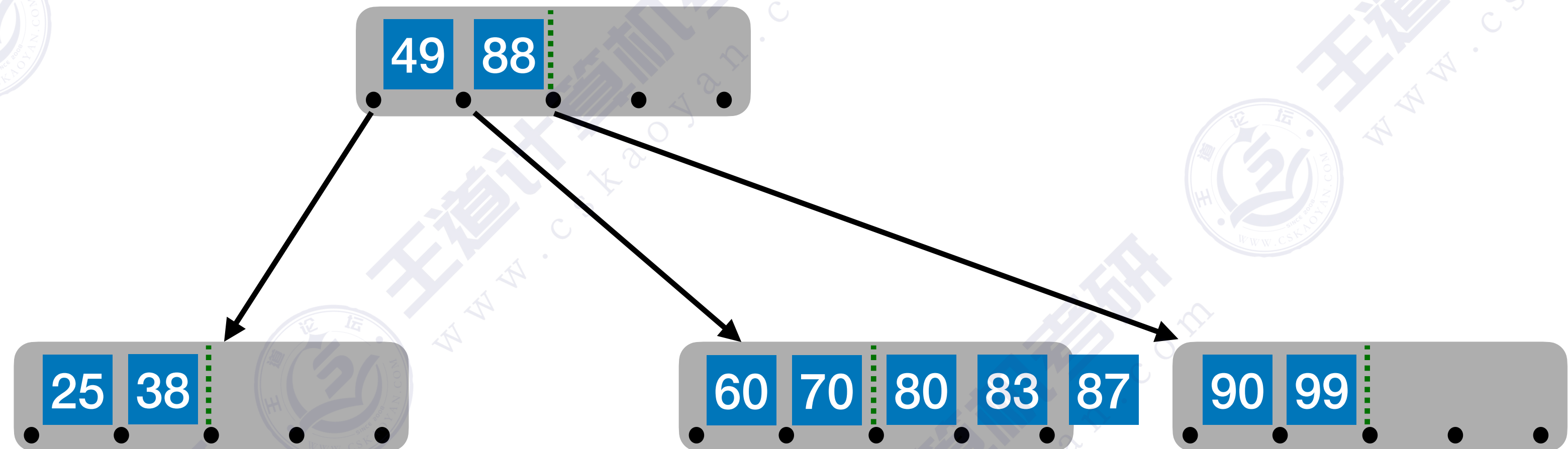


B树的插入



思考：80要放到父节点中，放在哪个位置合适？

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



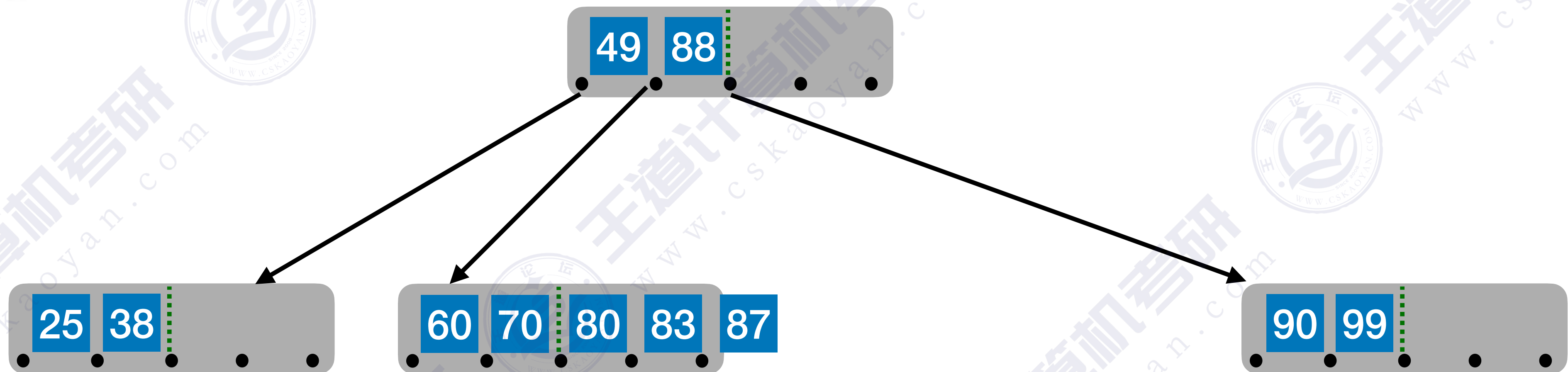
在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入



思考：80要放到父节点中，放在哪个位置合适？

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

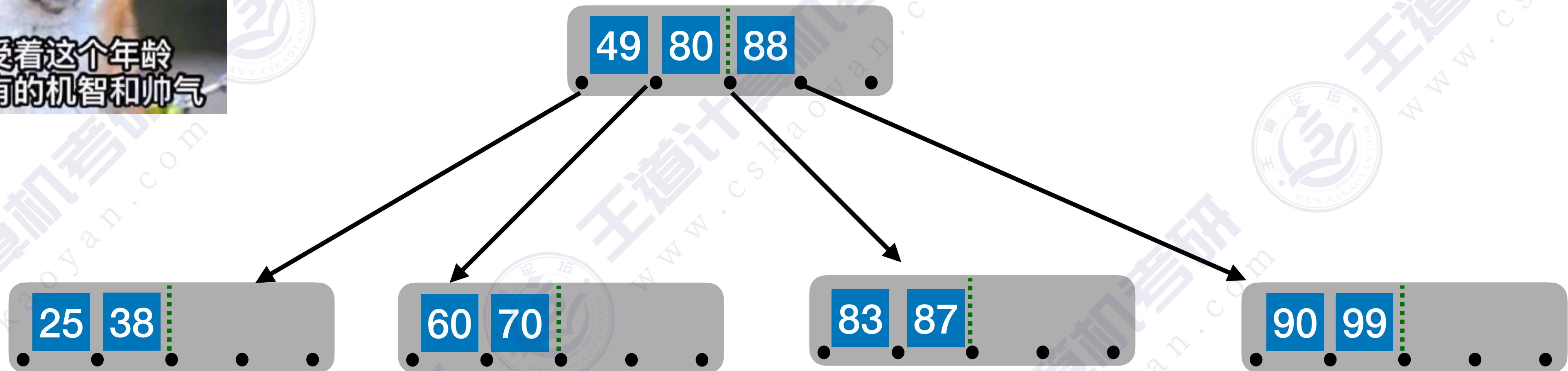


在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

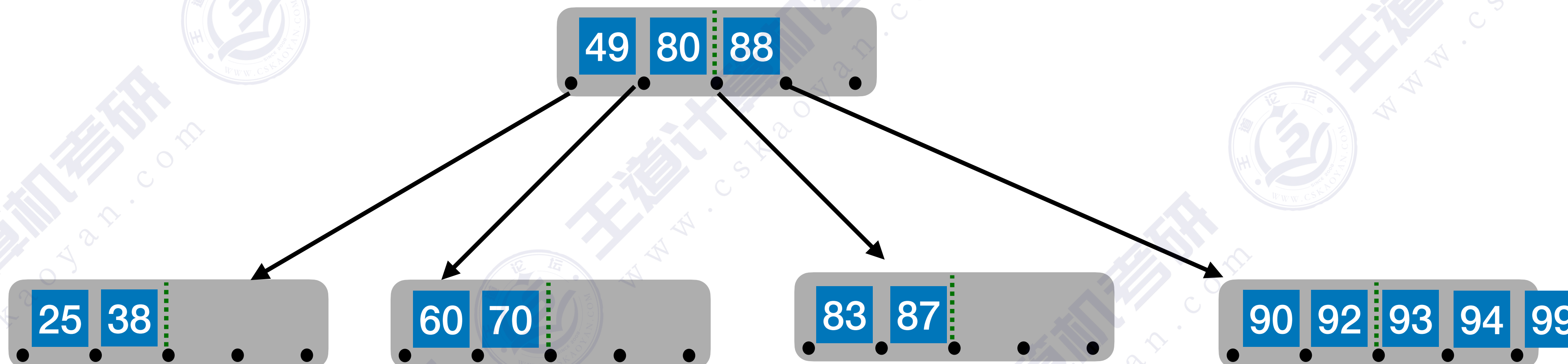


在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入



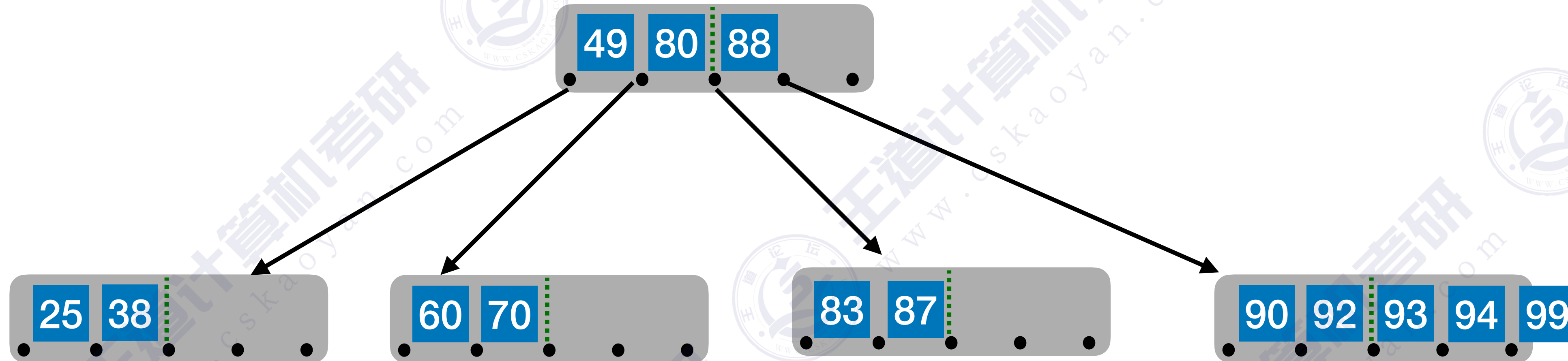
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

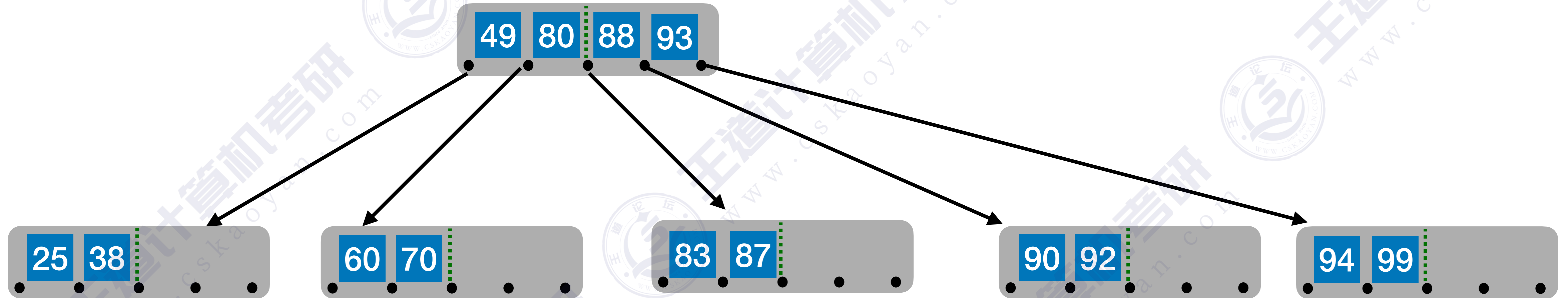


在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

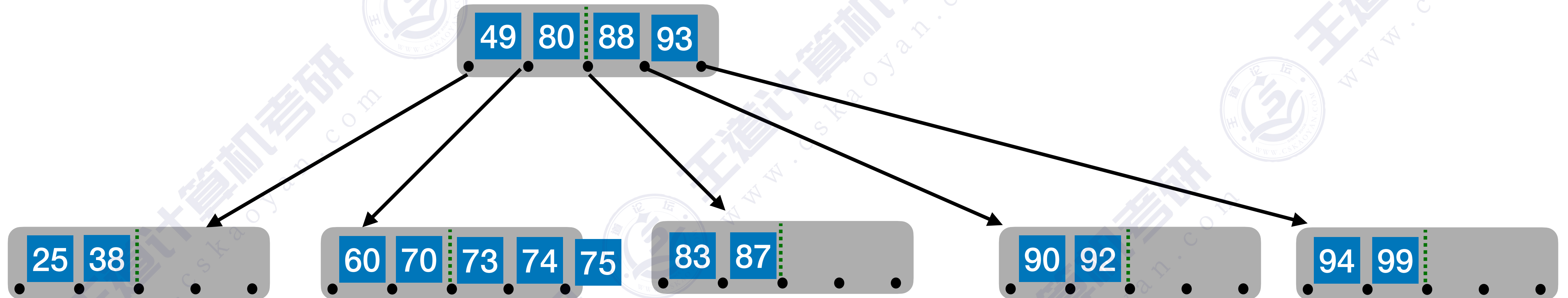


在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

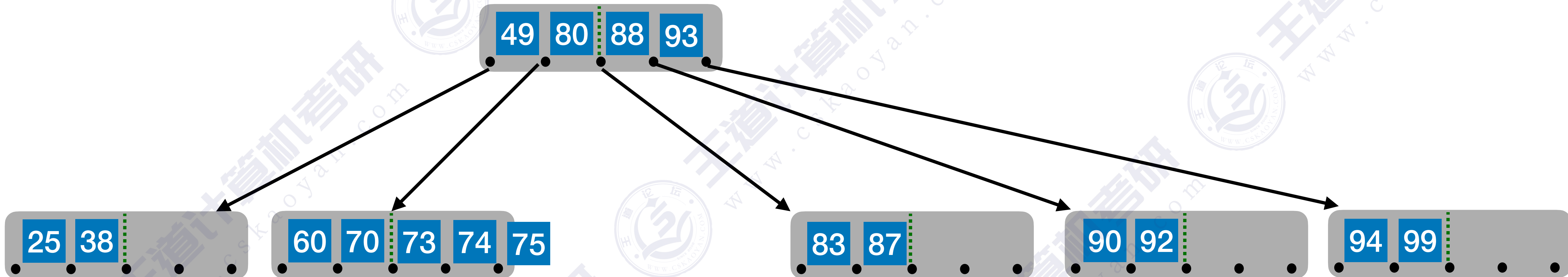


在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

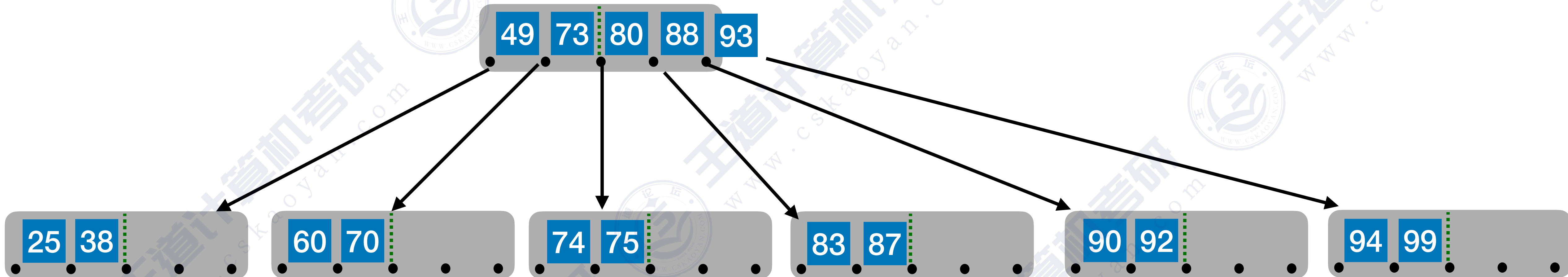


在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入



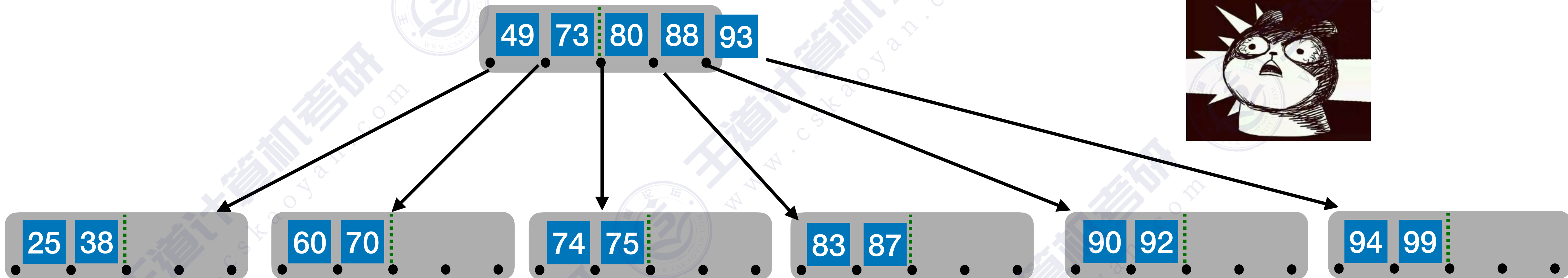
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

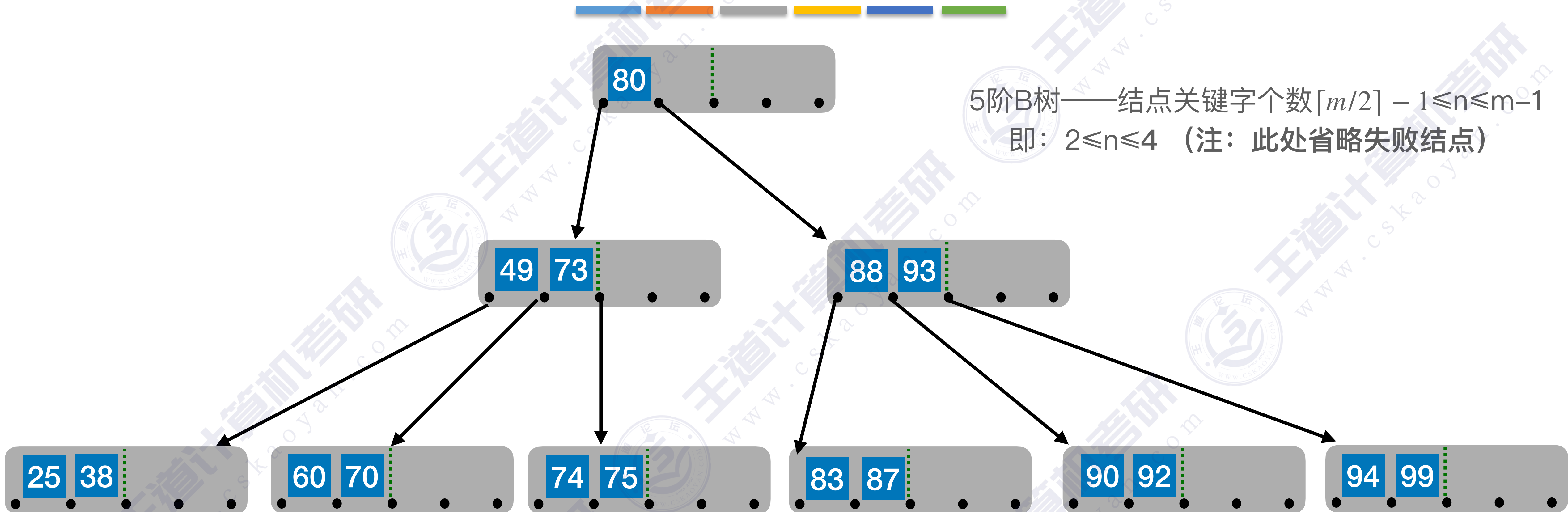
B树的插入

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



在插入key后，若导致原结点关键字数超过上限，则从中间位置 ($\lceil m/2 \rceil$) 将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置 ($\lceil m/2 \rceil$) 的结点插入原结点的父结点。若此时导致其父结点的关键字数也超过了上限，则继续进行这种分裂操作，直至这个过程传到根结点为止，进而导致B树高度增1。

B树的插入



在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点。若此时导致其父结点的关键字数也超过了上限，则继续进行这种分裂操作，直至这个过程传到根结点为止，进而导致B树高度增1。

B树的插入



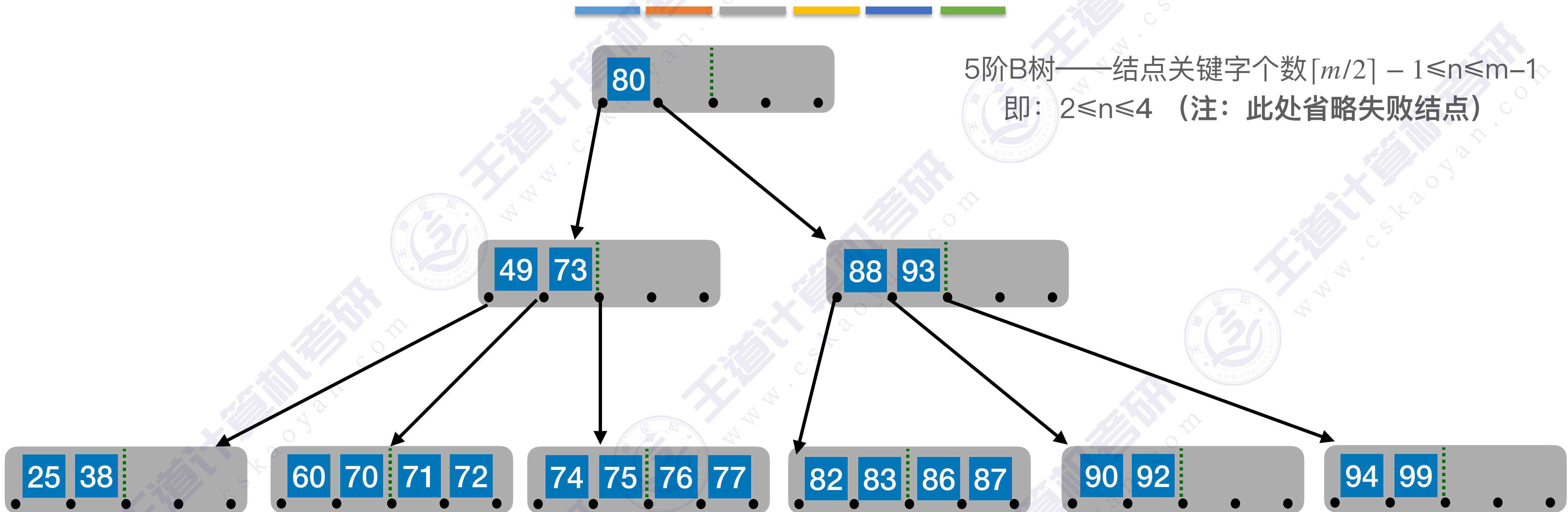
核心要求：

- ①对 m 阶B树——除根节点外，结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
- ②子树 $0 < \text{关键字} 1 < \text{子树} 1 < \text{关键字} 2 < \text{子树} 2 < \dots$

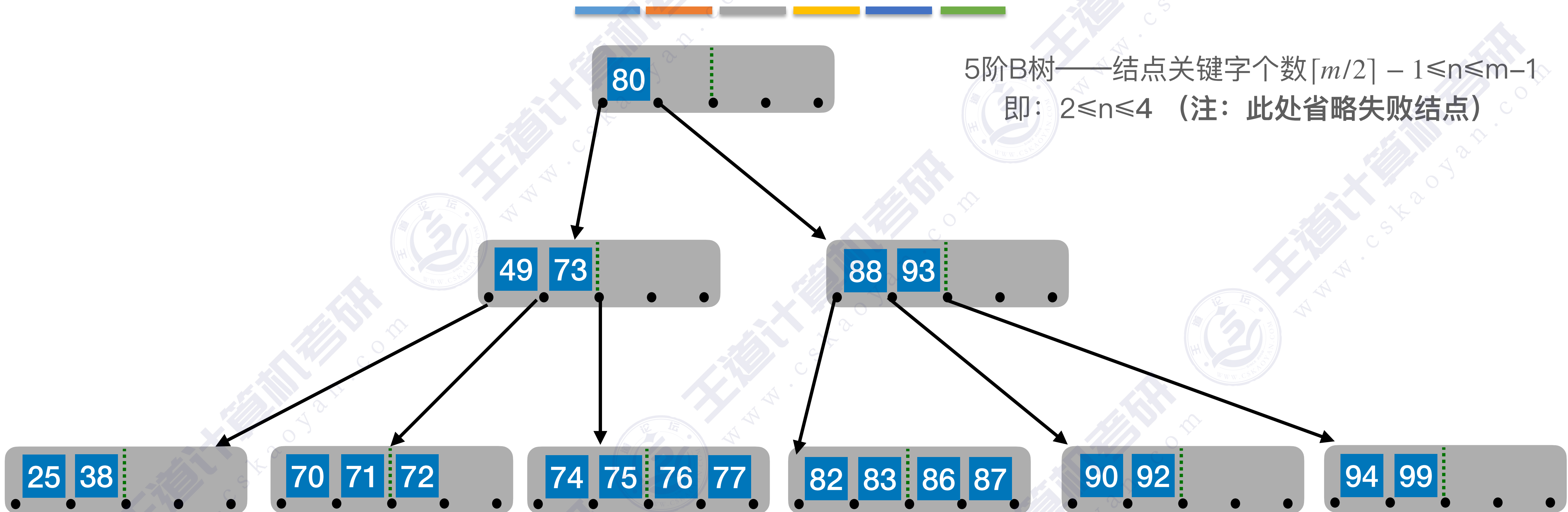
新元素一定是插入到最底层“终端节点”，用“查找”来确定插入位置

在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点。若此时导致其父结点的关键字数也超过了上限，则继续进行这种分裂操作，直至这个过程传到根结点为止，进而导致B树高度增1。

B树的删除

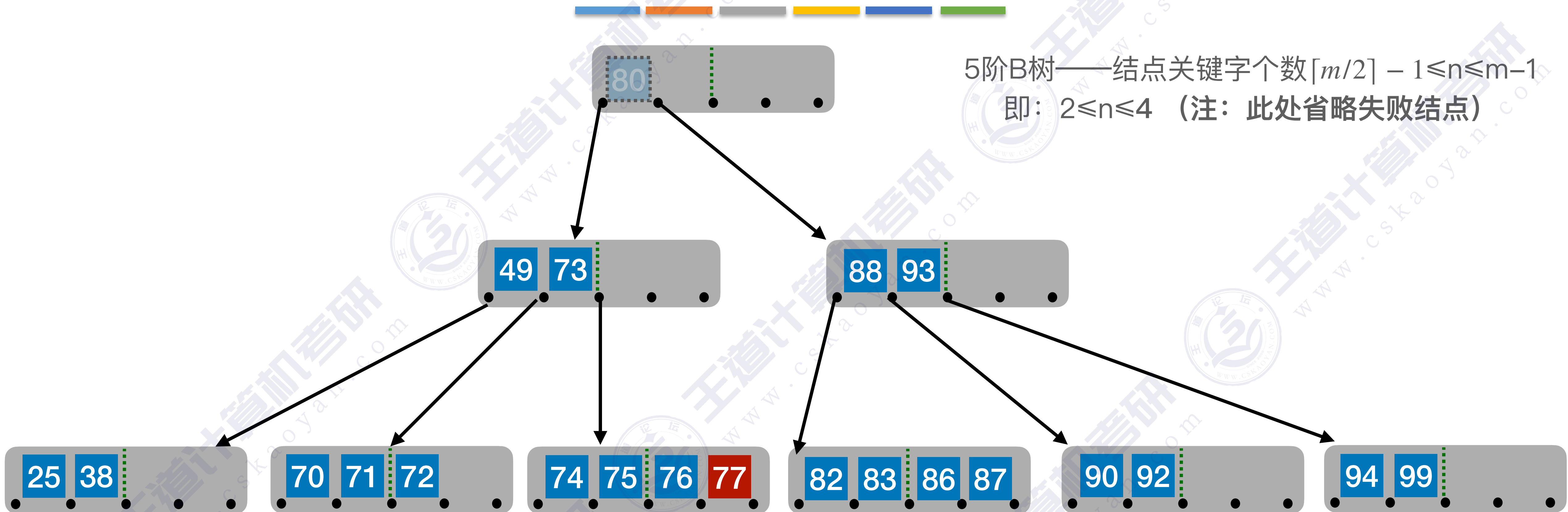


B树的删除



若被删除关键字在**终端节点**，则直接删除该关键字（要注意节点关键字个数是否低于下限 $\lceil m/2 \rceil - 1$ ）

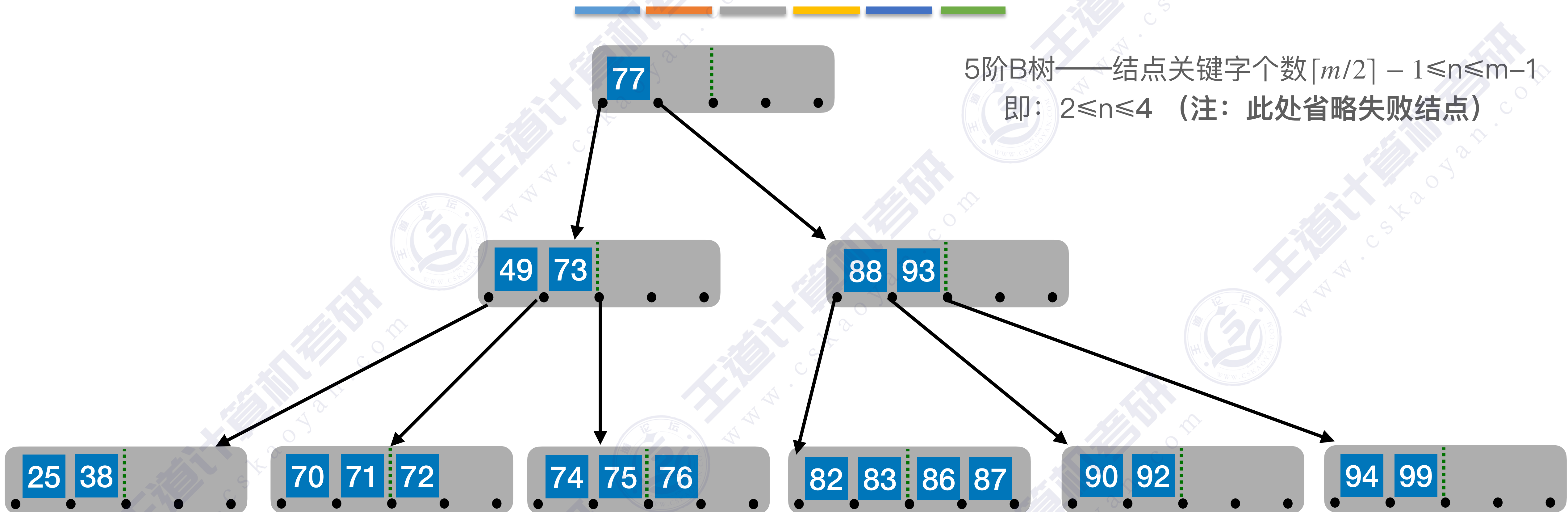
B树的删除



若被删除关键字在**非终端节点**，则用直接前驱或直接后继来替代被删除的关键字

直接前驱：当前关键字**左侧指针所指子树中“最右下”**的元素

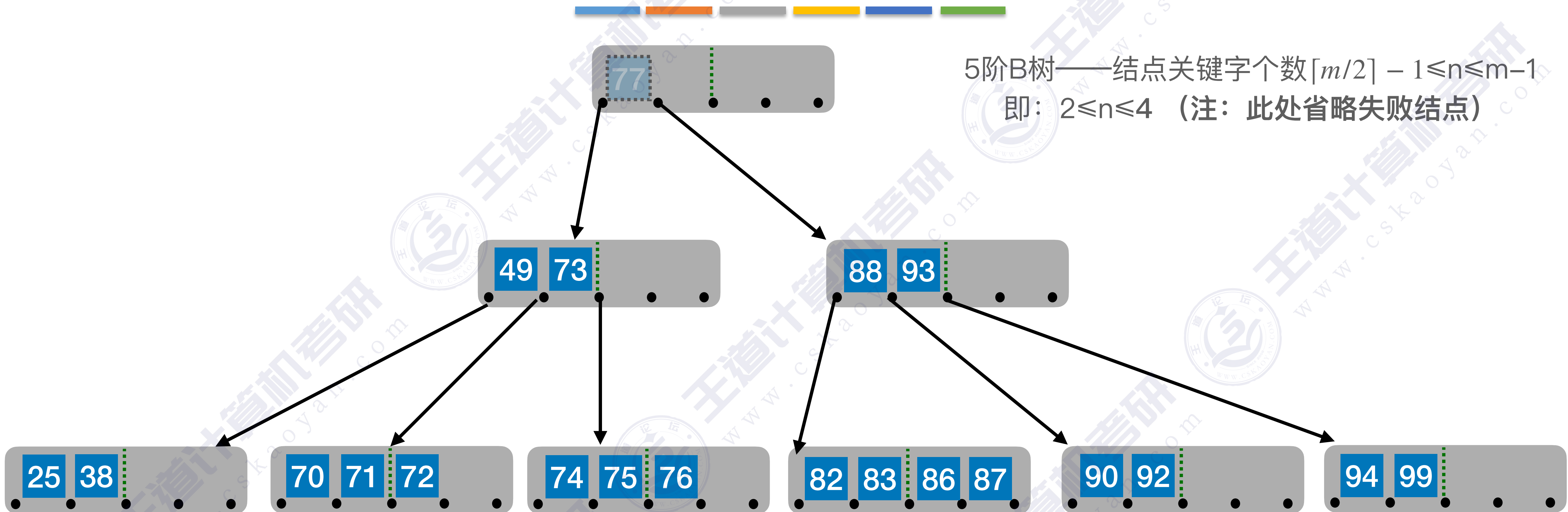
B树的删除



若被删除关键字在**非终端节点**，则用直接前驱或直接后继来替代被删除的关键字

直接前驱：当前关键字**左侧指针所指子树中“最右下”**的元素

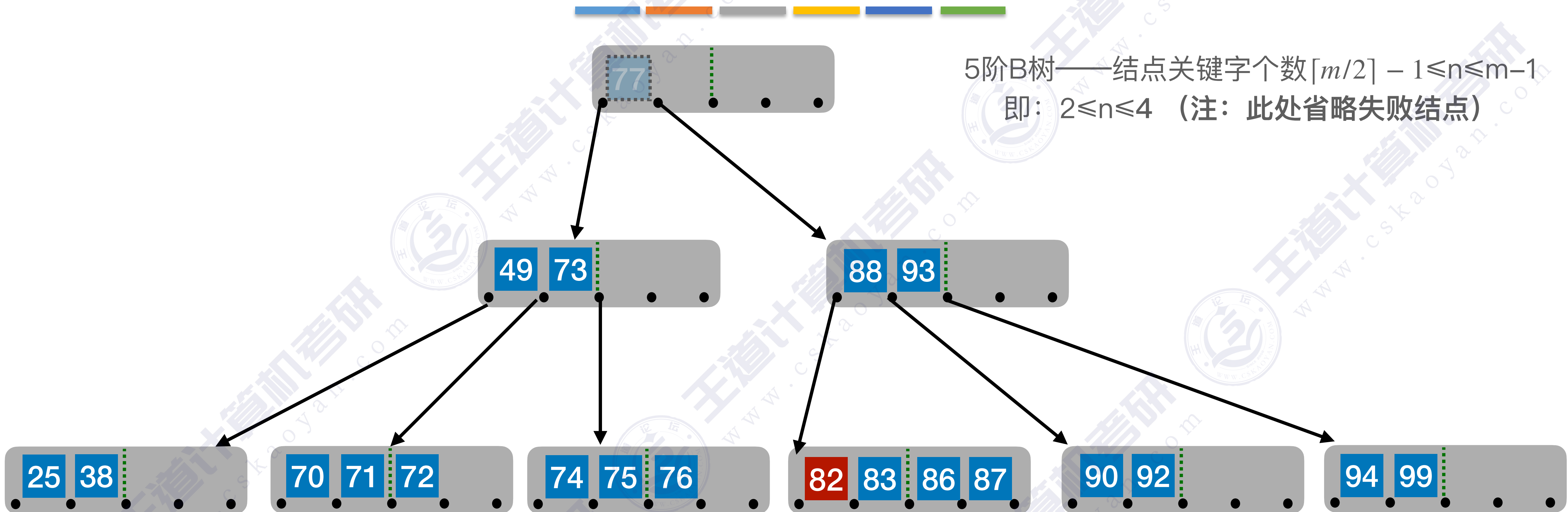
B树的删除



若被删除关键字在**非终端节点**，则用直接前驱或直接后继来替代被删除的关键字

直接前驱：当前关键字**左侧指针**所指子树中“**最右下**”的元素

B树的删除

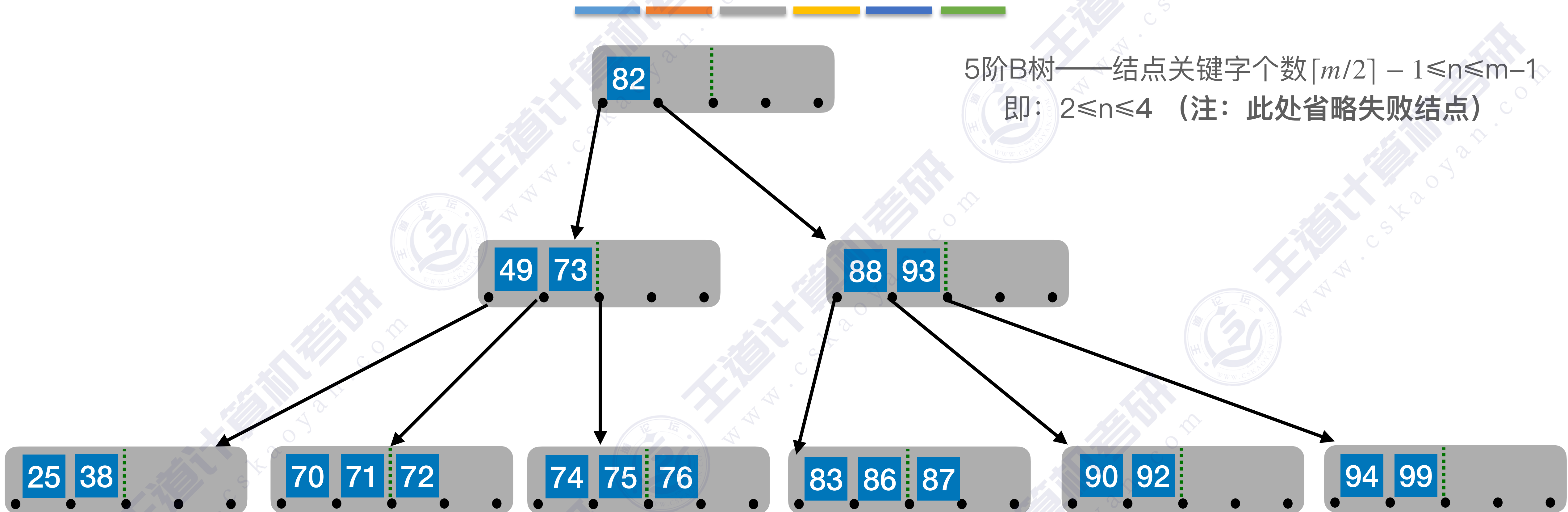


若被删除关键字在**非终端节点**，则用直接前驱或直接后继来替代被删除的关键字

直接前驱：当前关键字**左侧**指针所指子树中“**最右下**”的元素

直接后继：当前关键字**右侧**指针所指子树中“**最左下**”的元素

B树的删除



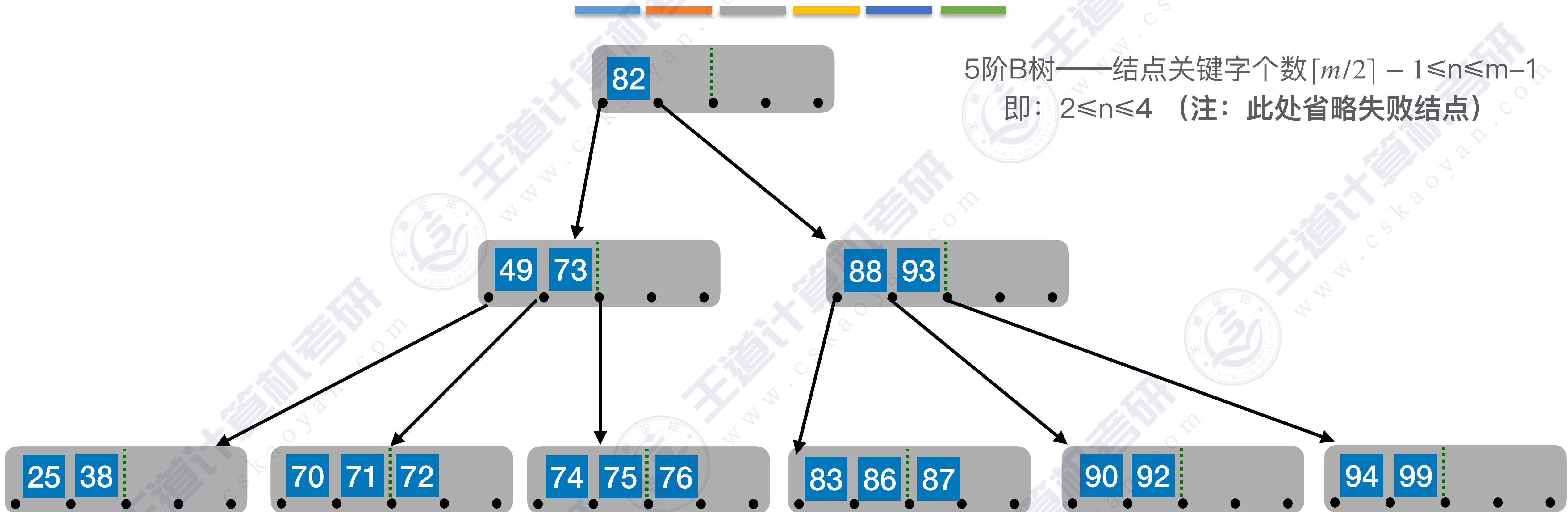
若被删除关键字在**非终端节点**，则用直接前驱或直接后继来替代被删除的关键字

直接前驱：当前关键字左侧指针所指子树中“最右下”的元素

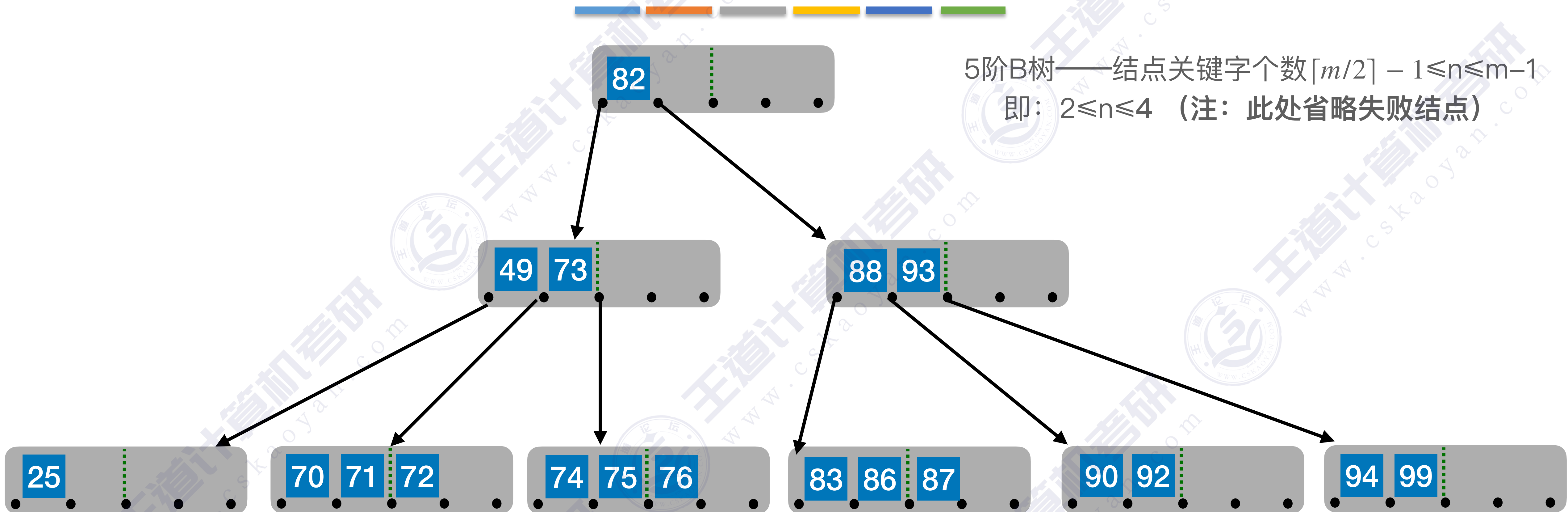
直接后继：当前关键字右侧指针所指子树中“最左下”的元素

对非终端结点关键字的删除，必然可以转化为对终端结点的删除操作

B树的删除

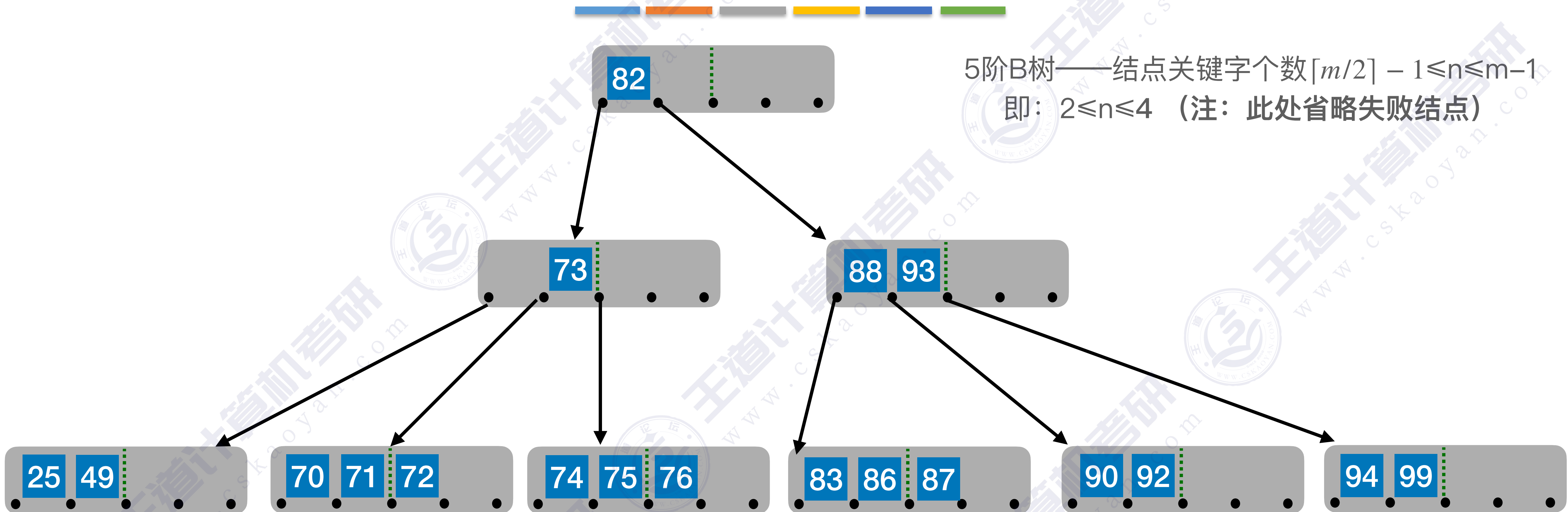


B树的删除



兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

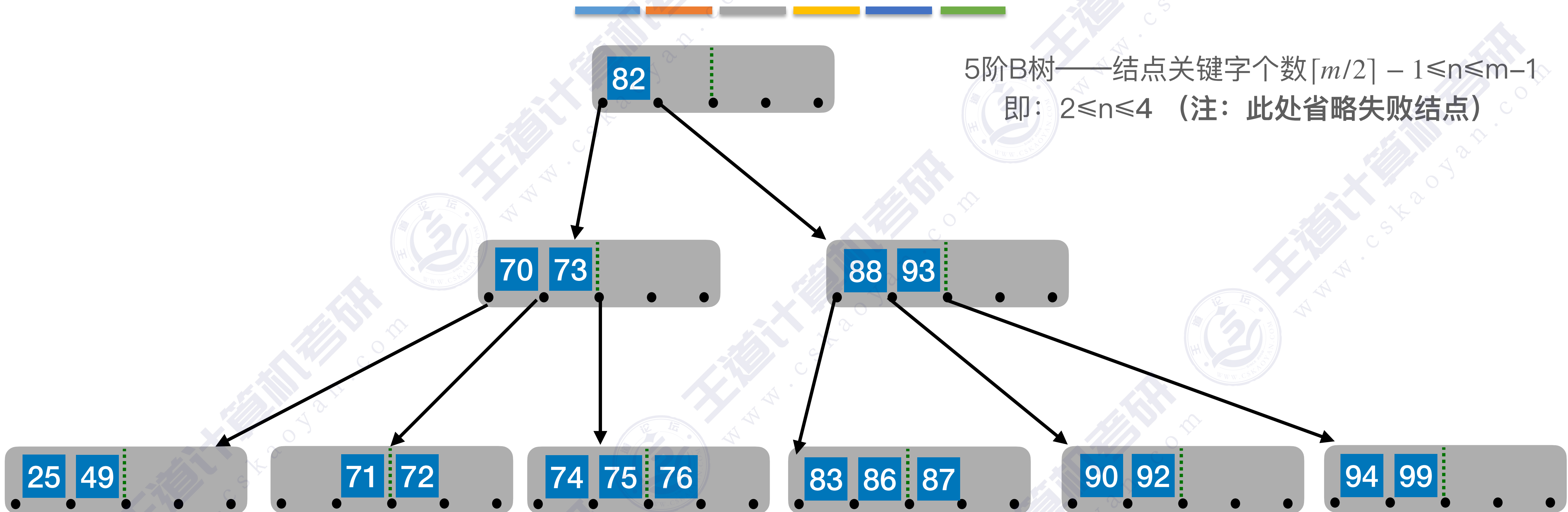
B树的删除



在？借点钱？

兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

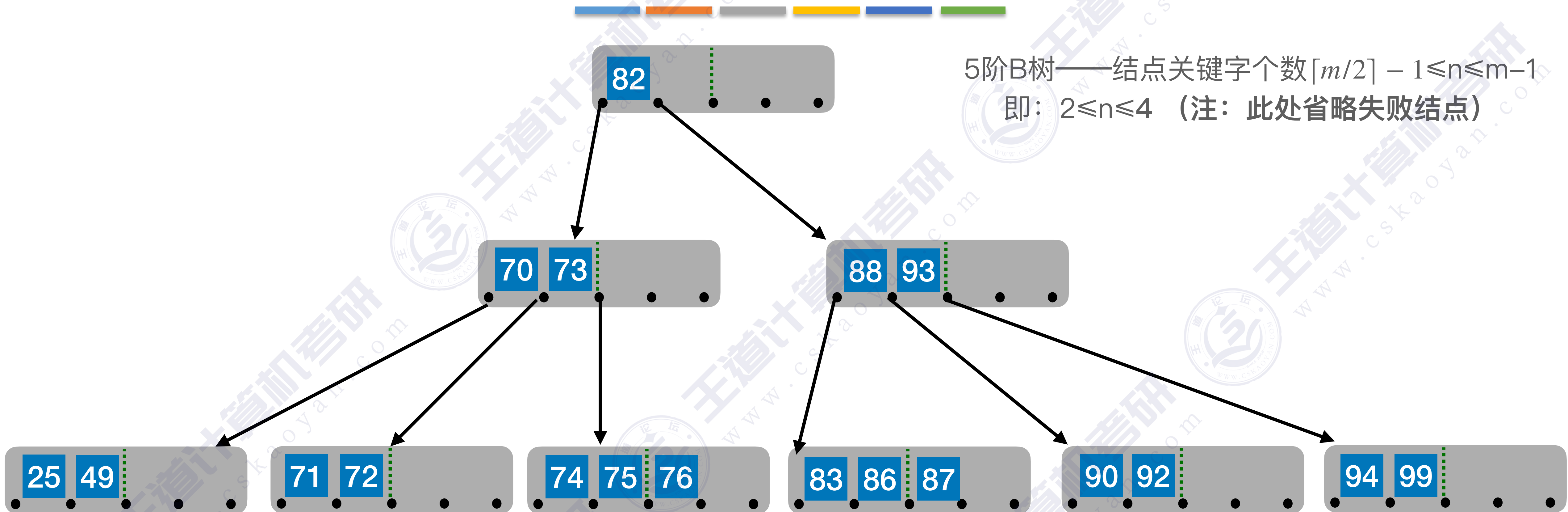
B树的删除



在？借点钱？

兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

B树的删除

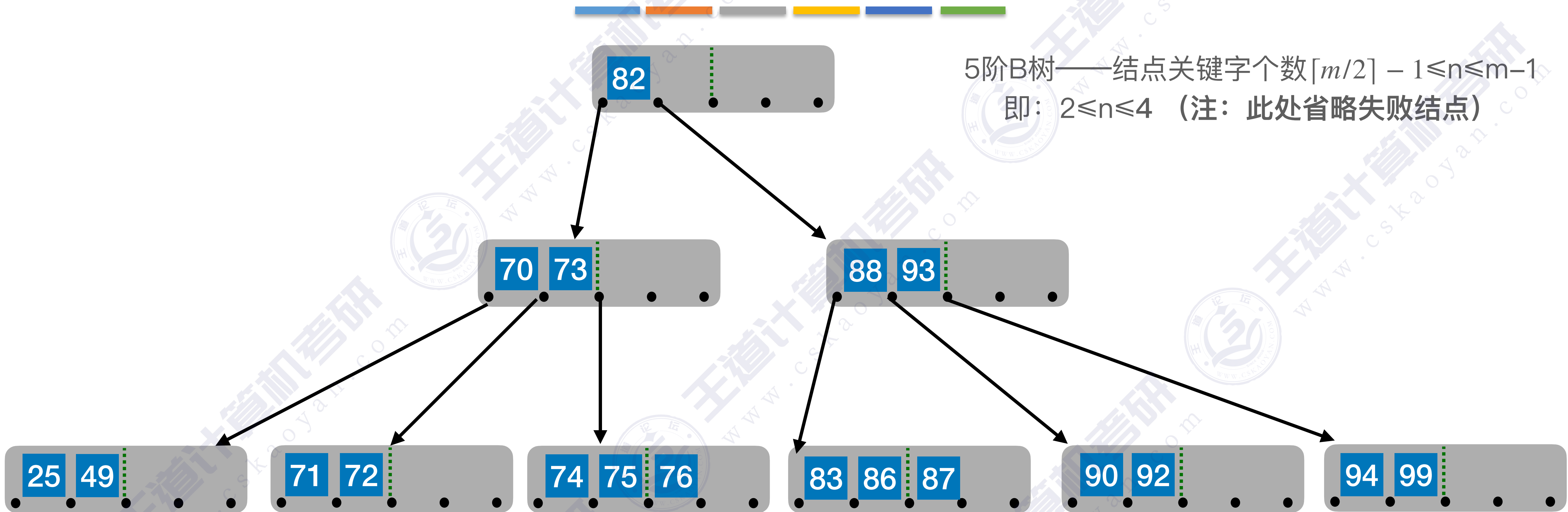


在？借点钱？

兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

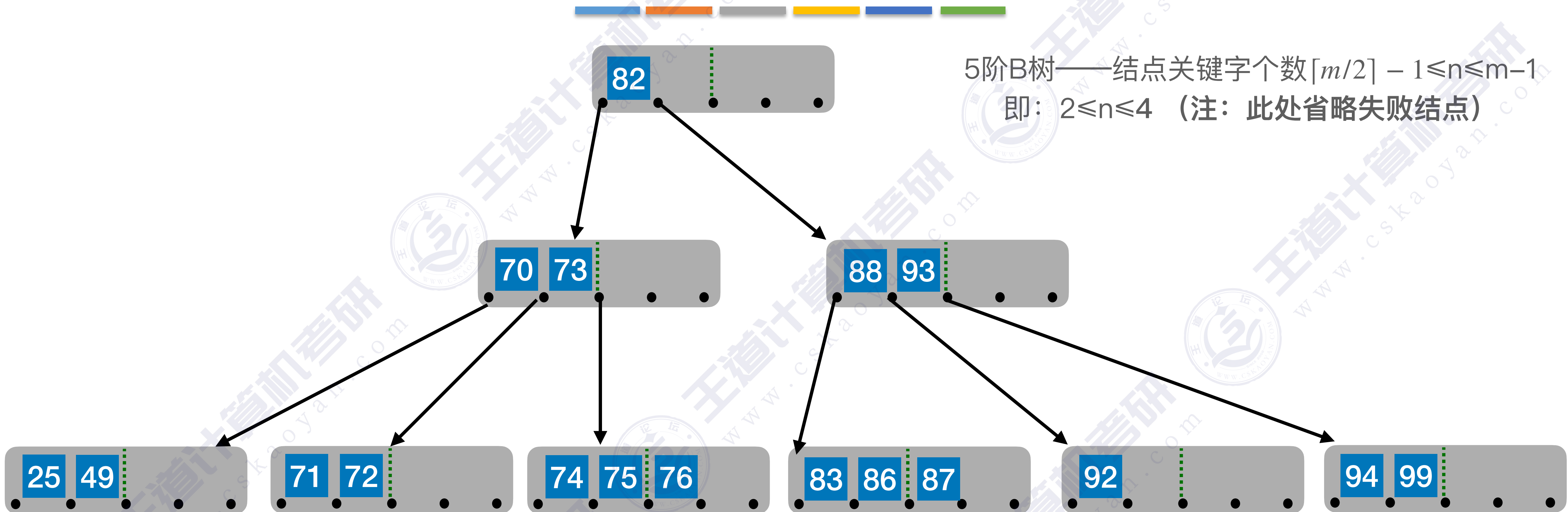
说白了，当**右兄弟很宽裕**时，用当前结点的**后继**、**后继的后继**来填补空缺

B树的删除



兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

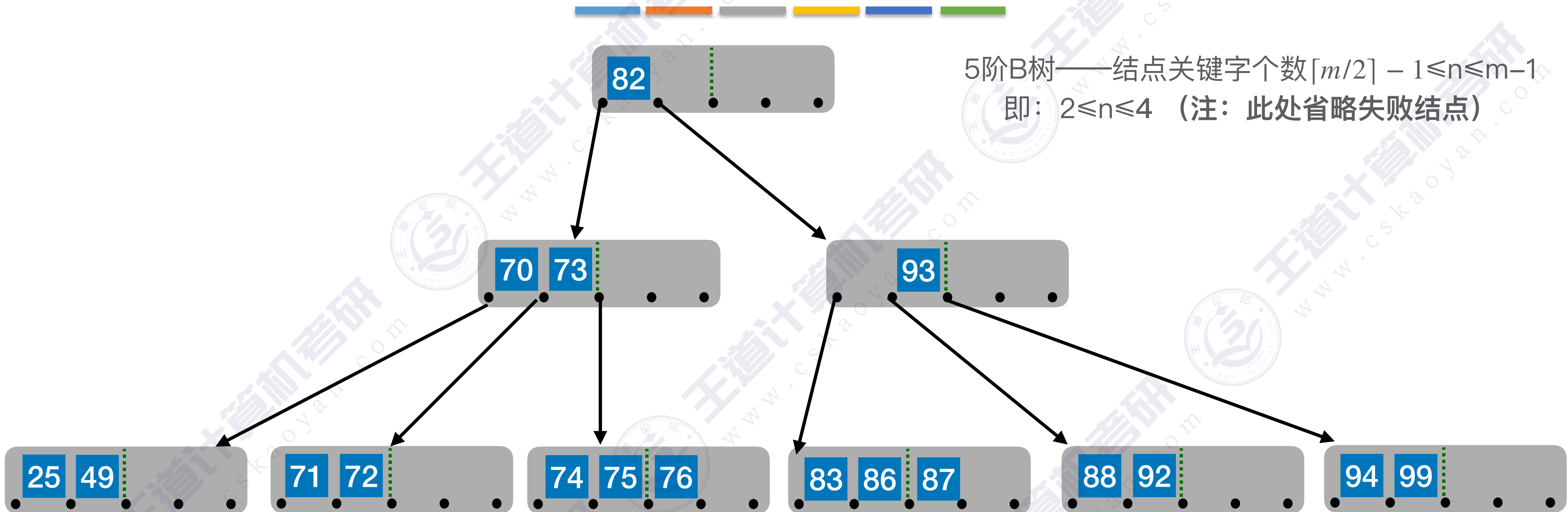
B树的删除



兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

当左兄弟很宽裕时，用当前结点的前驱、前驱的前驱来填补空缺

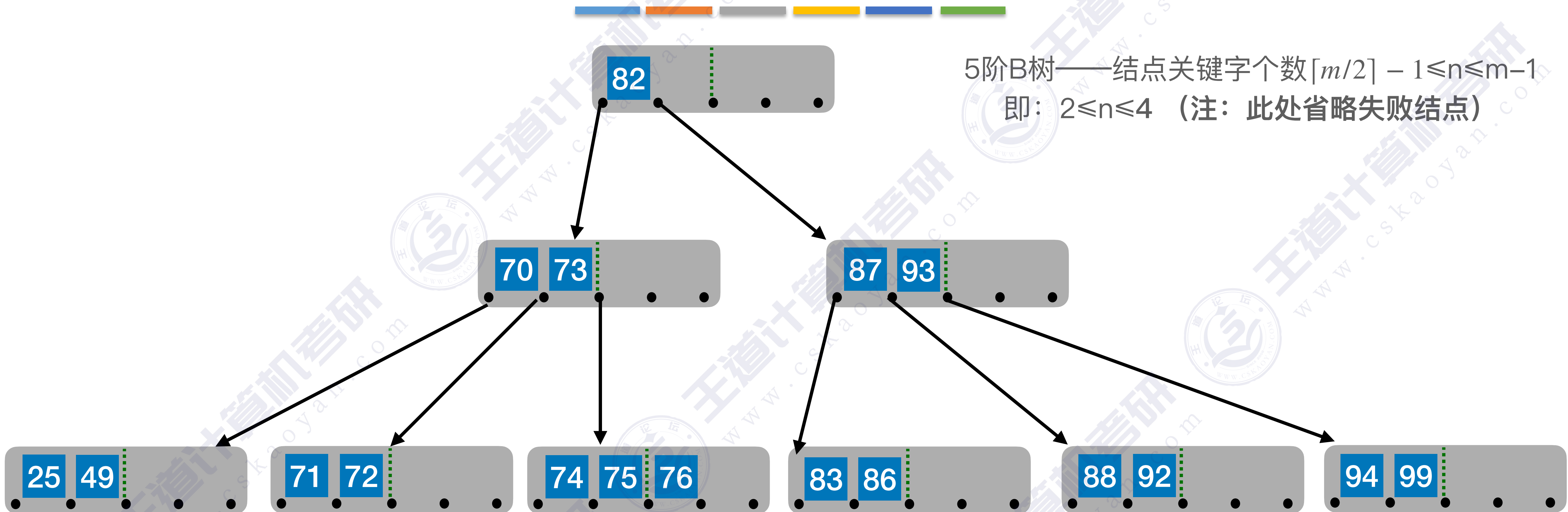
B树的删除



兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

当左兄弟很宽裕时，用当前结点的前驱、前驱的前驱来填补空缺

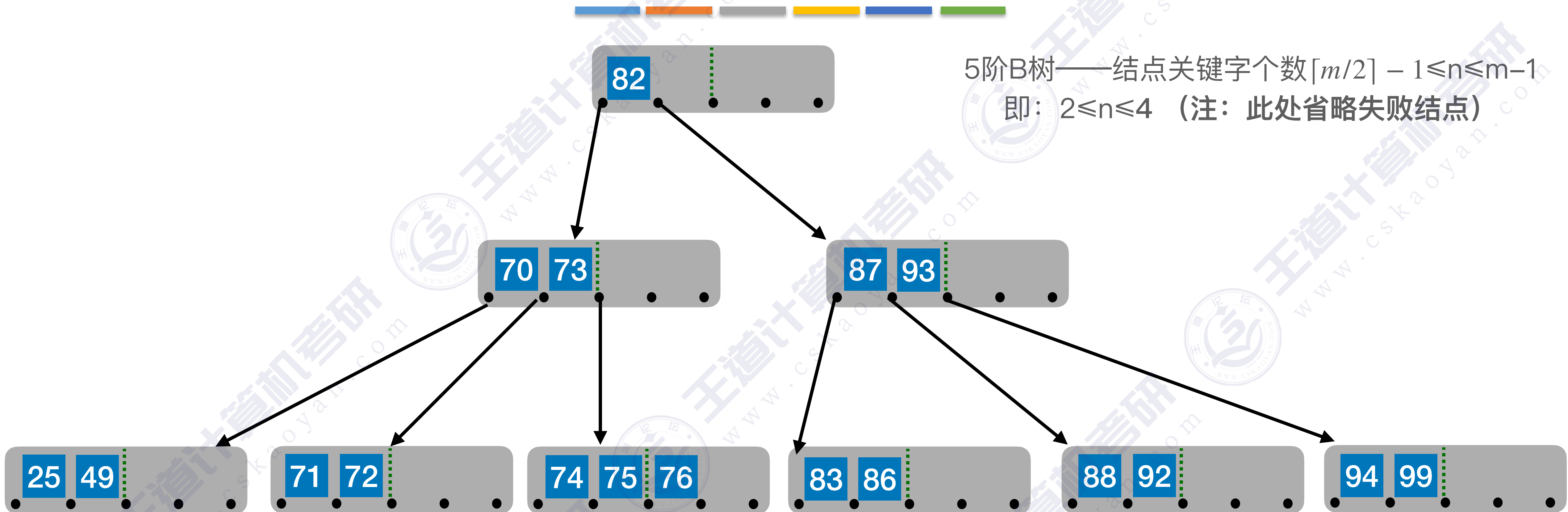
B树的删除



兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

当左兄弟很宽裕时，用当前结点的前驱、前驱的前驱来填补空缺

B树的删除

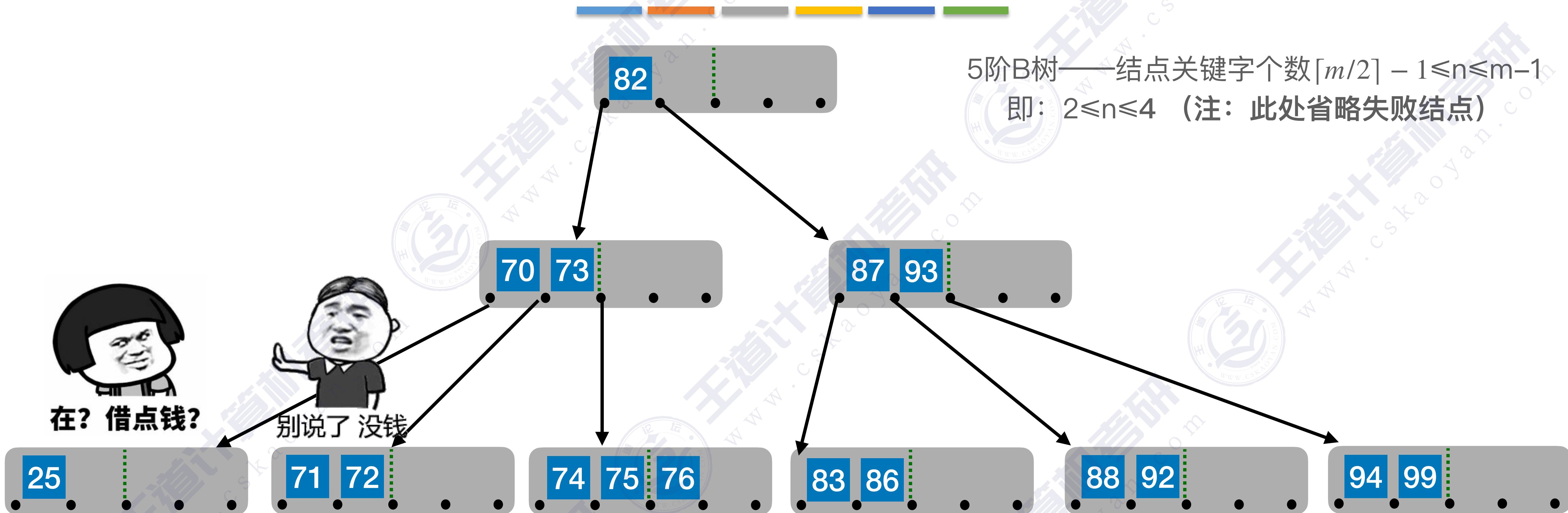


兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

本质：要永远保证 子树0<关键字1<子树1<关键字2<子树2<....

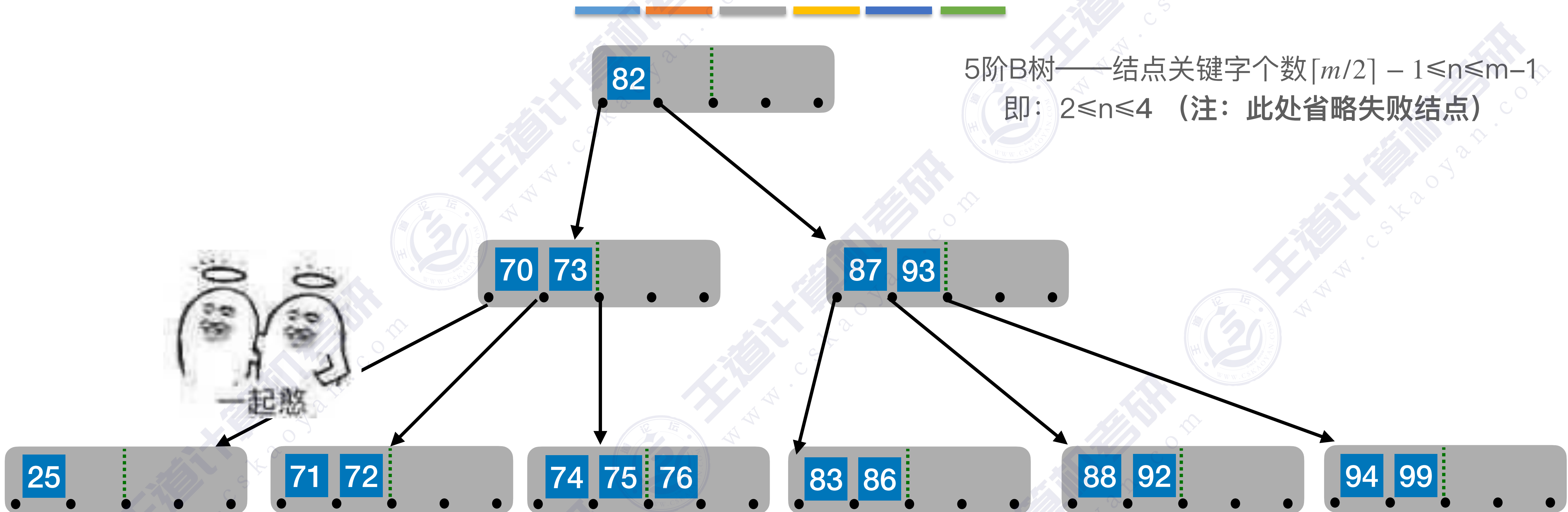
B树的删除

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



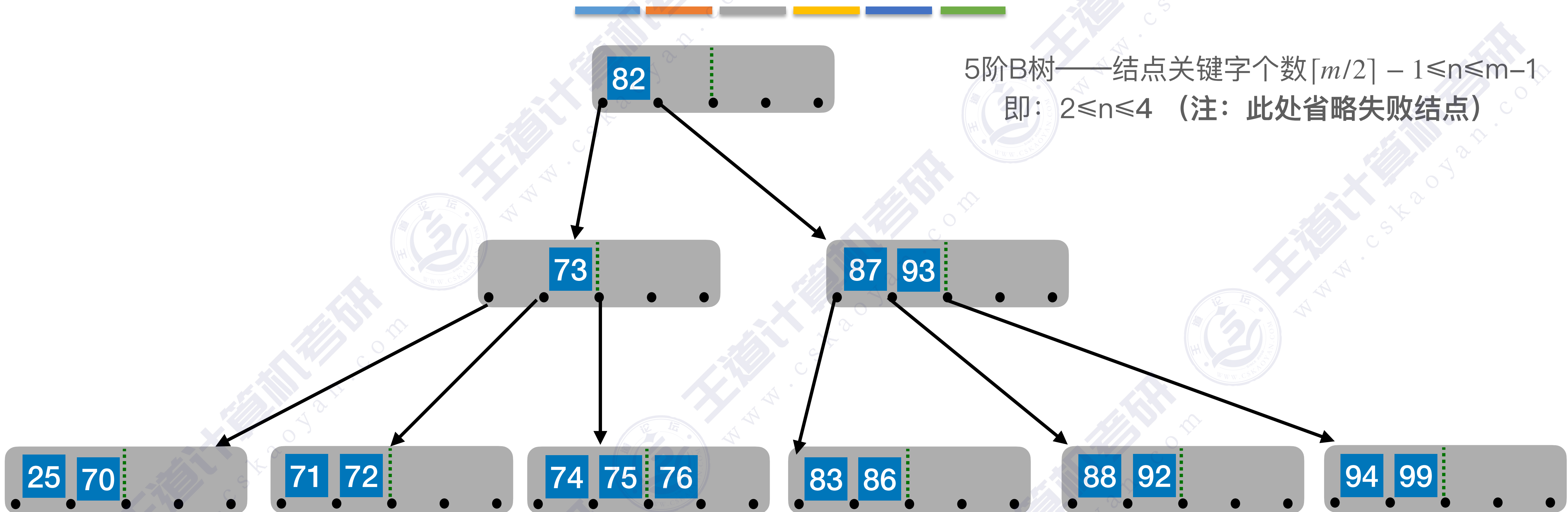
兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

B树的删除



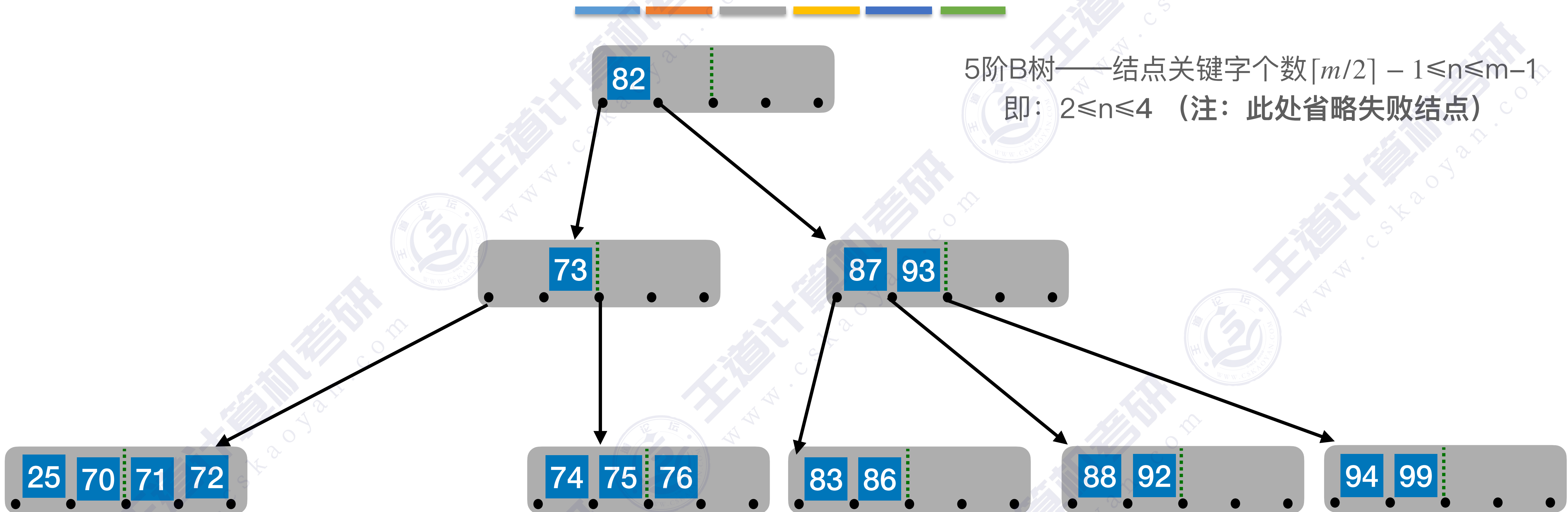
兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

B树的删除



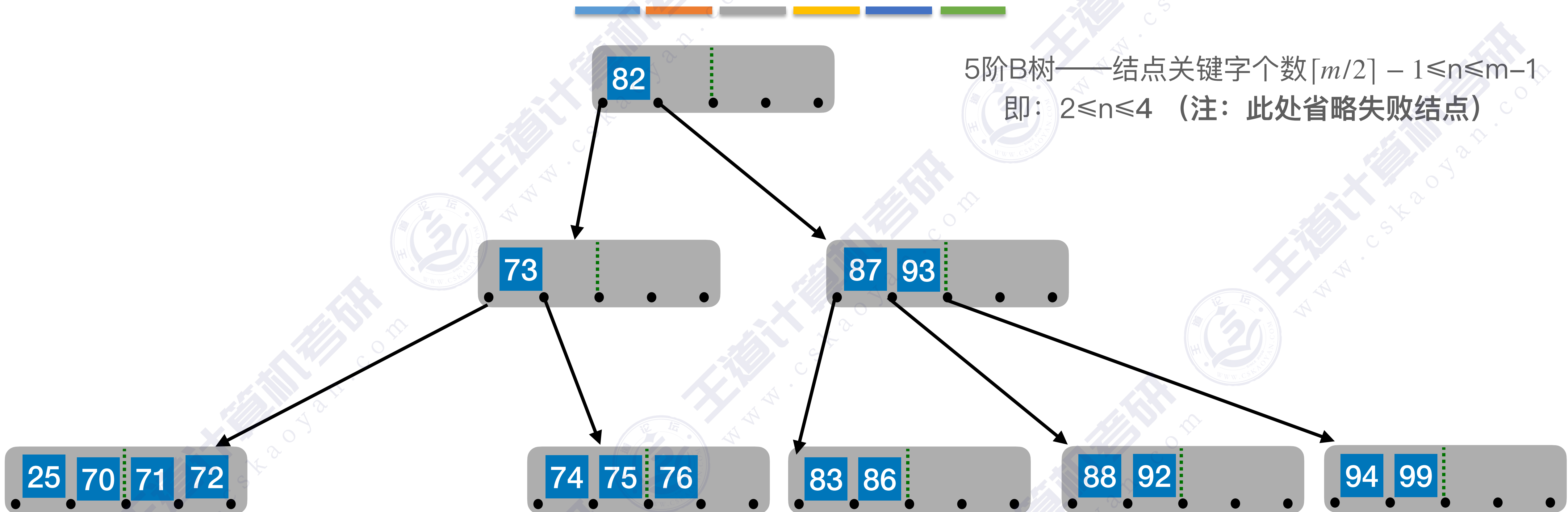
兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

B树的删除



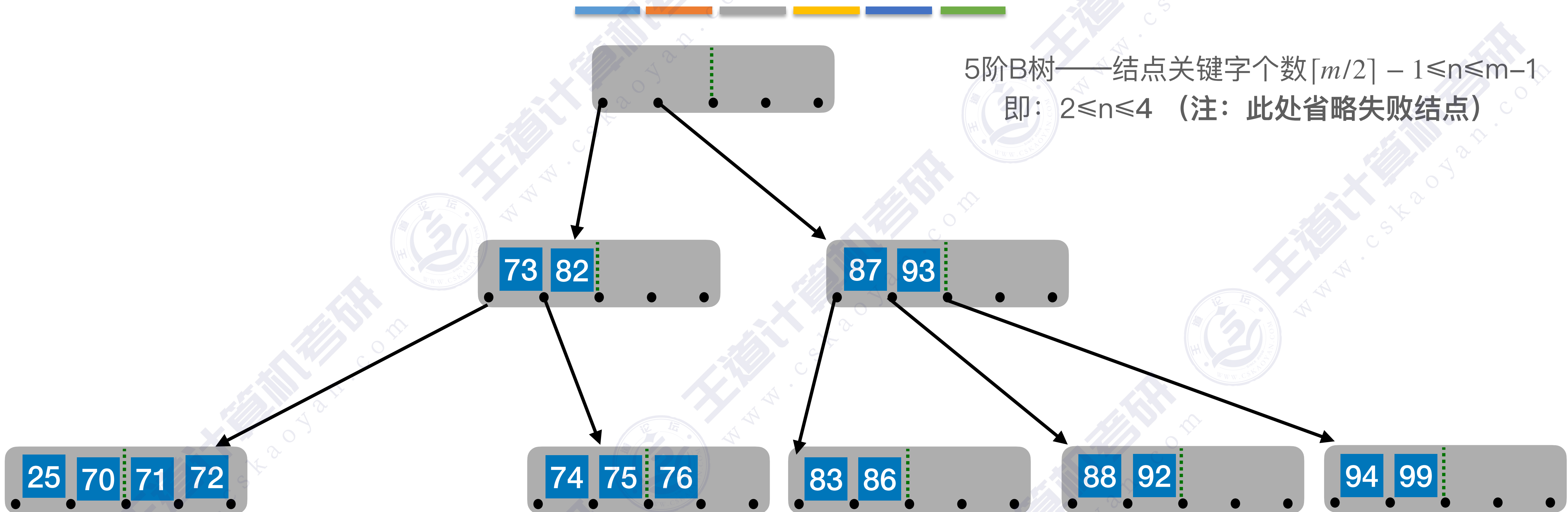
兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

B树的删除



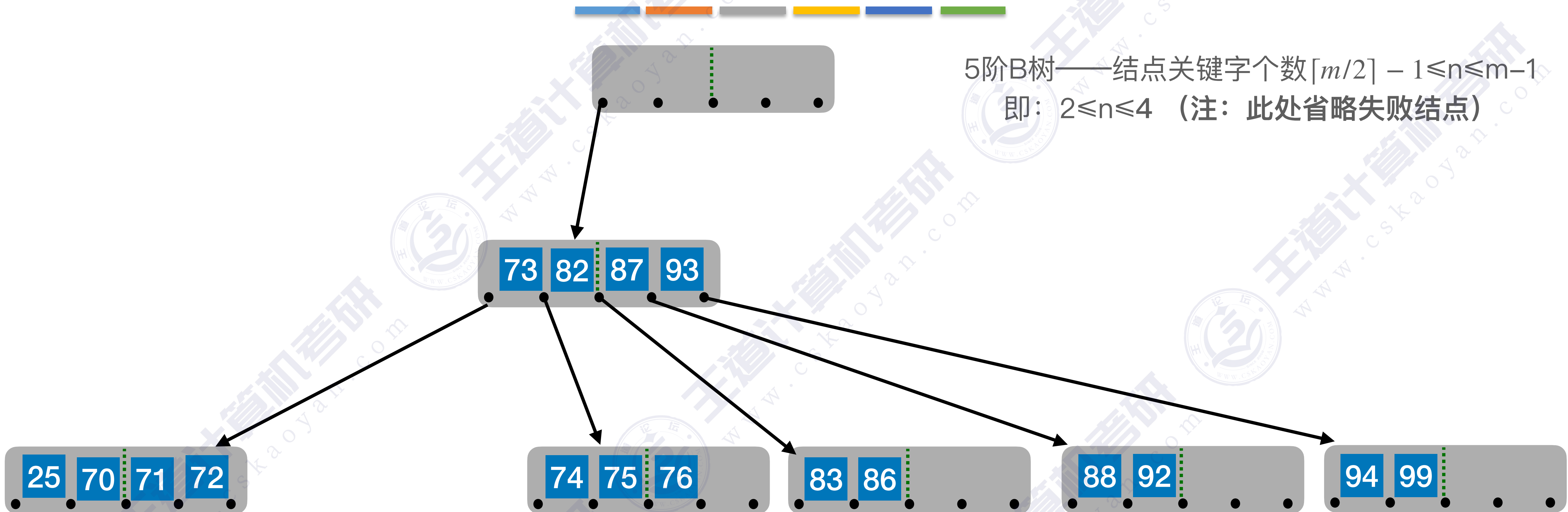
兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

B树的删除



兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

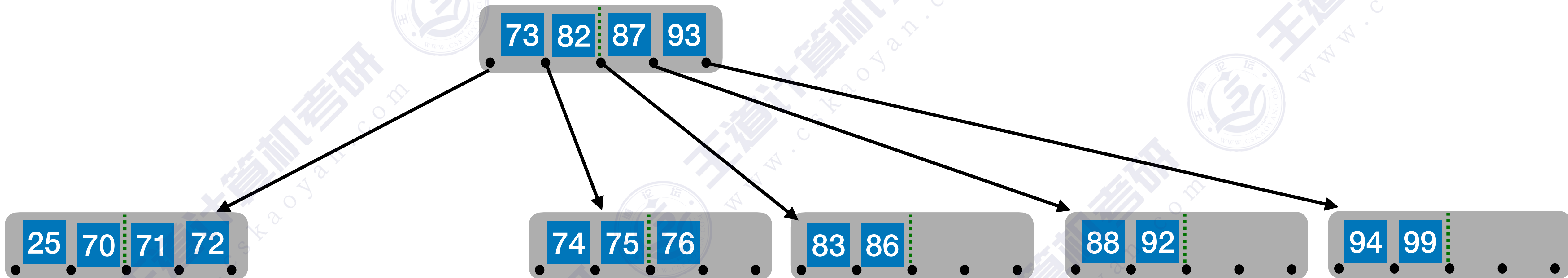
B树的删除



兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

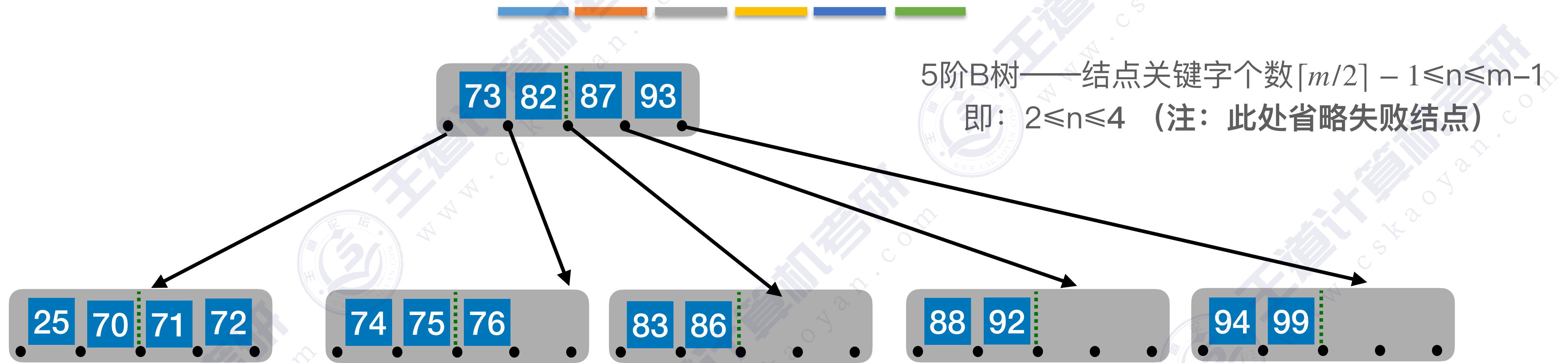
B树的删除

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

B树的删除



兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

在合并过程中，双亲结点中的关键字个数会减1。若其双亲结点是根结点且关键字个数减少至0（根结点关键字个数为1时，有2棵子树），则直接将根结点删除，合并后的新结点成为根；若双亲结点不是根结点，且关键字个数减少到 $\lceil m/2 \rceil - 2$ ，则又要与它自己的兄弟结点进行调整或合并操作，并重复上述步骤，直至符合B树的要求为止。

知识回顾与重要考点

通过“查找”确定插入位置（一定是在终端结点）

若插入后结点关键字个数未超过上限，则无需做其他处理

若插入后关键字个数超过上限，则需要将当前结点的中间元素放到父节点中，当前结点分裂为两个部分；该操作会导致父节点关键字个数+1，若父节点关键字个数也超过了上限，则需要再向上分裂；根节点的分裂会导致B树高度+1。

用其直接前驱或直接后继替代其位置，转化为对“终端结点”的删除

直接前驱：当前关键字左边指针所指子树中“最右下”的元素

直接后继：当前关键字右边指针所指子树中“最左下”的元素

删除后结点关键字个数未低于下限，无需任何处理

右兄弟够借，则用当前结点的后继、后继的后继依次顶替空缺

左兄弟够借，则用当前结点的前驱、前驱的前驱依次顶替空缺

左(右)兄弟都不够借，则需要与父结点内的关键字、左(右)兄弟进行合并。合并后导致父节点关键字数量-1，可能需要继续合并。

核心要求：

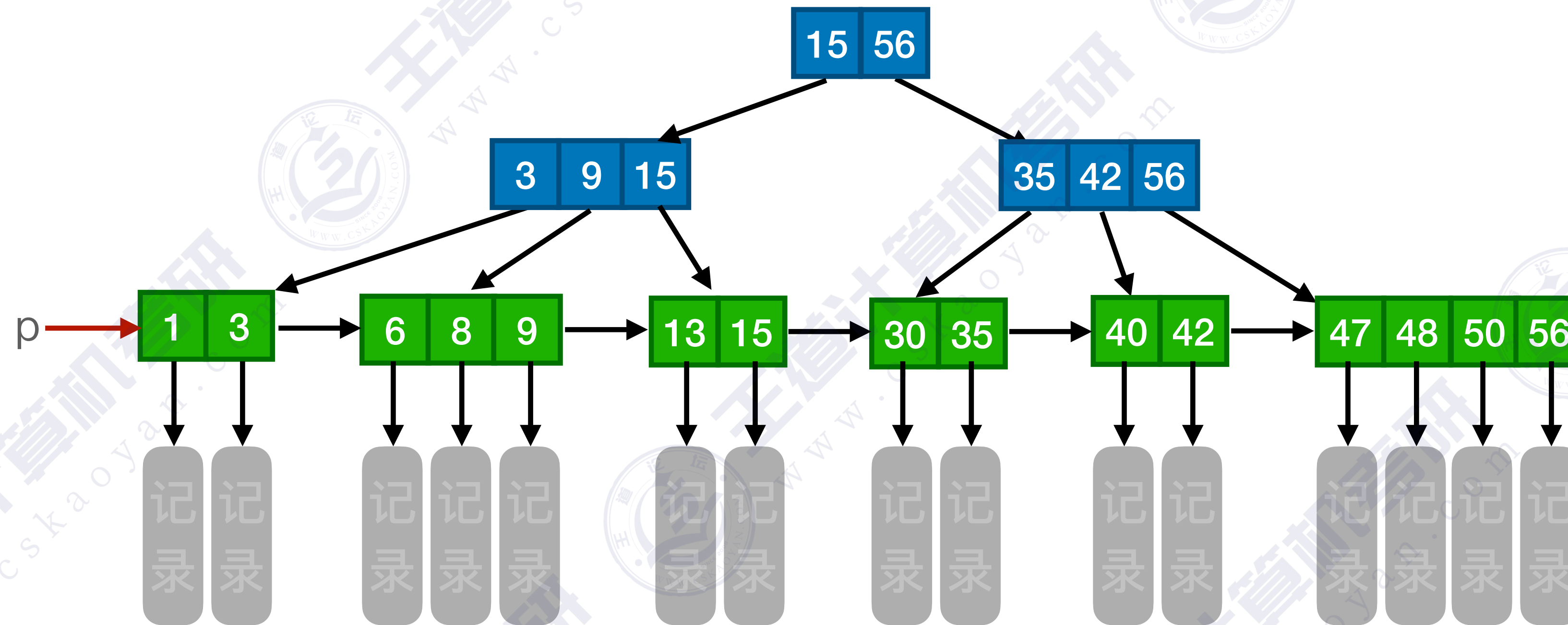
①对m阶B树——除根节点外，结点关键字个数 $[m/2] - 1 \leq n \leq m-1$

②子树0<关键字1<子树1<关键字2<子树2<....

本节内容

B+树

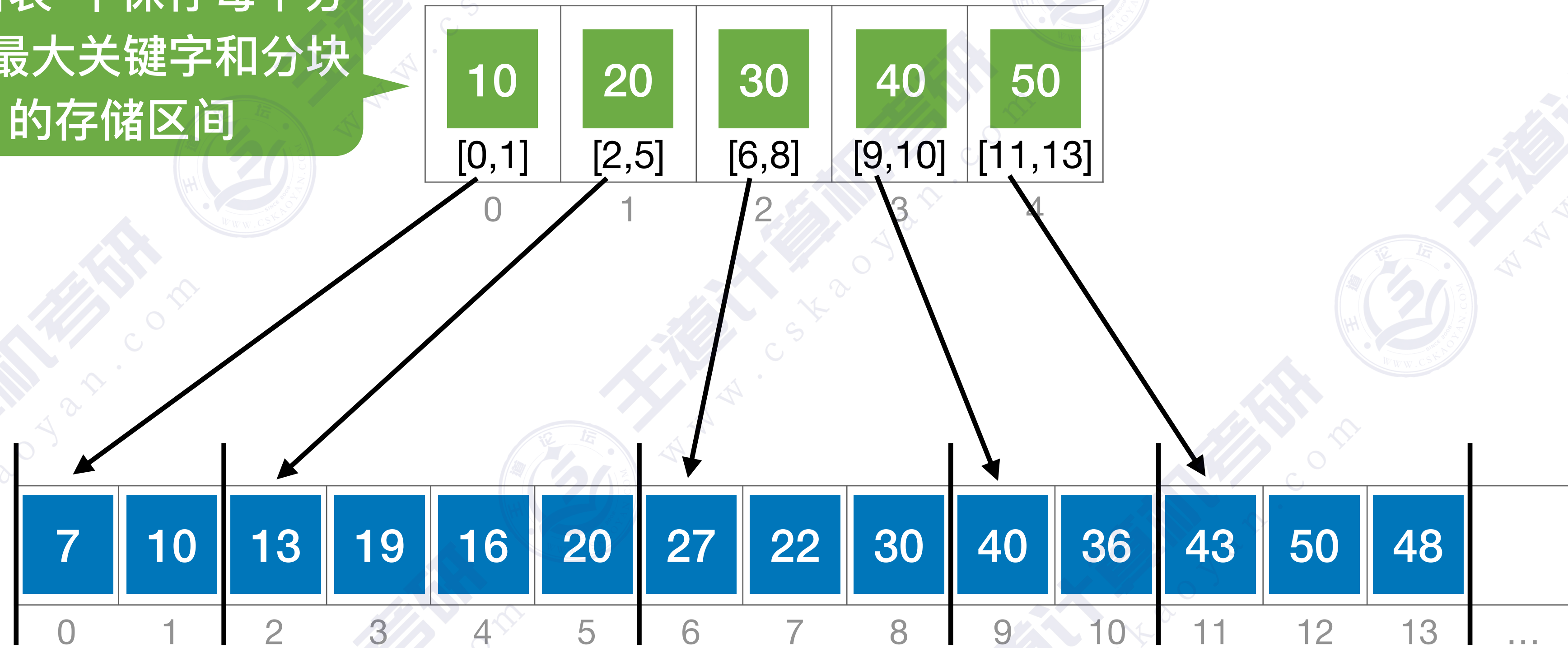
B+树



注：以上是一棵4阶B+树

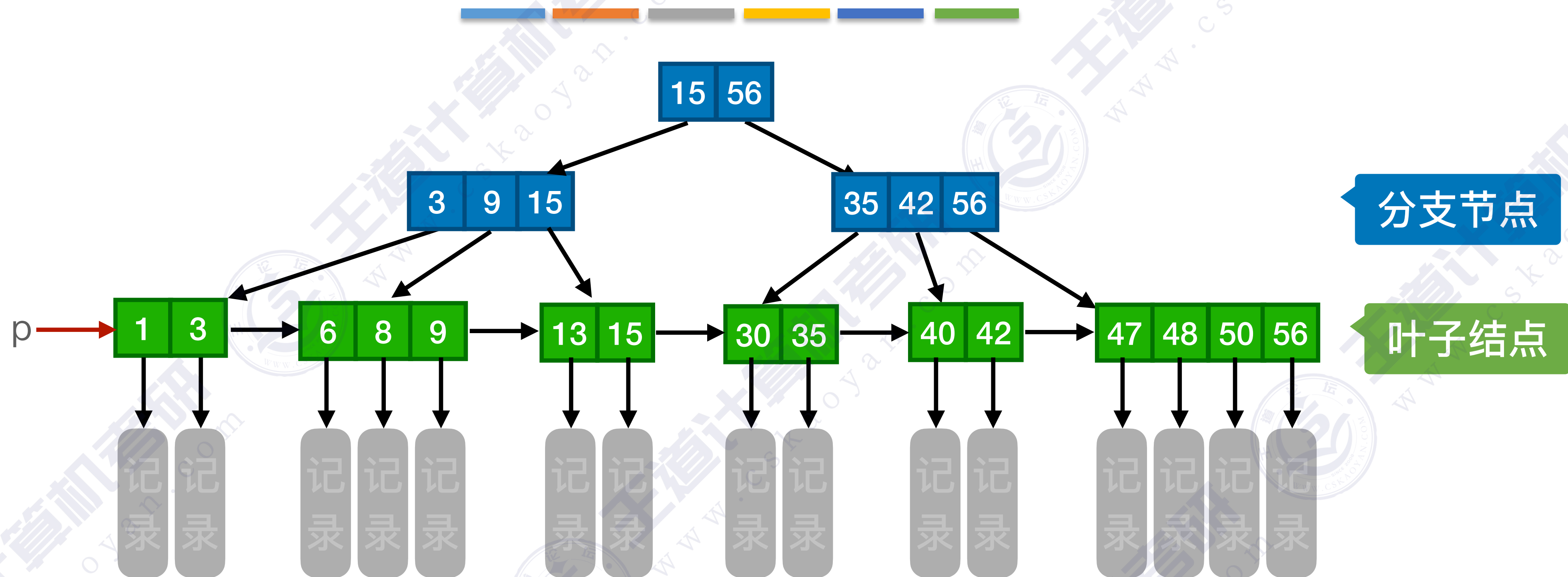
对比：分块查找

“索引表”中保存每个分块的最大关键字和分块的存储区间



特点：块内无序、块间有序

B+树



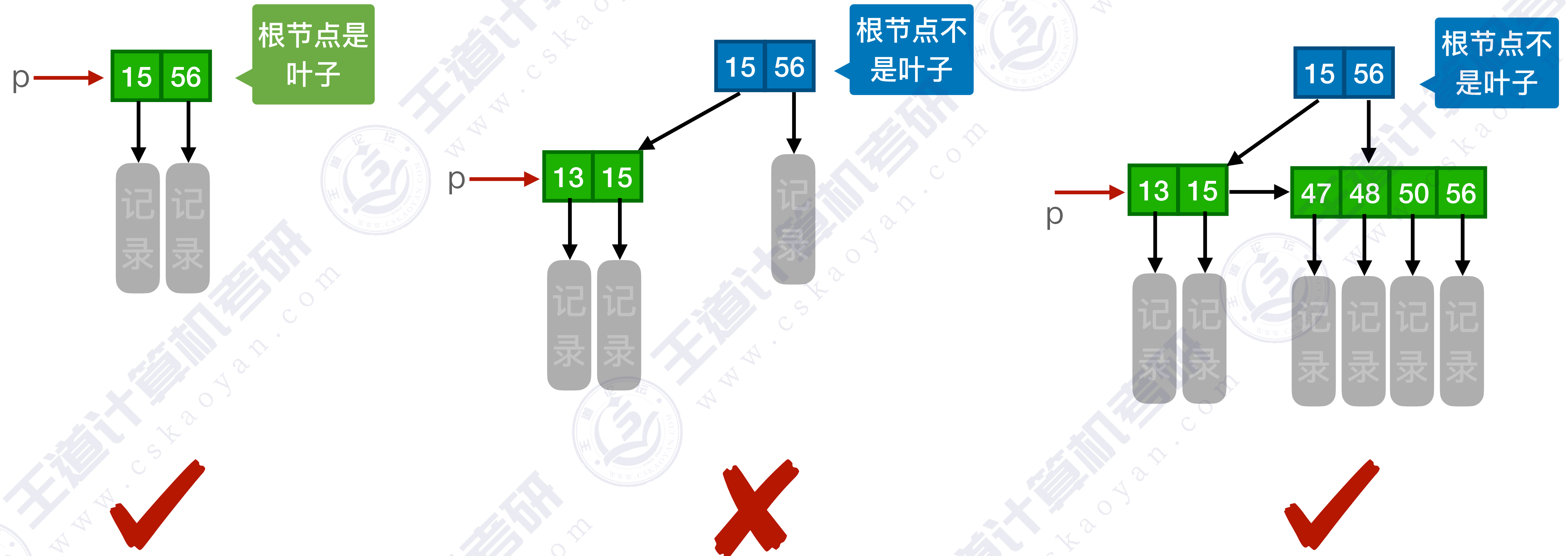
注：以上是一棵4阶B+树

一棵m阶的B+树需满足下列条件：

- 1) 每个分支结点最多有m棵子树（孩子结点）。
- 2) 非叶根结点至少有两棵子树，其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树。
- 3) 结点的子树个数与关键字个数相等。
- 4) 所有叶结点包含全部关键字及指向相应记录的指针，叶结点中将关键字按大小顺序排列，并且相邻叶结点按大小顺序相互链接起来。
- 5) 所有分支结点中仅包含它的各个子结点中关键字的最大值及指向其子结点的指针。

支持顺序查找

B+树



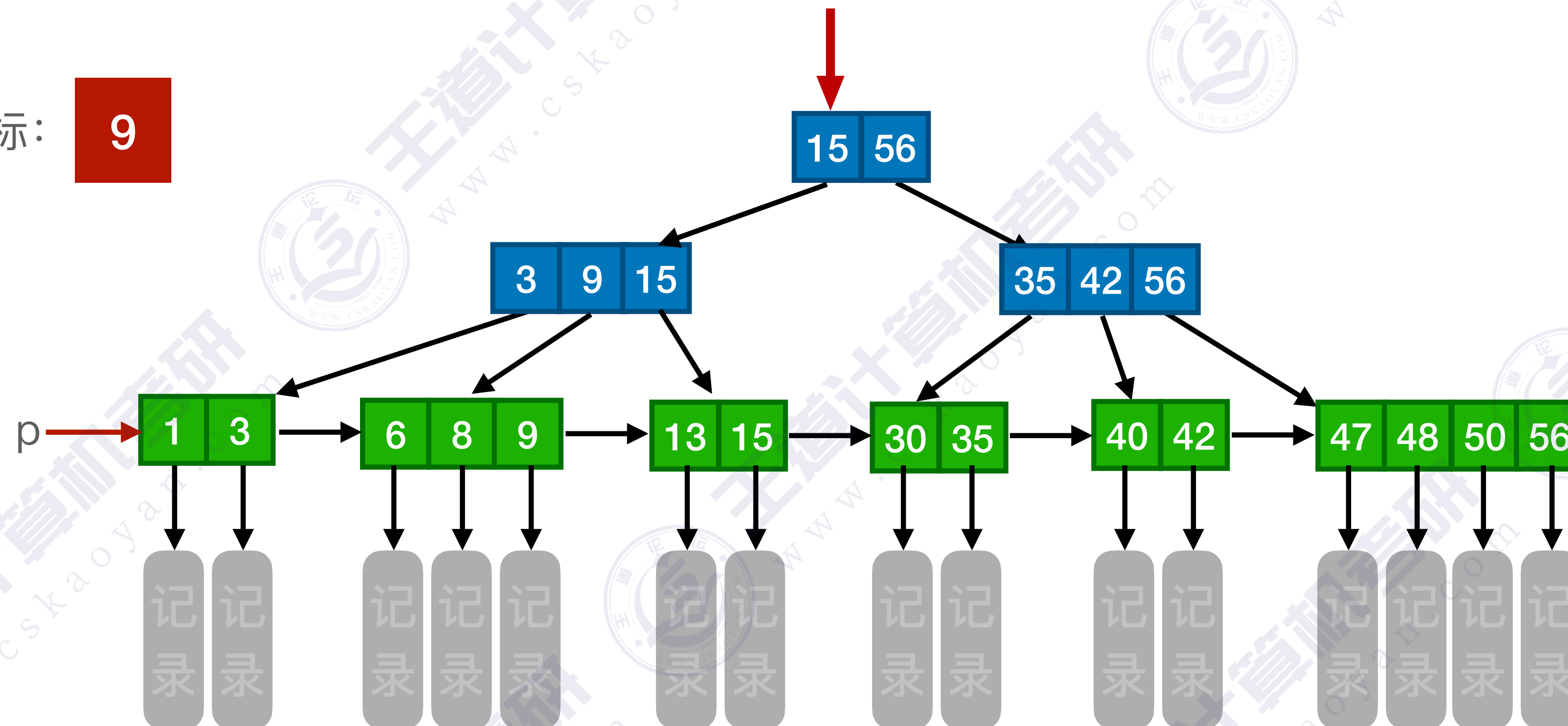
可以理解为：要追求“绝对平衡”，即所有子树高度要相同

2) 非叶根结点至少有两棵子树，其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树。

B+树的查找

查找目标:

9

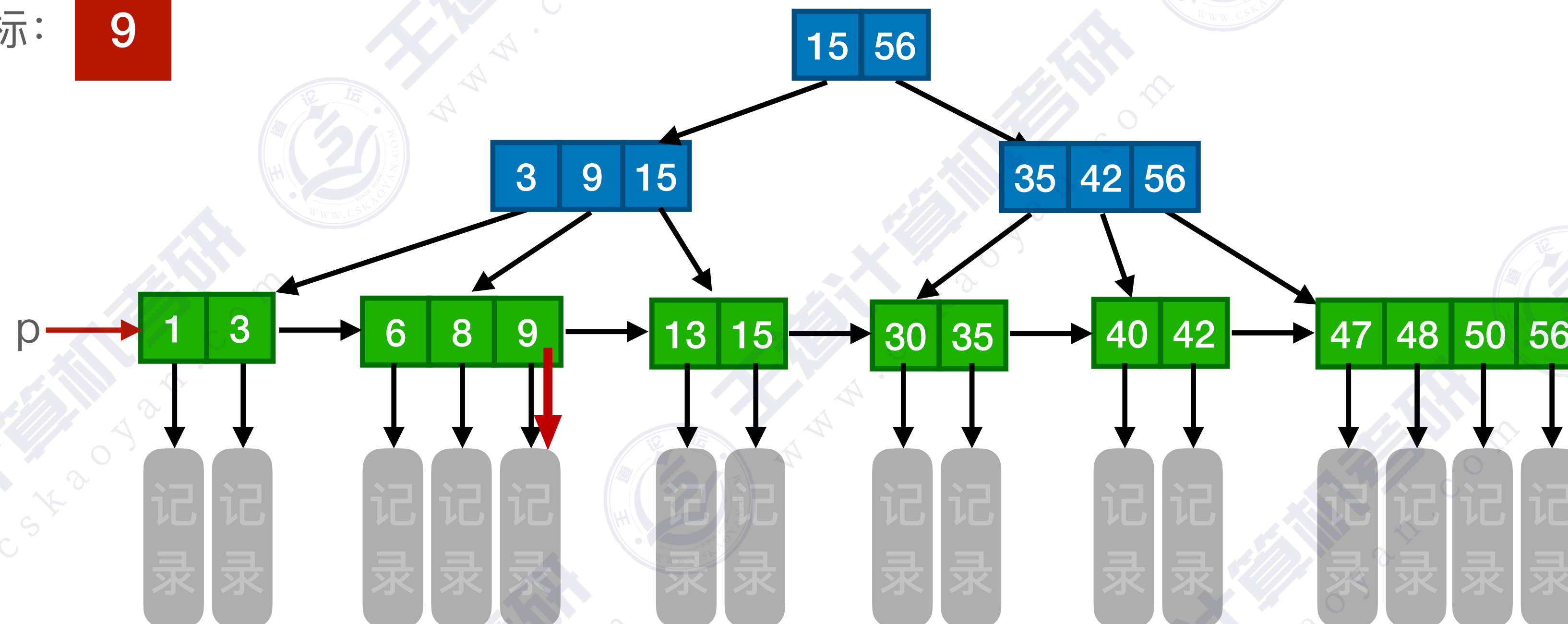


多路查找

B+树的查找

查找目标:

9

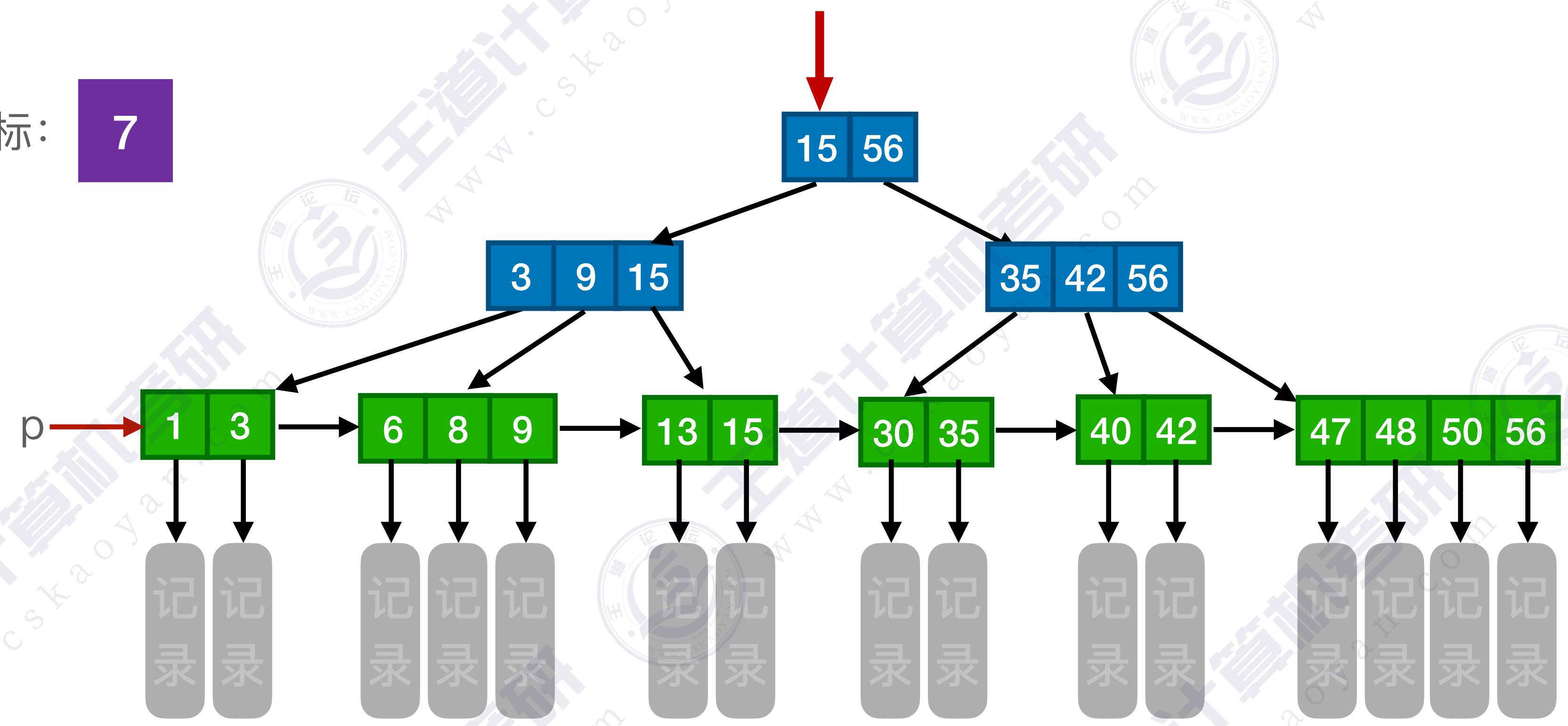


多路查找

B+树的查找

查找目标:

7



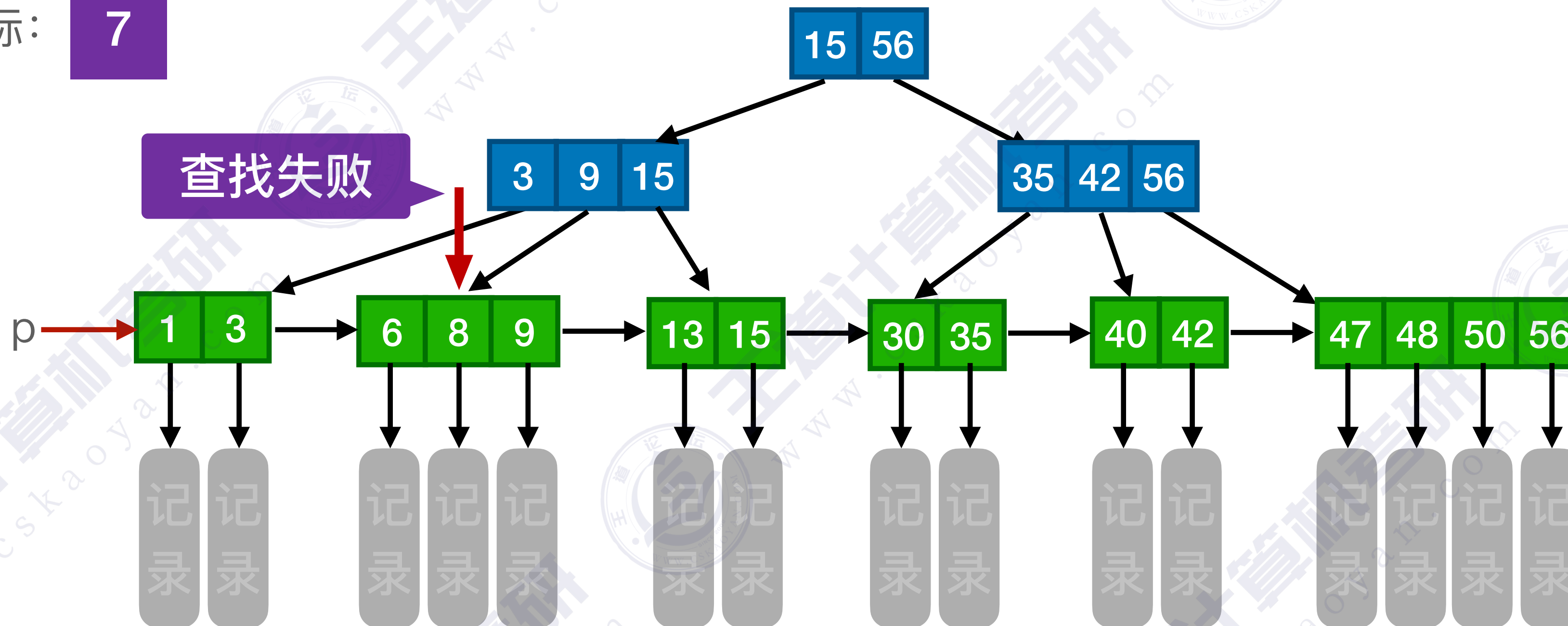
多路查找

B+树的查找

查找目标:

7

查找失败



多路查找

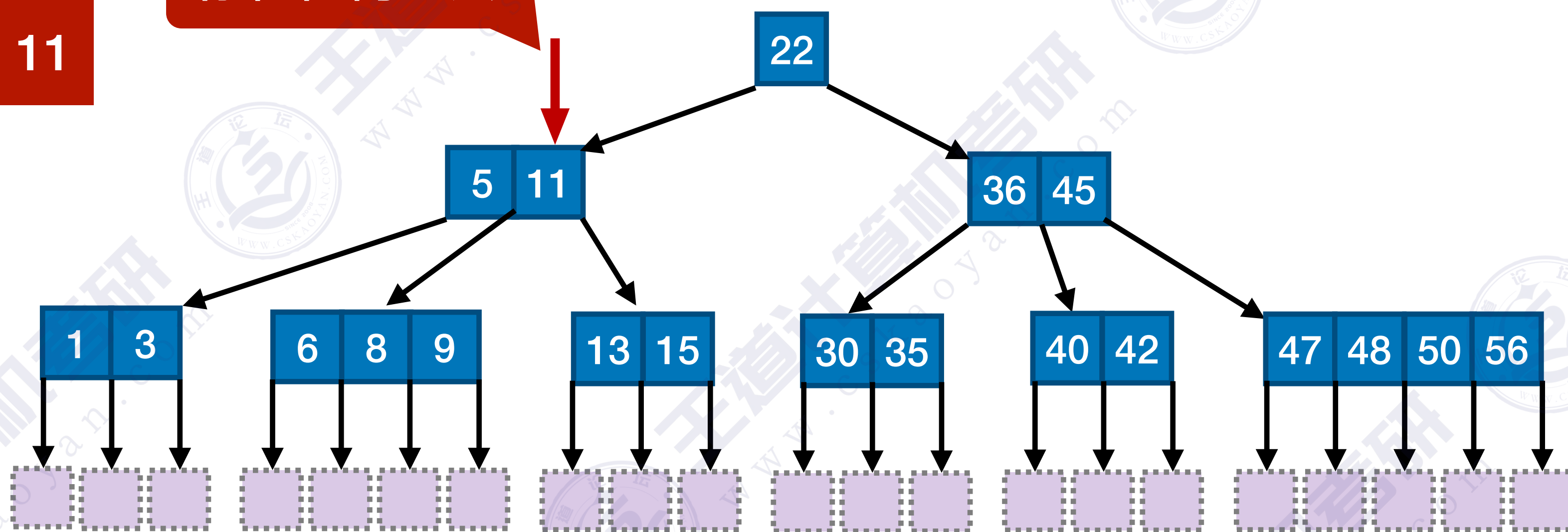
B+树中，无论查找成功与否，最终一定都要走到最下面一层结点

对比：B树的查找

查找目标：

11

查找成功，可能
停在任何一层

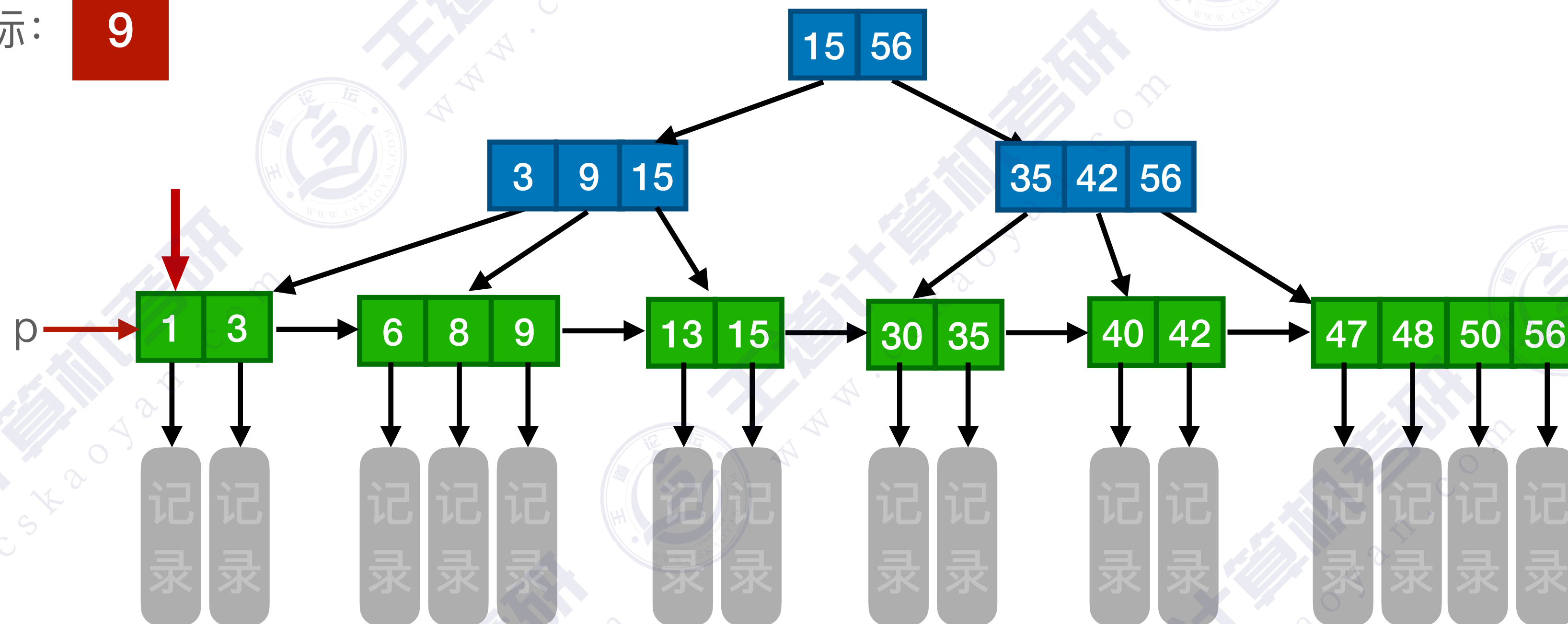


注：以上是一棵5阶B树

B+树的查找

查找目标:

9

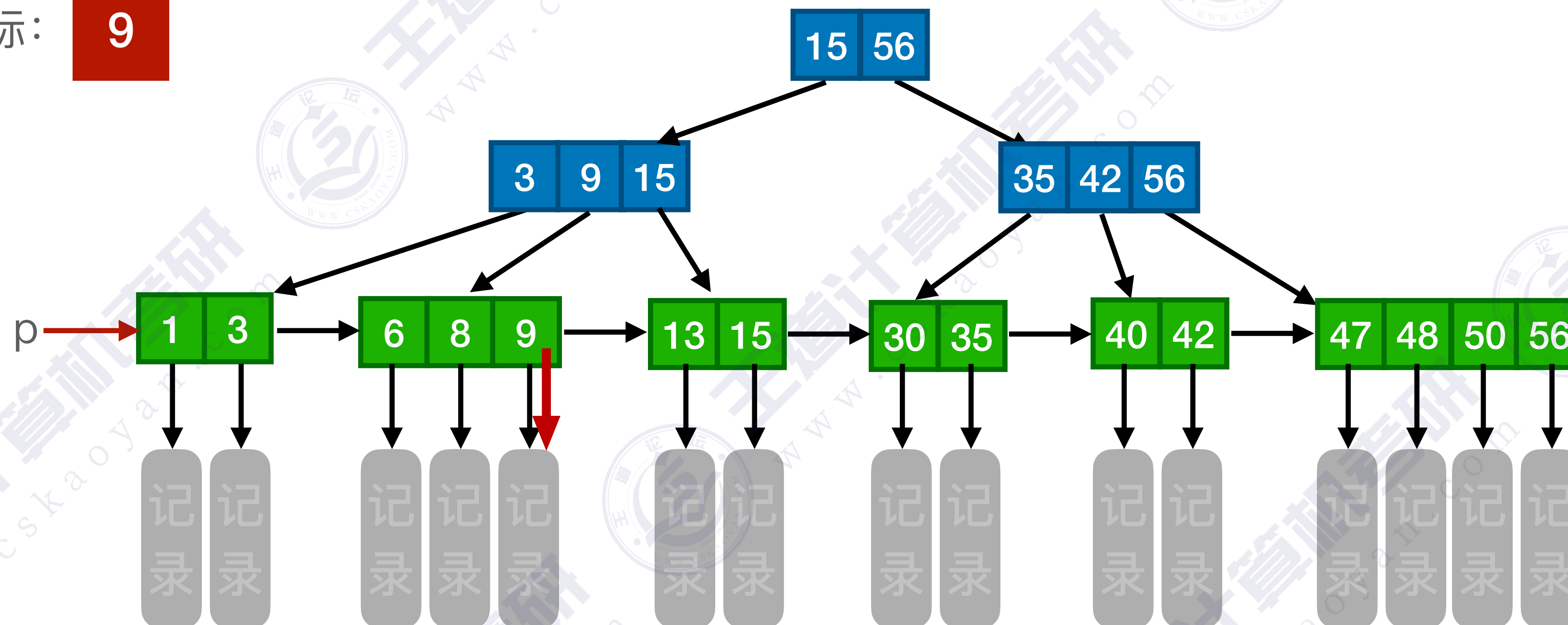


顺序查找

B+树的查找

查找目标:

9



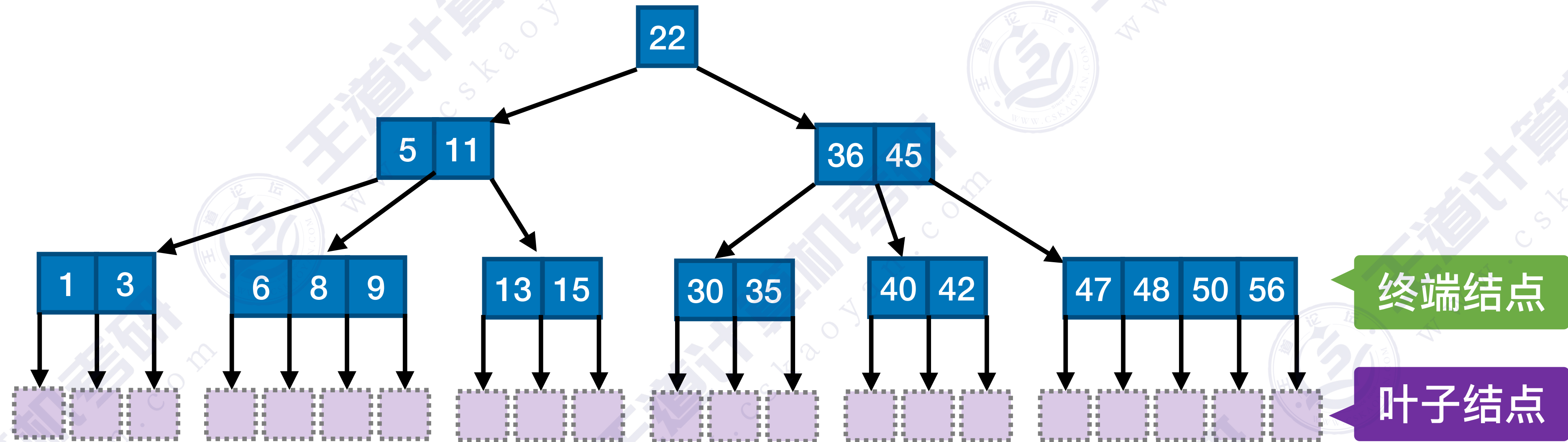
顺序查找



m阶B+树：

- 王道考研/CSKAOYAN.COM

B+树 VS B树



注：以上是一棵5阶B树

m阶B树：

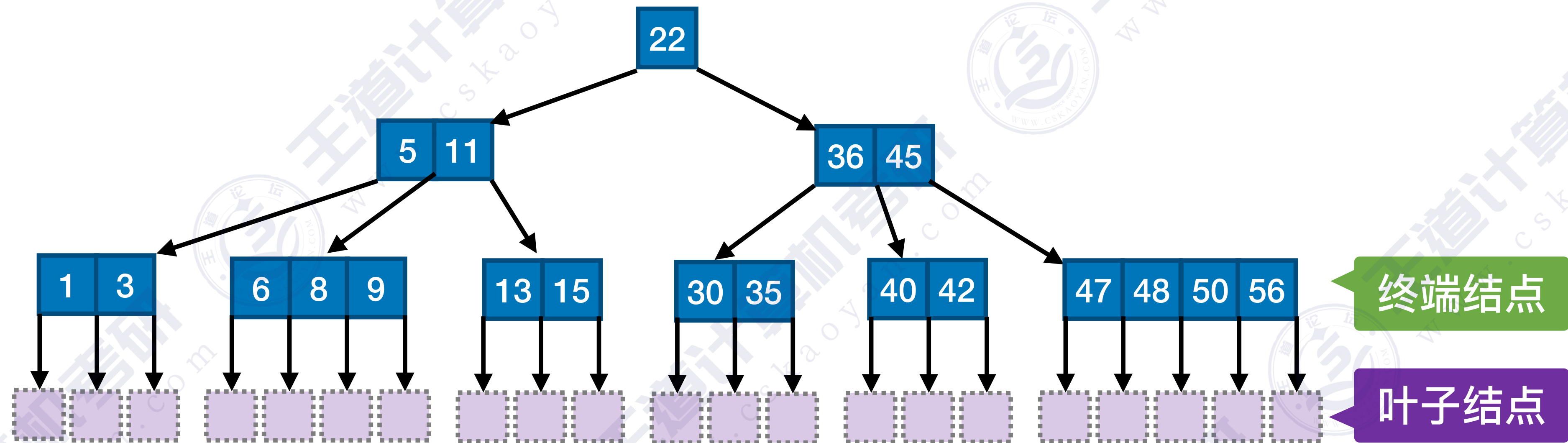
- 1) 结点中的n个关键字对应n+1棵子树



m阶B+树：

其他结点的关键字数 $n \in [\lceil m/2 \rceil, m]$

B+树 VS B树



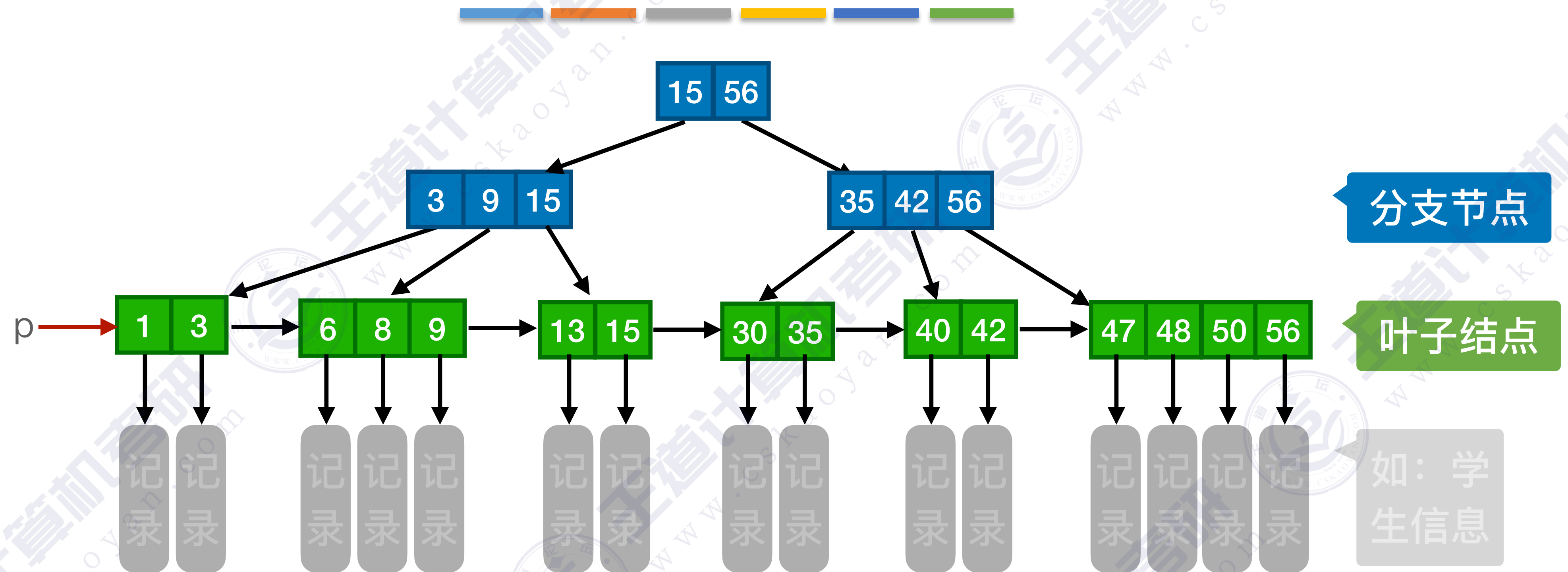
注：以上是一棵5阶B树

m阶B树：

2) 根节点的关键字数 $n \in [1, m-1]$ 。

其他节点的关键字数 $n \in [\lceil m/2 \rceil - 1, m-1]$

B+树 VS B树

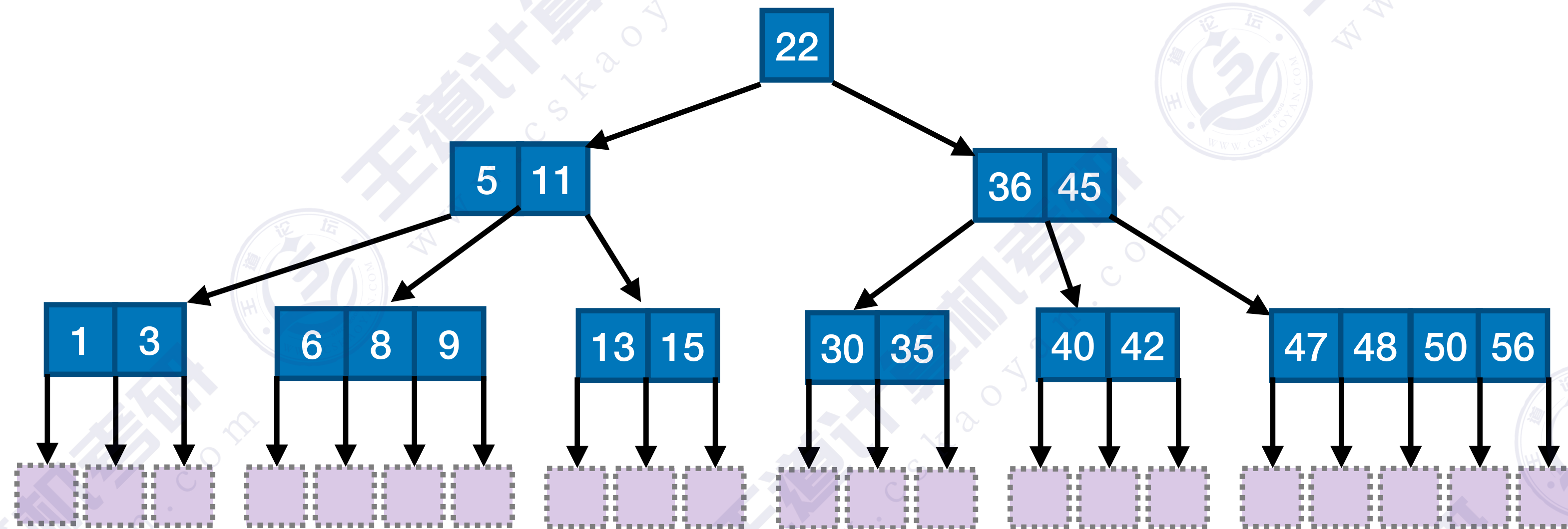


注: 以上是一棵4阶B+树

m阶B+树:

3) 在B+树中, 叶结点包含全部关键字, 非叶结点中出现过的关键字也会出现在叶结点中

B+树 VS B树

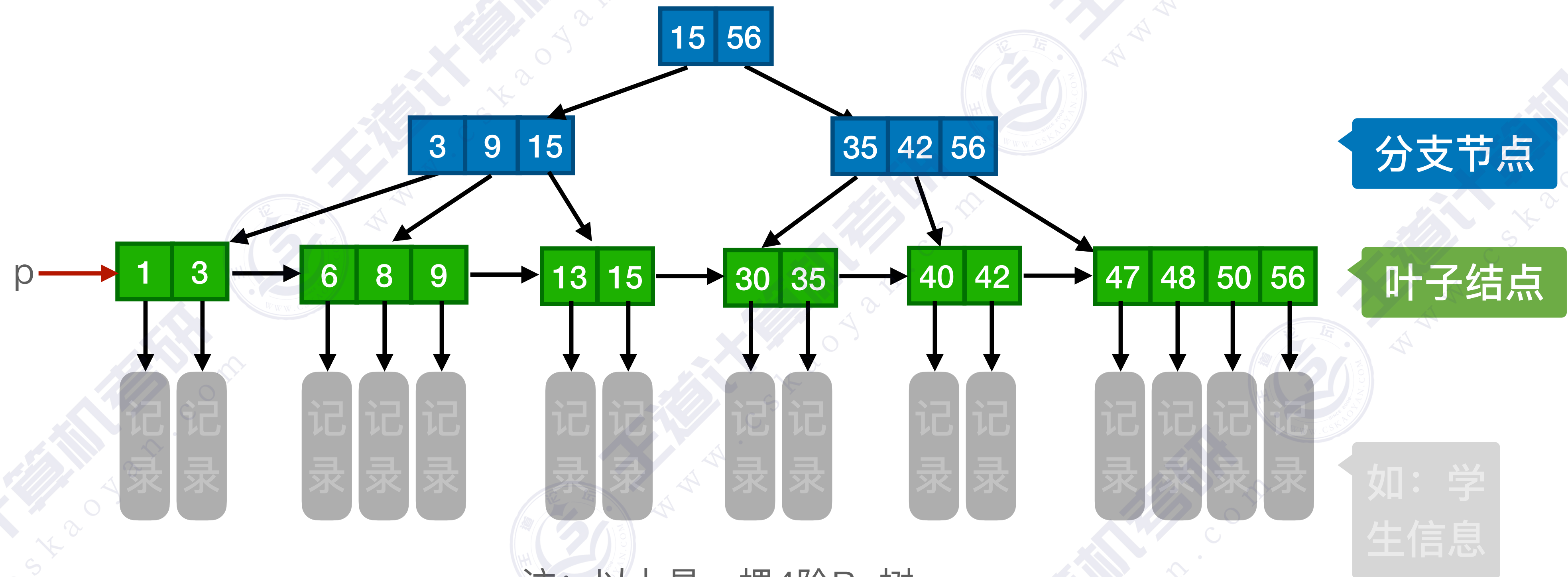


注：以上是一棵5阶B树

m阶B树：

3) 在B树中，各结点中包含的关键字是不重复的

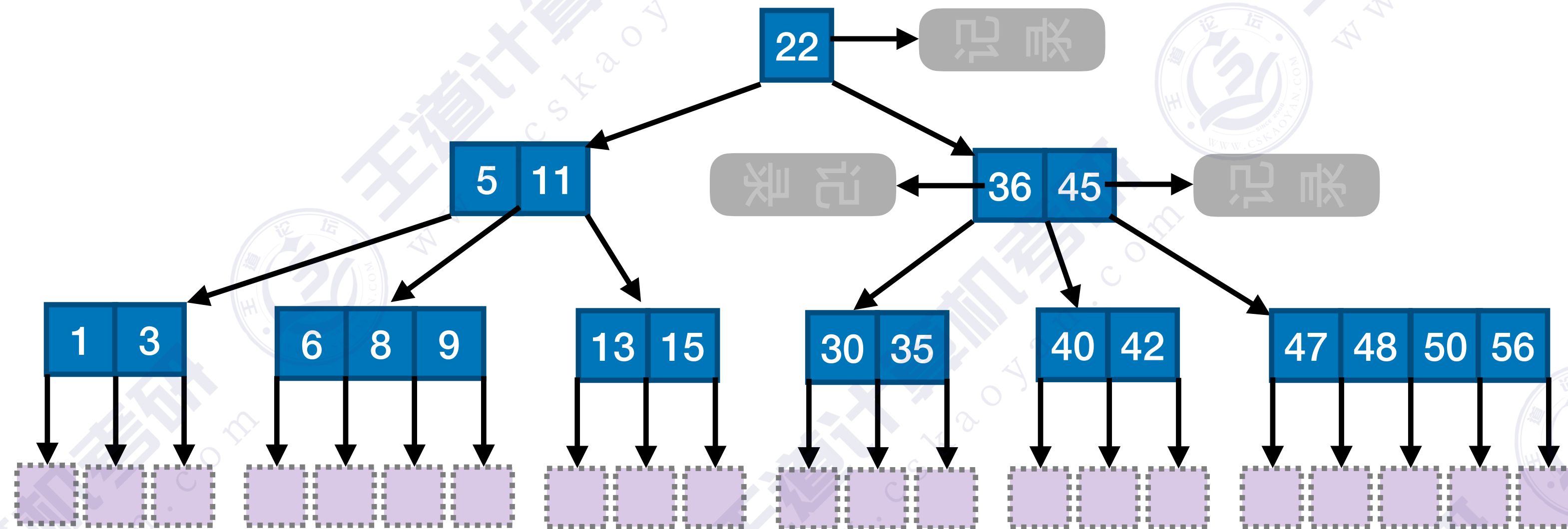
B+树 VS B树



m阶B+树：

4) 在B+树中，叶结点包含信息，所有非叶结点仅起索引作用，非叶结点中的每个索引项只含有对应子树的最大关键字和指向该子树的指针，不含有该关键字对应记录的存储地址。

B+树 VS B树

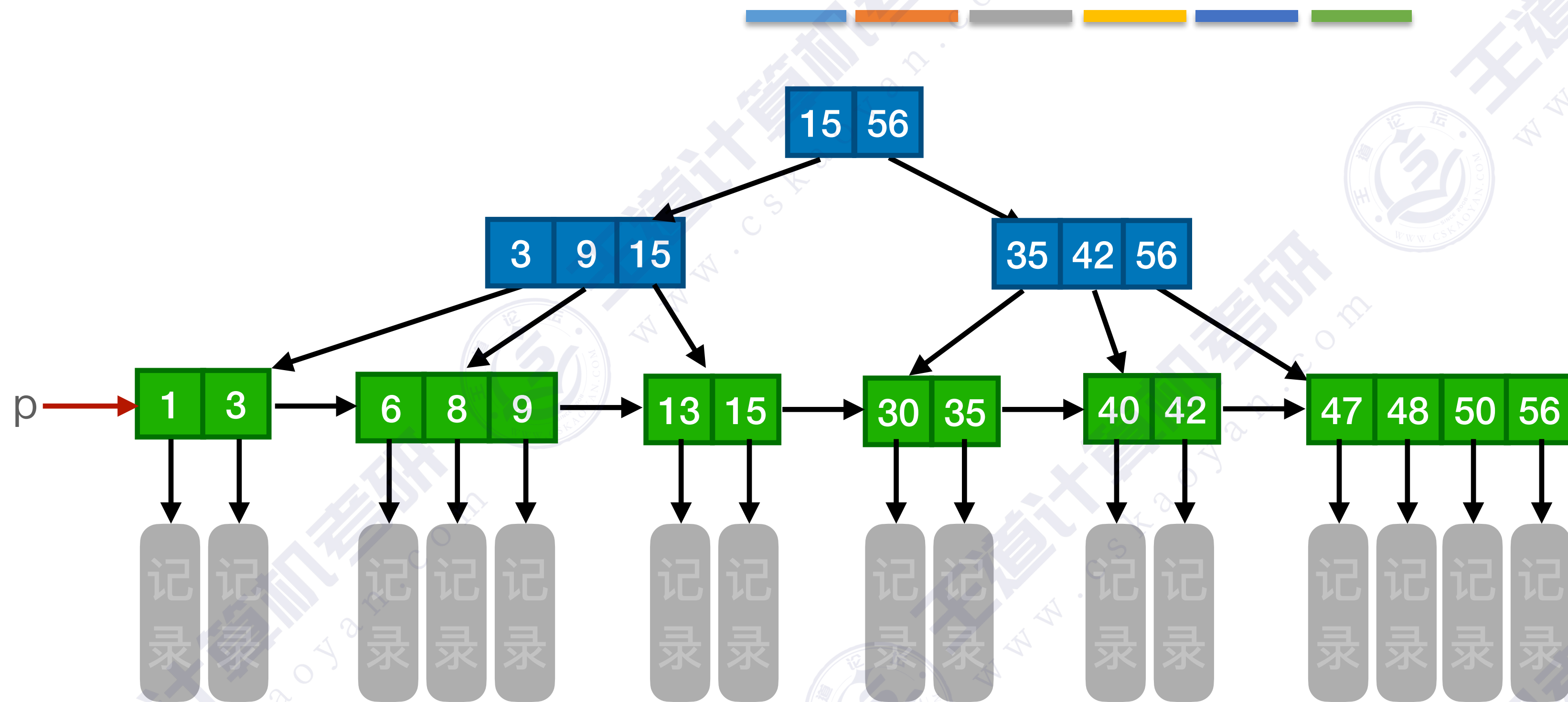


注：以上是一棵5阶B树

m阶B树：

4) B树的结点中都包含了关键字对应的记录的存储地址

B+树 VS B树



典型应用：关系型数据库的“索引”（如MySQL）

在B+树中，非叶结点不含有该关键字对应记录的存储地址。
可以使一个磁盘块可以包含更多个关键字，使得B+树的阶更大，树高更矮，
读磁盘次数更少，查找更快



WHY?

知识回顾与重要考点



	m阶B树	m阶B+树
类比	二叉查找树的进化——>m叉查找树	分块查找的进化——>多级分块查找
关键字与分叉	n个关键字对应n+1个分叉（子树）	n个关键字对应n个分叉
结点包含的信息	所有结点中都包含记录的信息	只有最下层叶子结点才包含记录的信息 (可使树更矮)
查找方式	不支持顺序查找。查找成功时，可能停在 任何一层结点，查找速度“不稳定”	支持顺序查找。查找成功或失败都会到达 最下一层结点，查找速度“稳定”
相同点	除根节点外，最少 $\lceil m/2 \rceil$ 个分叉（确保结点不要太“空”） 任何一个结点的子树都要一样高（确保“绝对平衡”）	